Multi-Cloud MERN Platform

Project report — HeroVired DevOps Capstone, Problem Statement #5: Multi-Cloud Deployment and Management with Terraform

Rishav Mukhopadhyay · Shekh Md Shuaib · Thupesh Kumar · Jilan Basha Shaikh

31 October 2026

Table of Contents

Summary 4
Results at a glance (final test, 30–31 Oct 2026) 4
Technology in one table 5
01 — Project overview 6
The problem 6
What we built 6
Goals → how they are met → proof 6
Design decisions (and why) 7
02 — Architecture 9
1. A request, end to end 9
2. The three clouds 9
3. Terraform layout — what lives how long 10
4. Inside each cluster 11
5. The stopped-standby model 11
6. Observability topology 12
7. Repositories 12
03 — Security 13
1. Identities — no keys anywhere 13
2. Secrets 15
3. Network doors 15
4. Supply chain 16
5. Least-privilege checks 17
04 — Setup guide: run it yourself 18
Option A — the shop on your laptop 18
Option B — the full multi-cloud platform in your accounts 18
05 — Operations runbook 22
Quick reference 22
1. Start a session 22
2. End a session 23
3. Deploy, roll back 23
4. Standbys 23
5. Drills 24
6. Your public IP changed mid-session 24
7. Remove a leftover AWS load balancer by hand 24
8. Rotate a secret 24
06 — CI/CD: from a pull request to three clouds 26
1. Continuous integration (GitHub Actions) 28
2. The hand-off to Jenkins 30
3. Continuous delivery (Jenkins in GKE) 30

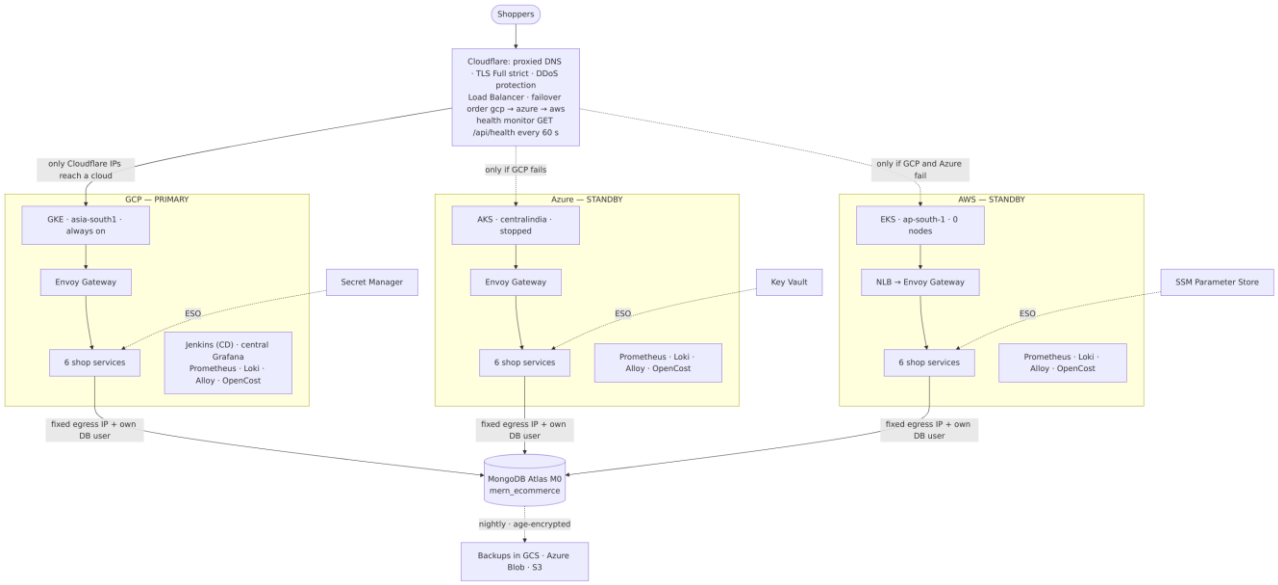
4. Why a deploy causes no downtime 32
5. Failure and edge cases 33
6. Where the pieces live 33
7. Manual operations 33
07 — Monitoring, alerts and cost 34
1. What runs where 34
2. Dashboards 34
3. Alerts → Slack (`#mern-alerts`) 35
4. Cost 36
08 — Disaster recovery and failover 41
1. What can fail, and what happens 41
2. Failover: the stopped-standby model 41
3. Backup and restore 42
4. Runbooks 43
5. Drill results (final test, 30 Oct 2026) 43
09 — Final end-to-end test report 45
Summary 45
A. Permanent foundations 45
B. Session start (`up.yml`) 55
C. Everything running 56
D. CI → CD: one merge, three clouds 69
E. Monitoring, alerts and cost 73
F. Standbys asleep 81
G. Disaster recovery 86
H. Teardown 90
10 — Troubleshooting 92
Sessions (`up.yml` / `down.yml`) and WSL 92
CI/CD 93
Data and security 93
11 — Limitations and possible improvements 94
Known limitations 94
Possible improvements 94

Summary

HeroVired DevOps Capstone · Problem Statement #5: Multi-Cloud Deployment and Management with Terraform GitHub organisation: `ppmcad16a-cloud-platform` · Live domain (during sessions): `https://rishavmukhopadhyay.store`

A polyglot MERN e-commerce shop, split into six microservices, runs on **Google Cloud (GKE), Microsoft Azure (AKS) and Amazon Web Services (EKS)**. Everything is created by Terraform, delivered by GitHub Actions + Jenkins, protected by keyless identities, watched by Prometheus/Grafana/Loki, backed up to all three clouds, and switched on and off with one Ansible command each.

Downloads: Project report (Word) · Presentation (PowerPoint, ~12 min)



Architecture

Results at a glance (final test, 30–31 Oct 2026)

What we proved	Result	Evidence
Whole platform from code in parallel (3 clusters + add-ons + shop)	`up.yml` green on all 3 clouds in **23 min 13 s** (warm GKE/AKS, cold EKS)	Test report §B
Same app in 3 clouds, one shared database	6/6 services ready in every cloud, `/api/health` = ok ×3	§C
CI → CD with one merge	PR → scan → **same image in 3 registries** → Jenkins → live in gcp, azure, aws in **4 min 57 s**	§D
Automatic cloud failover	GCP down → standbys woken **automatically** → shop back, **RTO 3 min 42 s** (target 20 min)	§G
Disaster recovery	Encrypted backup in GCS + S3 + Azure Blob; restore drill **passed from all 3**	§G
Monitoring & alerting	Central Grafana over 3 clouds; MernServiceDown + Cloudflare pool alerts in Slack, then RESOLVED	§E
Cost control	Real bill Oct 1–30: **AWS $6.76 · Azure ₹349.61 · GCP ₹474.75 (credits) ≈ US $17 for the month**	Monitoring & cost

Technology in one table

Area	Technology
Application	React (client), Node.js (api-gateway, catalog, notification, seed-db), Java 21 Spring Boot (identity), Python Flask + gunicorn (commerce)
Database	MongoDB Atlas M0 (AWS Mumbai), database `mern_ecommerce`, one DB user per cloud
Infrastructure as code	Terraform 1.16.6 — per-cloud stacks (bootstrap / foundation / cluster / add-ons) + `global-platform`
Clouds	GKE `asia-south1-a` · AKS `centralindia` · EKS `ap-south-1`
Kubernetes add-ons	Envoy Gateway v1.9.2 (Gateway API), External Secrets 2.12.0, AWS LB Controller 3.6.0, kube-prometheus-stack 92.6.0, Loki 7.3.0, Alloy 1.13.1, OpenCost 2.5.32, Jenkins 5.9.68
Global traffic	Cloudflare DNS, TLS Full (strict) with Origin CA certs, Load Balancer with `/api/health` monitors, Zero Trust Access, a Worker for automatic failover
Secrets	GCP Secret Manager · Azure Key Vault · AWS SSM Parameter Store, synced by External Secrets with workload identity
CI	GitHub Actions reusable workflows (`core-cloud-platform@v2`), gitleaks, Trivy, keyless OIDC
CD	Jenkins in GKE (Configuration-as-Code, Job DSL, Generic Webhook Trigger), Helm 4
Automation	Ansible 2.21 in WSL Ubuntu: `toolchain`, `preflight`, `up`, `down`, `standby`, `wake`, `deploy`, drills
Observability	Prometheus + Alertmanager, Grafana (central), Loki + Alloy, OpenCost, Slack alerts
Disaster recovery	`mongodump` + `age` encryption → GCS, S3, Azure Blob; scripted restore drills

01 — Project overview

The problem

> *"In multi-cloud environments, managing resources across multiple cloud providers (AWS, Azure, GCP) can be complex and error-prone, especially when attempting to maintain consistent configurations and enable failover across clouds."* — Capstone Problem Statement #5

The task is a Terraform-based solution that deploys an application to **AWS, Azure and GCP**, keeps their configuration consistent, fails over automatically when a cloud breaks, recovers data after a disaster, and shows health and cost of everything in one place.

What we built

One real application — a **MERN e-commerce shop** split into six microservices in three languages — running as three identical copies, one per cloud, behind one domain name.

Building block	In one sentence
Three clusters	GKE (primary, always on), AKS and EKS (standbys that sleep like a stopped EC2 instance and wake in minutes).
One front door	Cloudflare sends every shopper to the first healthy cloud in the order GCP → Azure → AWS and checks each cloud's `/api/health` every 60 s.
One database	MongoDB Atlas; each cloud has its own database user and its own fixed outgoing IP on the allow-list.
Everything as code	Terraform for all infrastructure (13 stacks in 4 platform repos), Helm for the workloads, Ansible to run a session in one command.
Keyless security	No cloud key is stored anywhere. GitHub Actions, Jenkins and the pods prove who they are with short-lived OIDC / workload-identity tokens.
CI → CD	A merged pull request is tested, scanned, pushed to three registries and deployed cloud-by-cloud with automatic rollback.
Observability	Prometheus, Loki and OpenCost in each cluster; one central Grafana; alerts to Slack.
Disaster recovery	Nightly encrypted database backups in all three clouds, plus restore drills that prove they work.

Goals → how they are met → proof

Project goal (problem statement)	How it is met	Proof
1. Multi-cloud deployment with Terraform for high availability	Terraform stacks per cloud (`bootstrap`, `foundation`, `cluster`, `addons`) + `global-platform`; same Helm chart and same image in all clouds	Architecture, Test §C
2. Automate infrastructure and configuration with reusable modules	Per-cloud `modules/` (network, cluster), a versioned observability module (`v1.1.0`) used by all three clusters, reusable GitHub workflows (`@v2`), Ansible playbooks with guarded `terraform apply`	Setup guide, Runbook
3. Load balancing and failover across clouds	Cloudflare Load Balancer, 3 pools, `/api/health` monitor, failover order;	Failover, Test §G — RTO **3 min 42 s**

Project goal (problem statement)	How it is met	Proof
	stopped standbys woken automatically by Cloudflare → Worker → GitHub Actions	
4. Disaster recovery, replication across clouds, tested failover	`age`-encrypted `mongodump` stored in GCS, S3 and Azure Blob; restore drill restores each copy into a scratch DB and compares document counts	Test §G — **3/3 restores passed**
5. Monitoring, alerts and cost visibility	kube-prometheus-stack + Loki + OpenCost per cloud, central Grafana, 5 custom alerts + Cloudflare pool alerts → Slack, cloud budgets	Monitoring & cost, Test §E

Tools asked for vs. tools used

Asked for	Used	Note
Terraform (HCL)	Terraform 1.16.6	all infrastructure, including DNS, load balancer, database users and vault secrets
Python (optional)	Python	commerce service; Ansible is Python-based
AWS, Azure, GCP	all three	Mumbai / Central India regions
Vault for credentials	each cloud's own vault (Secret Manager, Key Vault, SSM Parameter Store) + External Secrets Operator	a cloud's secrets never leave that cloud; no static cloud keys exist at all (OIDC)
Prometheus + Grafana	kube-prometheus-stack in every cluster + central Grafana	plus Loki/Alloy for logs and OpenCost for cost
PagerDuty or Slack	Slack	Alertmanager, Cloudflare notifications, Jenkins
GitHub/GitLab CI/CD	GitHub Actions (CI, image publishing, failover wake-up) + Jenkins (CD)	Terraform itself is applied through guarded Ansible playbooks — see Limitations

Deliverables

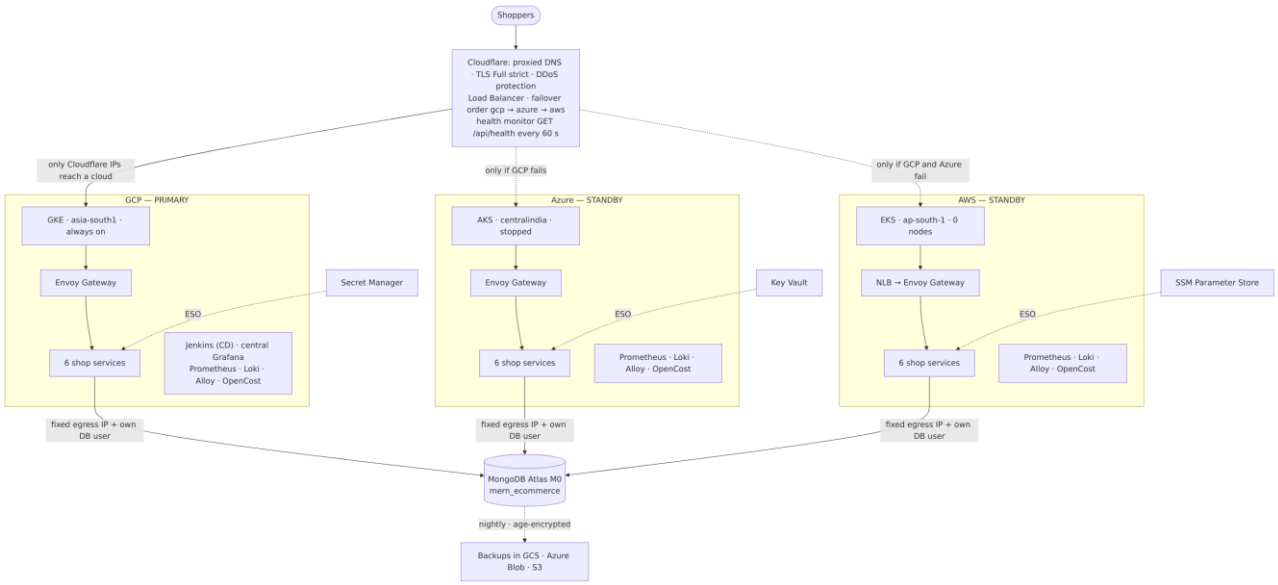
Deliverable	Where
Automated multi-cloud deployment system	repos `gcp-platform`, `azure-platform`, `aws-platform`, `global-platform`, `platform-deployments`, `platform-automation`
High availability and load balancing	Cloudflare LB in `global-platform/loadbalancer.tf`; failover wake-up in `global-platform/failover.tf` + `platform-automation/.github/workflows/failover-wake.yml`
Disaster recovery and data replication	`platform-deployments/pipelines/{backup,restore-drill}.Jenkinsfile`, `scripts/dr/`; chapter 08
Monitoring and cost dashboard	`observability-platform` (dashboards + alert rules), central Grafana; chapter 07
Comprehensive documentation	this handbook (setup, usage, DR, troubleshooting)

Design decisions (and why)

Decision	Why	Trade-off
Primary + stopped standbys instead of three always-on clouds	Standby nodes cost nothing while asleep (≈ US $212/month of node cost saved, see cost) but the configuration is complete, so waking is minutes, not a rebuild	A real outage costs a few minutes (measured RTO 3 min 42 s) instead of ~60 s

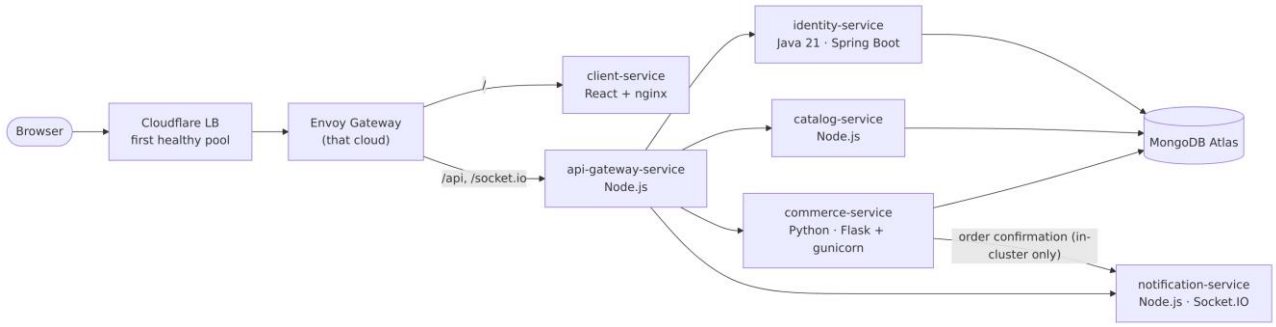
Decision	Why	Trade-off
Cloudflare in front instead of one cloud's DNS	A cloud-neutral front door cannot fail together with one of the clouds	One more vendor; Load Balancing is a paid add-on
One shared Atlas database	Same data in every cloud with zero sync logic; Atlas M0 is free	The database is a shared dependency — protected by 3-cloud backups, not live replication
Clusters exist only during working sessions	Pay only for hours used; the bill for October was ≈ US $17	Start-up takes ~25–40 minutes (parallel `up.yml`)
Jenkins for CD, GitHub Actions for CI	GitHub is good at build/test/publish per repo; Jenkins coordinates a cross-cloud rollout with gates and rollback	Jenkins lives in GKE, so a GCP outage needs the break-glass `deploy.yml`
Keyless everywhere	No secret to leak, rotate or forget	OIDC trust rules must be exact (repo id, branch)

02 — Architecture



Architecture

1. A request, end to end



Request flow

1. The browser resolves `rishavmukhopadhyay.store`. Cloudflare answers with its own edge address (the record is *proxied*), terminates TLS and forwards the request to the **first healthy pool** — GCP, then Azure, then AWS (`steering_policy = "off"` = failover order).
2. The request reaches that cloud's **Envoy Gateway** (Gateway API). Its load balancer accepts **only Cloudflare's IP ranges** plus the admin IP, so nobody can bypass Cloudflare.
3. Envoy routes `/` to `client-service` (React served by nginx) and `/api` + `/socket.io` to `api-gateway-service`.
4. The API gateway forwards to `identity`, `catalog`, `commerce` and `notification` over Kubernetes DNS. Services talk only inside their own cluster — there is **no VPN or network link between clouds**.
5. Services that store data connect to **MongoDB Atlas** through the cloud's NAT with a **fixed outgoing IP**; Atlas only accepts those IPs, and each cloud uses its own database user.

2. The three clouds

	GCP (primary)	Azure (standby)	AWS (standby)
Region	`asia-south1` (Mumbai), zone `-a`	`centralindia` (Pune)	`ap-south-1` (Mumbai), 2 AZs
Cluster	GKE Standard `prod-gke`, Kubernetes 1.36	AKS `prod-aks` (Free tier, Azure CNI Overlay, Entra RBAC), 1.36	EKS `prod-eks` (Pod Identity, EBS CSI, LB Controller), 1.37

	GCP (primary)	Azure (standby)	AWS (standby)
Nodes	2 × `e2-standard-2` (autoscale 2–3)	2 × `Standard_D2as_v5`	2 × `t3.large` (0 while asleep, max 3)
Network	VPC `prod`, nodes `10.30.0.0/20`, pods `10.30.128.0/17`	VNet `10.20.0.0/16`	VPC `10.10.0.0/16`, private node subnets
Outgoing IP	Cloud NAT with `prod-egress` (**permanent**, 34.93.192.167)	NAT Gateway + static public IP	NAT Gateway + Elastic IP
Entrance	static IP `prod-ingress` → Envoy LB	static IP `pip-prod-ingress` → Envoy LB	NLB (LB Controller) → Envoy
Registry	Artifact Registry `mern`	ACR `acrmern0botwx` (+ mirrors of ESO/OpenCost images)	ECR, one repo per service
Vault	Secret Manager	Key Vault `kv-mern-0botwx`	SSM Parameter Store `/mern/*`
Backups	GCS `…-mongo-backups`	Blob `stbackup0botwx/mongo-backups`	S3 `ppmcad16a-mongo-backups-…`
Extra role	Jenkins (CD), central Grafana	—	—

All three use the same machine shape (2 vCPU, 8 GB) so the same Helm values fit everywhere. Azure's free-trial limit of 4 vCPU per region is respected.

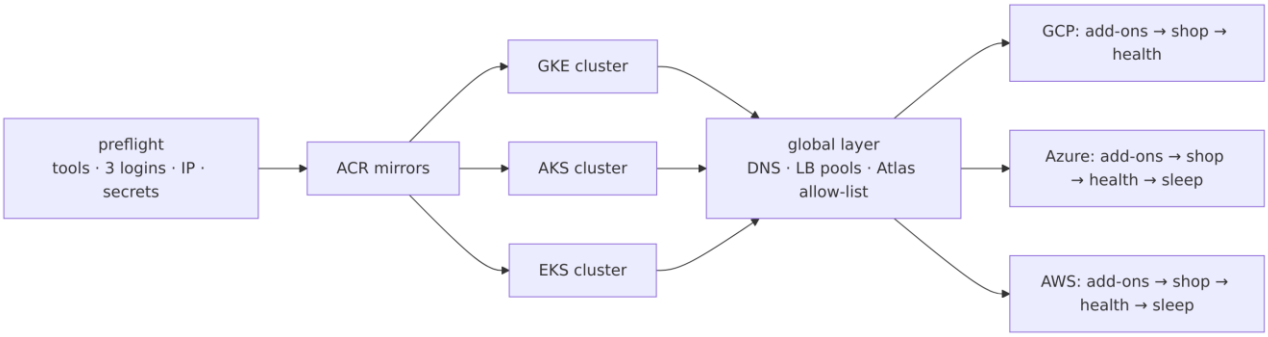
3. Terraform layout — what lives how long

Every cloud repo has the same four layers. Each layer is a separate Terraform stack with its own remote state (GCS / Azure Storage / S3, versioned).

Layer	Contents	Lifetime
`bootstrap/`	state bucket, GitHub OIDC trust + plan/apply identities, monthly budget	created once, by hand
`environments/prod/foundation/`	registry, vault, backup bucket, CI push identities, DR identities, failover identity, GKE egress IP	**permanent** — survives between sessions
`environments/prod/cluster/`	network, NAT, Kubernetes cluster, node pool, ingress IP	**per session** — created by `up.yml`, destroyed by `down.yml`
`environments/prod/addons/`	Envoy Gateway, External Secrets, monitoring stack, Jenkins (GCP), LB Controller (AWS), platform config chart	**per session**
`global-platform` (own repo)	Cloudflare DNS / LB / pools / monitors / Access / Worker / notifications, Atlas users + allow-list, app secrets written into all 3 vaults	**per session** (re-created by `up.yml`)

Stacks read each other's outputs through `terraform_remote_state` (for example, AKS and EKS read GKE's egress IP from GCP's *foundation* state so they can let Jenkins reach their APIs, and `global-platform` reads every cluster's ingress and egress IPs).

Because the GKE egress IP lives in the permanent foundation layer, the **three clusters no longer depend on each other and are created in parallel**:



Session start

4. Inside each cluster

One reusable Helm chart (`platform-deployments/charts/microservice`) deploys every service with:

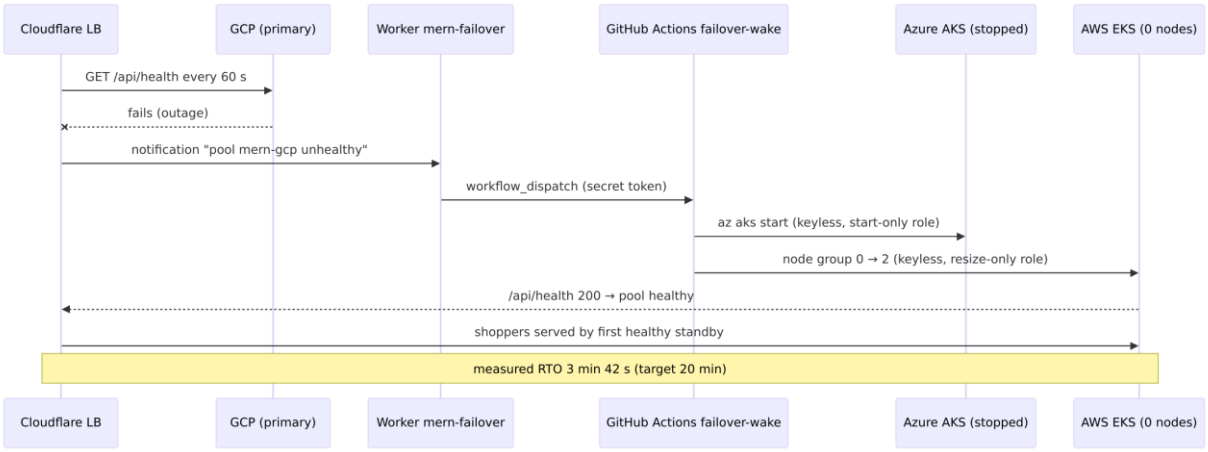
Feature	Setting	Why
Rolling update	`maxUnavailable: 0`, `maxSurge: 1`	a new pod must be Ready before an old one stops — zero downtime
Probes	startup + liveness (`/health`) + readiness (`/ready`, checks MongoDB)	traffic only reaches pods that can really serve
Graceful stop	`preStop` drain + 30 s grace period	in-flight requests finish during a rollout
Autoscaling	HPA 2–4 replicas at 70 % CPU (notification: 1 replica — Socket.IO keeps chat state in memory)	absorbs load spikes
Disruption budget	PDB `minAvailable: 1`	node maintenance never removes the last copy
Hardening	non-root, read-only root filesystem, no privilege escalation	smaller blast radius
Network policy	ingress allowed only from the gateway / the services that need it	a compromised pod cannot reach everything
Secrets	`ExternalSecret` from the cloud vault (`cloud-vault` ClusterSecretStore)	no secret in git or in Helm values
Routing	`HTTPRoute` for `/`, `/api`, `/socket.io` on the shared Envoy Gateway	one public entrance per cloud
Metrics	`ServiceMonitor`	Prometheus scrapes `/metrics`

Per-service settings are in `values/services/<service>.yaml`; per-cloud differences (registry, hostnames, cloud label) in `values/clouds/<cloud>.yaml`.

5. The stopped-standby model

State	GCP	Azure	AWS
Normal	serving everyone	`az aks stop` — control plane and VMs off	node group `0` nodes (control plane stays)
GCP outage	down	woken: `az aks start`	woken: node group `0 → 2`
Waking time (measured)	—	≈ 6 min	≈ 3 min

Waking is done by (a) **automatically**: a Cloudflare notification “pool `mern-gcp` unhealthy” → Worker `mern-failover` → GitHub Actions `failover-wake.yml`, which logs in keylessly with identities that may *only* start AKS / resize the node group; or (b) **by hand**: `ansible-playbook playbooks/wake.yml -e cloud=azure`. A woken standby first catches up to the primary’s image versions (`sync-shop`), because Jenkins skips sleeping clouds.



Failover sequence

6. Observability topology

- Every cluster runs Prometheus + Alertmanager, Loki + Alloy and OpenCost, all installed from the **same tagged module** (`observability-platform//modules/mern-observability?ref=v1.1.0`), so rules and alerts are identical everywhere.
- The **central Grafana** in GKE queries GCP locally and Azure/AWS through `obs-azure.` / `obs-aws.` hostnames — read-only (`GET`) and protected by a Cloudflare Access service token.
- Each cloud's Alertmanager sends to Slack on its own, so alerts still arrive when GCP (and the central Grafana) is down.

7. Repositories

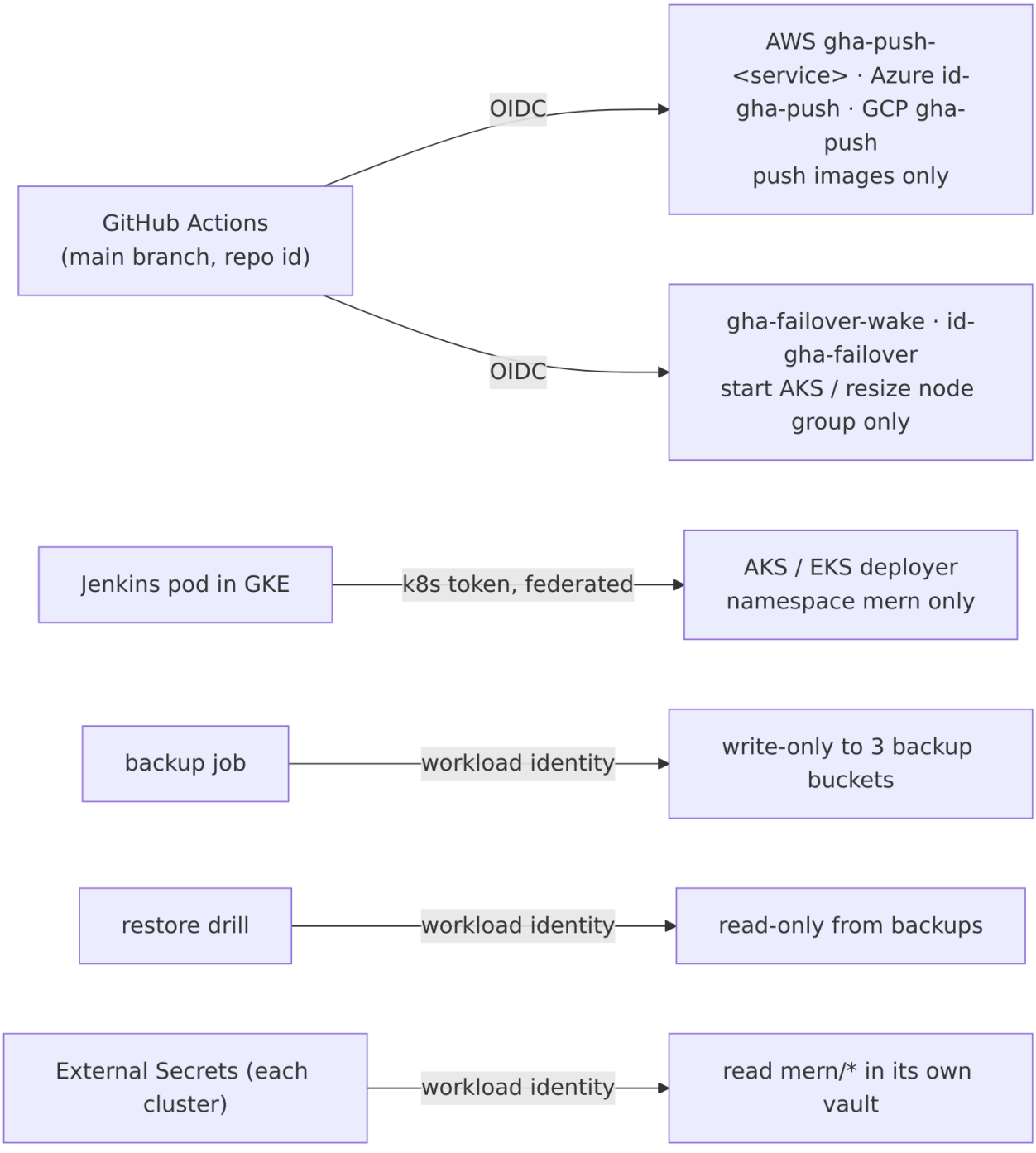
Repository	Purpose
`api-gateway-service`, `identity-service`, `catalog-service`, `commerce-service`, `notification-service`, `client-service`, `seed-db-service`	application code, tests (coverage gates), Dockerfile, a 30-line `main.yaml` that calls the shared CI
`core-cloud-platform`	reusable GitHub workflows (node/java/python, docker-build, trigger-deploy, security-scan) + learning guides
`gcp-platform`, `azure-platform`, `aws-platform`	Terraform per cloud (bootstrap, foundation, cluster, addons) and its modules
`global-platform`	Cloudflare, Atlas, shared secrets, failover Worker
`platform-deployments`	Helm chart, values, Jenkinsfiles (deploy, backup, restore-drill), DR scripts, local Docker Compose
`observability-platform`	versioned Terraform module: alert rules, recording rules, dashboards
`platform-automation`	Ansible playbooks + the `failover-wake` workflow
`.github`	organisation profile page

Every repository protects `main`: changes arrive through pull requests with required checks.

03 — Security

The platform follows three rules: **no stored credentials**, **every identity can do exactly one job**, and **every network door is closed unless it must be open**.

1. Identities — no keys anywhere

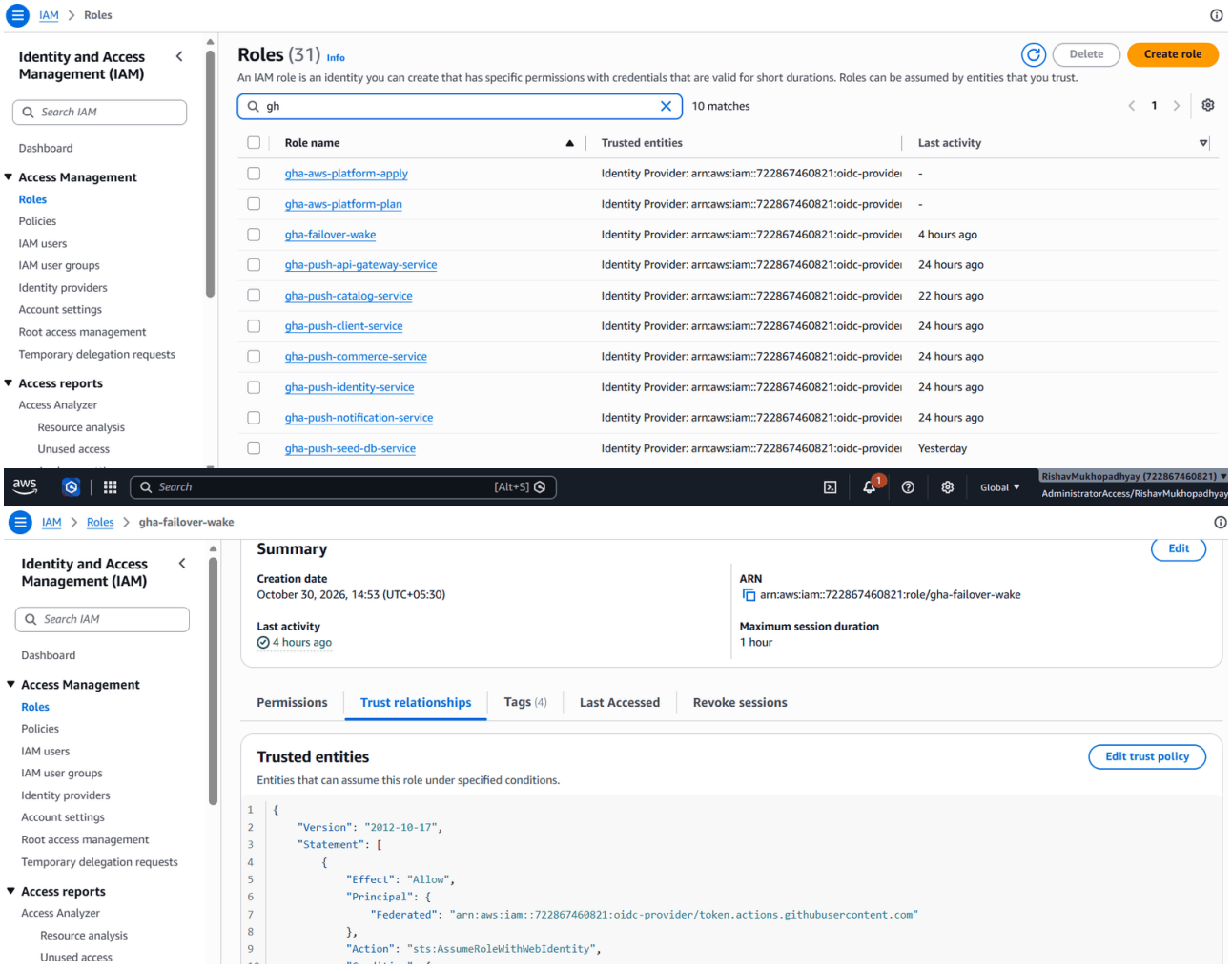


Identities

Who	How it proves who it is	What it may do — and nothing else
GitHub Actions in a service repo	GitHub OIDC token, trusted only for the **main branch of that repo**, matched by the repo's permanent numeric id	push images: AWS `gha-push-<service>` (its *own* ECR repo only), Azure `id-gha-push` (AcrPush), GCP `gha-push` (Artifact Registry writer +

Who	How it proves who it is	What it may do — and nothing else
		read 3 CI secrets)
GitHub Actions `failover-wake`	OIDC, `platform-automation` main branch	Azure `id-gha-failover`: custom role *read + start* the AKS cluster; AWS `gha-failover-wake`: *describe + resize* the node group
Jenkins (pod in GKE)	its Kubernetes service-account token, federated to Azure and AWS	deploy into namespace `mern` of AKS/EKS only; read AKS power state / EKS node group size (to skip sleeping standbys)
Backup job	workload identity	**write-only** to the three backup buckets (no read, no delete — even in Azure, via a custom role)
Restore drill	workload identity	**read-only** from the backup buckets; Atlas user `restore-drill` may only touch the scratch database `mern_restore_drill`
External Secrets (each cluster)	workload identity / Pod Identity	read `mern/*` secrets in **its own cloud's** vault only
Cluster nodes	node identity	pull from that cloud's registry only
You (admin)	SSO / `az login` / `gcloud auth login`	everything — used only from the laptop, never stored in CI

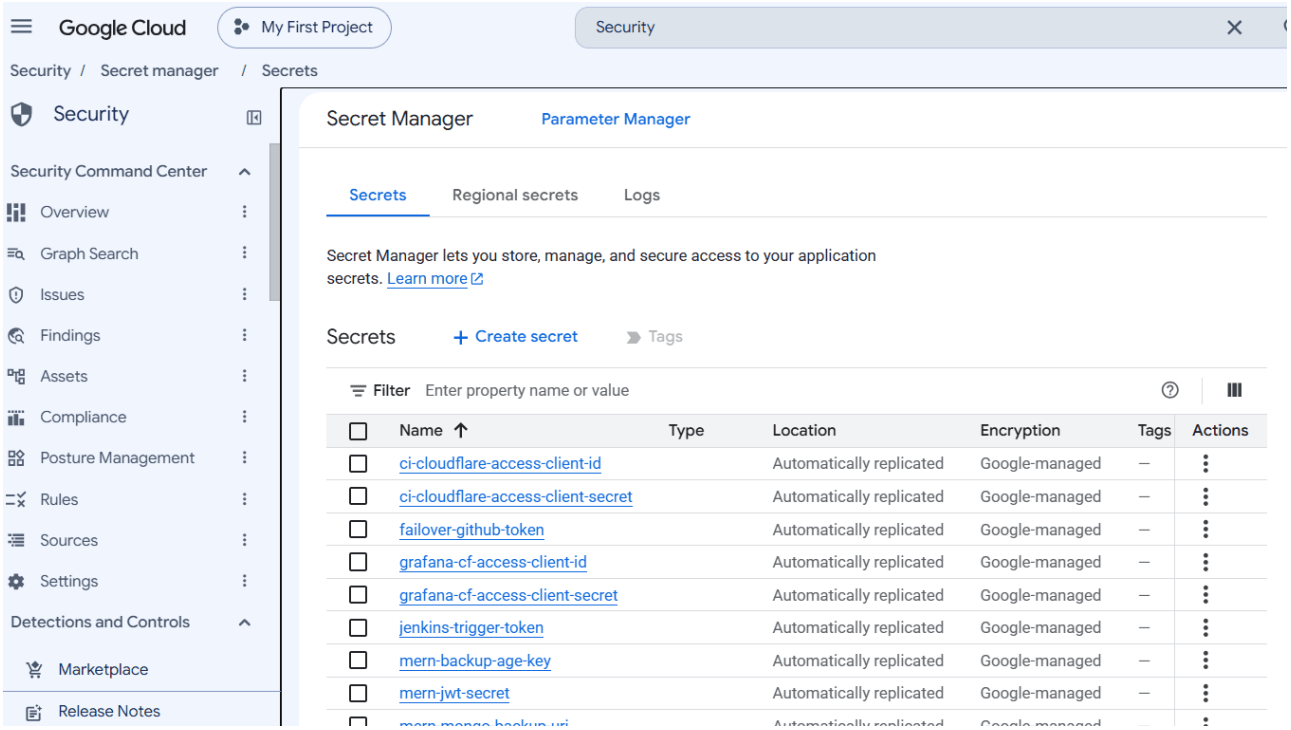
Immutable OIDC subject. The trust rules use GitHub's immutable subject format `repo:ppmcad16a-cloud-platform@308835253/<repo>@<repo-id>:ref:refs/heads/main`. A repository that is deleted and re-created under the same name gets a new id and does **not** inherit cloud access.



2. Secrets

Secret	Created by	Stored in	Read by
`mern-mongo-uri` (one DB user per cloud)	`global-platform` (Atlas provider)	each cloud's own vault	that cloud's services via External Secrets
`mern-jwt-secret` (identical in all clouds, ≥ 32 chars)	`global-platform` (`random_password`)	all three vaults	identity, gateway, notification — a login made in one cloud stays valid after failover
`mern-origin-tls-crt/-key`	Cloudflare Origin CA via Terraform	all three vaults	Envoy Gateway (TLS between Cloudflare and the cluster)
`mern-seed-admin-password`	`global-platform`	all three vaults	seed job only
`mern-slack-webhook-url`	by hand, once, in GCP Secret Manager; `global-platform` copies it	all three vaults + Cloudflare notification webhook	Alertmanager (each cloud), Jenkins, Cloudflare
CI tokens (Cloudflare Access service token, Jenkins trigger token)	`global-platform`	GCP Secret Manager	GitHub Actions (`gha-push` may read exactly these)
`failover-github-token`	by hand (fine-grained PAT, `actions:write` on one repo)	GCP Secret Manager → Worker secret binding	Cloudflare Worker
`mern-backup-age-key` (private key)	by hand, once	GCP Secret Manager, copied to Key Vault and SSM **outside** `/mern/`	restore drill only — no cluster can read it

Rules: no secret value is ever committed; `terraform.tfvars` files are git-ignored and contain only IP addresses and e-mails; global provider tokens (Cloudflare, Atlas) are typed into the session with hidden input and kept only in memory; Jenkins scripts use `set +x` before any command that handles a token.

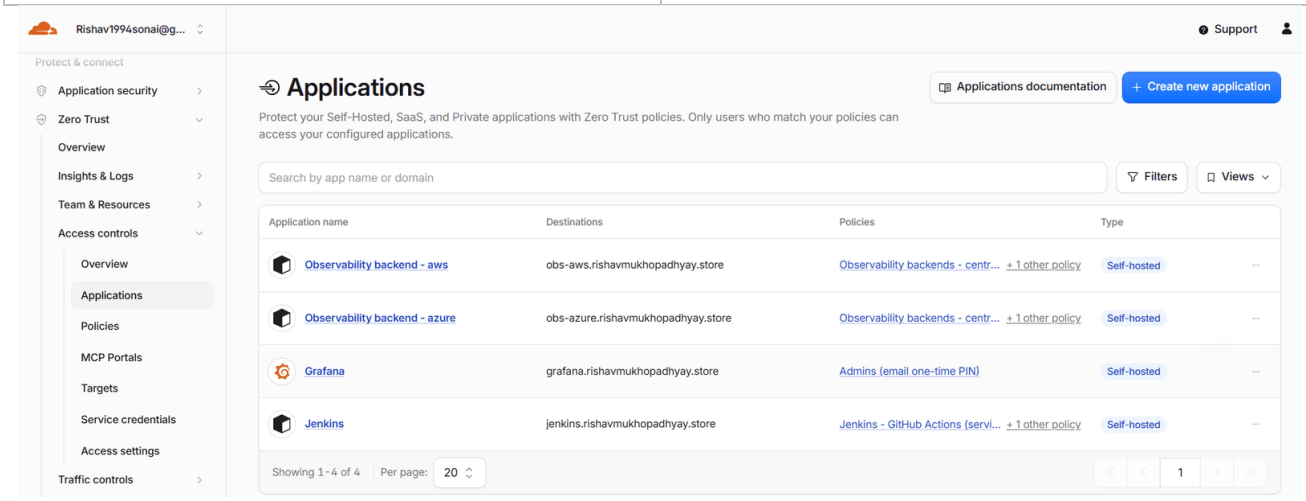


Secret Manager — names only

3. Network doors

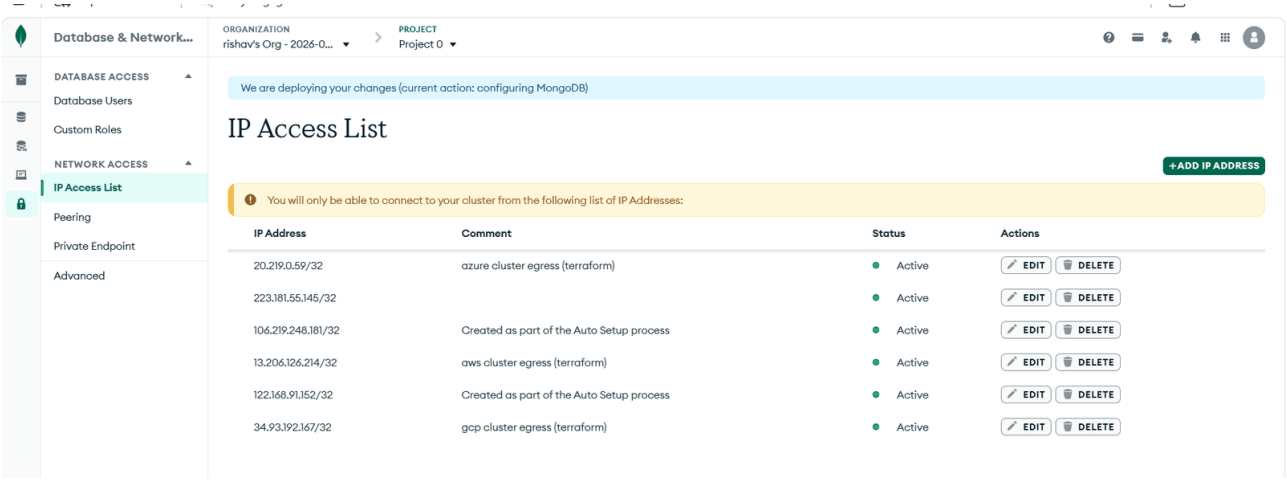
Door	Who may come in

Door	Who may come in
Cloudflare edge	the internet (HTTPS only, TLS 1.2+, *Always use HTTPS*)
Each cloud's entrance (Envoy LB)	**Cloudflare IP ranges** (read live from cloudflare.com) + the admin IP
Kubernetes API of each cluster	the admin IP + GKE's fixed egress IP (so Jenkins can deploy)
Pods	only the callers listed in their NetworkPolicy (gateway, specific services, Prometheus)
MongoDB Atlas	the three clusters' fixed egress IPs (managed by Terraform)
Jenkins, Grafana, `obs-*`	Cloudflare **Access**: admins by e-mail one-time PIN; GitHub Actions and Grafana by service token
TLS mode	**Full (strict)**: Cloudflare verifies the origin's certificate



Cloudflare Access applications

> **Finding from the final test:** the Atlas IP access list also contained a manually added `0.0.0.0/0` entry from early local development, which would have allowed connections from anywhere. It was removed on 31 Oct 2026; the list now holds only the Terraform-managed cluster IPs (plus three legacy personal IPs scheduled for removal). See Troubleshooting.



Atlas IP access list after the fix

4. Supply chain

Control	Where
Every third-party GitHub Action pinned to a **full commit SHA**	`core-cloud-platform/.github/workflows/*`
New tool / provider / chart versions adopted only when **≥ 14 days old**	team rule; Terraform lock files committed (incl. `linux_amd64` hashes)

Control	Where
Trivy downloaded from the official release and **checksum-verified**, then blocks fixable HIGH/CRITICAL vulnerabilities	`docker-build.yaml`
gitleaks scans the **whole git history** of every repo on every PR	`security-scan.yaml`
Images are tagged `main-<sha>` and **immutable** in all three registries	ECR / ACR / Artifact Registry settings
The **same digest** is pushed to all registries — what was scanned is what runs	`docker-build.yaml`
Workflows run with `permissions: contents: read` (+ `id-token: write` only where a cloud login is needed)	all workflows

5. Least-privilege checks

Check	How to see it
The backup identity can write a backup but cannot list or read one	tested with a throw-away pod (`AccessDenied` on list/read)
`gha-push-<service>` may push only to its own ECR repository	`aws-platform/environments/prod/foundation/ci-push.tf` — the policy's `Resource` is that one repository
The Jenkins deployer is limited to namespace `mern`	Kubernetes `Role`/`RoleBinding` in the `platform-config` chart (no `ClusterRole`)
The failover identity can start AKS but not stop, delete or change it	custom role "MERN failover - start AKS": `managedClusters/read` + `start/action` only
Jenkins/Grafana are not reachable without Cloudflare Access	an unauthenticated request receives the Access login page, not the application

04 — Setup guide: run it yourself

There are two ways to run this project:

Option	What you get	Needs	Time	Cost
A. Local	the whole shop (6 services + MongoDB) on your laptop	Docker Desktop	~30 min	free
B. Full platform	the multi-cloud platform in **your own** AWS, Azure and GCP accounts	3 cloud accounts, a domain on Cloudflare, Atlas, Slack, a GitHub org	1 day the first time, then ~30 min per session	≈ US $1 per working hour while all clusters run

Commands are marked **PowerShell** (Windows) or **Ubuntu** (WSL). Never mix them: a prompt starting with `PS C:\` is PowerShell, a prompt like `user@host:/mnt/c/...$` is Ubuntu.

Option A — the shop on your laptop

Prerequisites: Git, Docker Desktop (4 GB RAM free), JDK 21 (only to build the Java service).

```
# PowerShell — clone the services and the deployment repo side by side
mkdir mern; cd mern
foreach ($r in "api-gateway-service","identity-service","catalog-service","commerce-
service","notification-service","client-service","seed-db-service","platform-deployments") {
  git clone https://github.com/ppmcad16a-cloud-platform/$r.git
}

cd platform-deployments\local
copy .env.example .env
# edit .env: JWT_SECRET (at least 32 random characters), SEED_ADMIN_EMAIL, SEED_ADMIN_PASSWORD
python -c "import secrets; print(secrets.token_urlsafe(48))"     # a good JWT_SECRET

cd ..\..\identity-service; .\mvnw.cmd -B package -DskipTests; cd ..\platform-deployments\local
docker compose up --build -d                    # first build takes several minutes
docker compose ps                               # wait until every service is "healthy"
docker compose --profile seed run --rm seed     # once: admin user + 100 sample products
```

Open **http://localhost:8080**. Check the backend with `curl http://localhost:3000/api/health` — every service should say `"up"`. Stop with `docker compose down` (add `-v` to delete the local database). Details: `platform-deployments/local/README.md`.

Option B — the full multi-cloud platform in your accounts

B1. Accounts and services you need

Item	What exactly	Notes
AWS	an account on the paid plan (credits are fine), an SSO/admin user	the AWS free plan blocks EKS
Azure	a subscription with ≥ 4 vCPU quota in your region	free trials allow 4 vCPU per region — the design fits that
GCP	a project with billing linked	trial accounts cap at 8 vCPU
Domain on Cloudflare	a zone you own + the **Load Balancing** add-on	needed for the global front door and failover
Cloudflare API token	zone DNS/SSL/Load Balancing edit +	typed in at session start, never stored

Item	What exactly	Notes
	account Access, Workers, Notifications edit	
MongoDB Atlas	an M0 (free) cluster, an **organisation API key** with *Project Owner* on that project, your IP on the key's access list	typed in at session start
Slack	a channel + an *incoming webhook* URL	stored once in GCP Secret Manager
GitHub	an organisation where you are Owner	branch protection + OIDC trusts are per repo

B2. Your copy of the repositories

Create your organisation, then copy every repository into it (history included):

```
# PowerShell — needs the GitHub CLI (gh auth login first)
$org = "<your-org>"
$repos = "core-cloud-platform","api-gateway-service","identity-service","catalog-service","commerce-service","notification-service","client-service","seed-db-service","gcp-platform","azure-platform","aws-platform","global-platform","platform-deployments","observability-platform","platform-automation"
foreach ($r in $repos) {
  git clone --mirror https://github.com/ppmcad16a-cloud-platform/$r.git "$r.git"
  gh repo create "$org/$r" --public
  git -C "$r.git" push --mirror "https://github.com/$org/$r.git"
  git clone "https://github.com/$org/$r.git"
}
```

Then replace the project-specific values. Find every place with one search in the folder that holds all repos:

```
Get-ChildItem -Recurse -File -Exclude *.lock.hcl | Select-String -List -Pattern "722867460821|9c0602c2|0fd463ca|project-899ced78|692591977615|rishavmukhopadhyay|ppmcad16a|0botwx|011ab039|9b10c019|6a00a473|fnonccf|308835253|rishav1994sonai" | Select-Object Path -Unique
```

Value in this project	Meaning	Replace with
`722867460821`, `ap-south-1`	AWS account id, region	yours (`aws sts get-caller-identity`)
`9c0602c2-…`, `0fd463ca-…`, `centralindia`	Azure subscription id, tenant id, region	yours (`az account show`)
`project-899ced78-a554-40c2-96c`, `692591977615`, `asia-south1`	GCP project id, project **number**, region	yours (`gcloud projects describe <id>`)
`ppmcad16a-cloud-platform`, `308835253`	GitHub org name and org **id**	yours (`gh api orgs/<org> --jq .id`)
repo ids in `*/environments/prod/foundation/ci-push.tf` and `failover-identity.tf`	GitHub numeric repo ids used in OIDC trust rules	`gh api repos/<org>/<repo> --jq .id` for each repo
`rishavmukhopadhyay.store`	public domain	your Cloudflare zone
`011ab039…`, `9b10c019…`	Cloudflare zone id, account id	Cloudflare dashboard → zone overview
`6a00a473…`, `capstoneproject.fnonccf.mongodb.net`, `CapstoneProject`	Atlas project id, SRV host, cluster name	Atlas → Project settings / Connect
`0botwx`, `o398md`	random suffixes of Azure names	generated by Terraform — update the references after your foundation apply
`rishav1994sonai@gmail.com`	budget alert e-mail	yours
`AGE_RECIPIENT` in `platform-deployments/pipelines/backup.Jenkinsfile`	backup **public** key	yours (step B6)

B3. Workstation (Windows 11 + WSL Ubuntu 24.04)

```
# PowerShell (admin), once
wsl --install -d Ubuntu-24.04
winget install Git.Git GitHub.cli Docker.DockerDesktop

# Ubuntu — Ansible in its own Python environment, then every pinned tool with one playbook
sudo apt-get update && sudo apt-get install -y python3-venv wslu
python3 -m venv ~/.venvs/ansible && source ~/.venvs/ansible/bin/activate
pip install "ansible-core==2.21.5" "kubernetes==37.0.1" "python-debian==1.1.1"
cd /mnt/c/<path-to-your-repos>/platform-automation
ansible-galaxy collection install -r requirements.yml
ansible-playbook playbooks/toolchain.yml -K      # terraform, kubectl, helm, kubelogin, aws, az, gcloud (checksum-verified)
```

Edit `platform-automation/inventory/group_vars/all.yml`: `repos_root` (your repos folder as seen from WSL), `image_tags_file`, the account ids, `domain`, `acr_name`.

> **Network tip:** if Helm chart downloads time out after exactly 30 s, one of GitHub's download addresses may be unreachable from your ISP — test and pin a working one as described in Troubleshooting.

B4. Log in (Ubuntu)

```
gcloud auth login && gcloud auth application-default login
BROWSER=wslview az login --tenant <tenant-id> && az account set --subscription <subscription-id>
aws configure sso --profile HeroviredProfile          # first time; later: aws sso login --profile HeroviredProfile
```

B5. Bootstrap each cloud (once)

The bootstrap stack creates the bucket that will hold its own state, so it is applied with local state first and then moved:

```
# Ubuntu — repeat for gcp-platform, azure-platform, aws-platform
cd <repos>/gcp-platform/bootstrap
mv backend.tf backend.tf.off          # 1. no remote state yet
terraform init && terraform apply     #    creates state bucket, GitHub OIDC trust, plan/apply identities, budget
mv backend.tf.off backend.tf          # 2. now move the state into the new bucket
terraform init -migrate-state         #    answer "yes"
```

B6. One-time secrets (stored by hand, never in git)

```
# Ubuntu
# 1. Slack webhook -> GCP Secret Manager (global-platform copies it to the other two vaults)
printf '%s' 'https://hooks.slack.com/services/…' | gcloud secrets create mern-slack-webhook-url --data-file=-

# 2. Backup encryption key pair. The PRIVATE key goes only into the vault (and your password manager);
#    the PUBLIC key (age1…) goes into AGE_RECIPIENT in backup.Jenkinsfile.
sudo apt-get install -y age
age-keygen -o /tmp/age.key && grep 'public key' /tmp/age.key
gcloud secrets create mern-backup-age-key --data-file=/tmp/age.key && shred -u /tmp/age.key

# 3. Failover token: a fine-grained GitHub token with Actions: read & write on platform-automation only
read -rsp "PAT: " T; printf '%s' "$T" | gcloud secrets create failover-github-token --data-file=-; unset T
```

B7. Permanent foundations

```
# Ubuntu — repeat for gcp-platform, azure-platform, aws-platform
cd <repos>/gcp-platform/environments/prod/foundation
terraform init && terraform plan      # read it
terraform apply                       # registry, vault, backup bucket, CI/DR/failover identities (GCP: also the egress IP)
```

Create the git-ignored inputs:

```
MYIP=$(curl -s https://api.ipify.org)/32
for c in gcp azure aws; do for s in cluster addons; do
  echo "admin_cidr = \"$MYIP\"" > <repos>/$c-platform/environments/prod/$s/terraform.tfvars
done; done
cat > <repos>/global-platform/terraform.tfvars <<EOF
admin_emails      = ["you@example.com"]
workers_subdomain = "<your-workers-subdomain>"
EOF
```

B8. First images

One green run on `main` per service fills all three registries. The copy in B2 already started a run per repo, but it failed at the cloud login because the OIDC trusts did not exist yet — re-run it now:

```
# PowerShell — re-run the latest main-branch run of every service repo
foreach ($r in "api-gateway-service","identity-service","catalog-service","commerce-
service","notification-service","client-service","seed-db-service") {
  $id = gh run list -R "<your-org>/$r" --branch main -L 1 --json databaseId -q ".[0].databaseId"
  gh run rerun $id -R "<your-org>/$r"
}
```

Also turn on branch protection for `main` in every repo (pull request + required checks), as described in Repositories & shared CI.

Write the tags they produced (`main-<sha>`, visible in each run's summary) into the file named by `image_tags_file`:

```
{ "identity-service": "main-xxxxxxx", "catalog-service": "main-xxxxxxx", "commerce-service": "main-
xxxxxxx",
  "notification-service": "main-xxxxxxx", "api-gateway-service": "main-xxxxxxx", "client-service":
"main-xxxxxxx",
  "seed-db-service": "main-xxxxxxx" }
```

B9. First session

```
# Ubuntu, in platform-automation — type the two provider secrets once (hidden, memory only)
read -rsp "Cloudflare API token: " CLOUDFLARE_API_TOKEN; echo; export CLOUDFLARE_API_TOKEN
read -rsp "Atlas public key: " MONGODB_ATLAS_PUBLIC_KEY; echo; export MONGODB_ATLAS_PUBLIC_KEY
read -rsp "Atlas private key: " MONGODB_ATLAS_PRIVATE_KEY; echo; export MONGODB_ATLAS_PRIVATE_KEY

time ansible-playbook playbooks/up.yml -e keep_awake=true       # 25-40 min

# once: admin user + sample catalogue (the database is shared, so run it in one cloud only)
TAG=main-xxxxxxx     # the seed-db-service tag from your image-tags file
IMG=asia-south1-docker.pkg.dev/<gcp-project>/mern/seed-db-service:$TAG
sed "s#IMAGE_PLACEHOLDER#$IMG#" ../platform-deployments/jobs/seed-db.yaml | kubectl --context <gke-
context> apply -f -
kubectl --context <gke-context> -n mern wait --for=condition=complete job/seed-db --timeout=5m
```

Open `https://<your-domain>`, `https://jenkins.<your-domain>` and `https://grafana.<your-domain>` (Cloudflare Access e-mails you a PIN; passwords: `terraform output -raw jenkins_admin_password` / `grafana_admin_password` in `gcp-platform/environments/prod/addons`).

From now on every merged pull request deploys itself (CI/CD). At the end of a session run `ansible-playbook playbooks/down.yml` (Runbook).

B10. Checklist — is everything right?

Check	Expected
`for c in gcp azure aws; do curl -s https://$c.<domain>/api/health; done`	`"status":"ok"` three times
Cloudflare → Load Balancing	3 pools healthy (or gcp healthy + standbys critical after `standby.yml`)
Grafana → *MERN shop – multi-cloud overview*	one row per cloud, 6 services with ready pods
Jenkins	jobs `deploy-mern`, `backup-mongo`, `restore-drill`
`ansible-playbook playbooks/drill-restore.yml`	`Restore drill PASSED`

05 — Operations runbook

All commands run in **Ubuntu (WSL)** inside `platform-automation`, after:

```
source ~/.venvs/ansible/bin/activate
cd /mnt/c/Users/dell/Documents/HeroVired/Capstone/repos/platform-automation
aws sso login --profile HeroviredProfile      # AWS SSO sessions expire - renew at the start of every session
```

Every playbook that changes Terraform goes through `tasks/tf-apply.yml`: **plan → check the kind of each change → apply exactly that plan**. Session start may only *create/update*; session end may only *delete/update*. A plan that would replace or delete something unexpectedly stops the run before anything happens.

Quick reference

I want to…	Command	Time
start a session (standbys go to sleep at the end)	`ansible-playbook playbooks/up.yml`	25–40 min
start with all three clouds awake	`ansible-playbook playbooks/up.yml -e keep_awake=true`	25–40 min
put the standbys to sleep	`ansible-playbook playbooks/standby.yml` (or `-e cloud=azure`)	~5 min
wake one standby	`ansible-playbook playbooks/wake.yml -e cloud=aws`	3–10 min
deploy without Jenkins (break-glass)	`ansible-playbook playbooks/deploy.yml -e service=catalog-service -e tag=main-ba5cc42`	~3 min
prove the backups restore	`ansible-playbook playbooks/drill-restore.yml`	~10 min
failover drill (GCP fails)	`ansible-playbook playbooks/drill-failover.yml` (`-e wake_via=manual` to wake by playbook)	25–30 min
end the session — delete everything that costs per hour	`ansible-playbook playbooks/down.yml`	30–40 min

Tip: wrap long runs as `time ansible-playbook … 2>&1 | tee ~/run-$(date +%F).log` to keep a log and the duration.

1. Start a session

1. Make sure your public IP is in the six `terraform.tfvars` files (`admin_cidr`) — preflight checks it.
2. Type the provider secrets once (hidden, memory only):

   ```
   read -rsp "Cloudflare API token: " CLOUDFLARE_API_TOKEN; echo; export CLOUDFLARE_API_TOKEN
   read -rsp "Atlas public key: " MONGODB_ATLAS_PUBLIC_KEY; echo; export MONGODB_ATLAS_PUBLIC_KEY
   read -rsp "Atlas private key: " MONGODB_ATLAS_PRIVATE_KEY; echo; export MONGODB_ATLAS_PRIVATE_KEY
   ```

3. `ansible-playbook playbooks/up.yml`. Order: preflight → ACR mirrors → **GKE + AKS + EKS in parallel** → global layer → each cloud independently: add-ons → shop (6 services at once) → health check → standbys sleep.
4. Result: `https://<domain>` served by GCP; Azure stopped, AWS at 0 nodes.

The versions deployed come from `C:\Users\dell\image-tags.json`; keep it updated after each release (Jenkins records what it deployed in Slack).

2. End a session

```
ansible-playbook playbooks/down.yml
```

Order: preflight → wake sleeping standbys (so Kubernetes can remove its own load balancers and disks) → AKS + EKS: add-ons destroy → **state must be empty** → **leftover check** (no `k8s-*` load balancer, no orphaned disk) → cluster destroy → GKE the same → global layer (delete/update only).

If the leftover check stops the run, Kubernetes did not finish cleaning up. Wait 5 minutes and re-run, or remove the leftover by hand (section 7). **Never** continue with `-refresh=false`: it can wipe state while the real resources keep running.

Afterwards, prove nothing is billing:

```
gcloud container clusters list; gcloud compute instances list; gcloud compute forwarding-rules list
az aks list -o table
aws eks list-clusters --region ap-south-1 --profile HeroviredProfile
aws elbv2 describe-load-balancers --region ap-south-1 --profile HeroviredProfile --query
"LoadBalancers[].LoadBalancerName"
aws ec2 describe-nat-gateways --region ap-south-1 --profile HeroviredProfile --filter
Name=state,Values=available --query "NatGateways[].NatGatewayId"
```

Only the permanent foundation remains (registries, vaults, backup buckets, state buckets, GCP's `prod-egress` IP).

3. Deploy, roll back

Situation	What to do
normal release	merge the pull request — CI pushes the image and Jenkins deploys GCP → Azure → AWS (CI/CD)
roll back	Jenkins → `deploy-mern` → *Build with Parameters* → the **previous** `IMAGE_TAG`
deploy to some clouds only	same, with `TARGETS=gcp` (comma list)
Jenkins is down (e.g. GCP outage)	`ansible-playbook playbooks/deploy.yml -e service=<svc> -e tag=<tag>` — deploys to every **awake and reachable** cloud with the same rolling update + rollback; `-e target=azure` for one cloud
an image reached only some registries	the CI run is green with a warning; re-run the workflow when the cloud is back (it fills the gap and then deploys), or deploy now with `TARGETS=` the clouds that have it

A sleeping standby is skipped by Jenkins and caught up automatically when it wakes (`sync-shop` compares its versions with GCP's).

4. Standbys

Action	Command	What happens
sleep	`playbooks/standby.yml`	Azure: `az aks stop`. AWS: node group min/desired 0.
wake	`playbooks/wake.yml -e cloud=<azure|aws>`	start → nodes Ready → shop healthy → catch up to GCP's versions
automatic wake	nothing — Cloudflare notification → Worker → GitHub Actions `failover-wake.yml`	runs only when the **gcp** pool becomes unhealthy

Known behaviour: AWS sleep can take up to 30 minutes because the PodDisruptionBudgets (`minAvailable: 1`) refuse to evict the last copy of each service while the whole node group is going away. The playbook reports "asleep" as soon as the *setting* is 0. Finish the drain at once:

```
kubectl --context arn:aws:eks:ap-south-1:722867460821:cluster/prod-eks drain -l
eks.amazonaws.com/nodegroup=default \
  --ignore-daemonsets --delete-emptydir-data --disable-eviction --force --timeout=180s
```

Also wait until a woken AKS has finished starting (`az aks show … --query provisioningState` = `Succeeded`) before stopping it again — Azure refuses a stop during a start.

5. Drills

Drill	Command	Pass criteria
restore	`playbooks/drill-restore.yml`	Jenkins `restore-drill` runs for `gcs`, `s3`, `azure`; each downloads the newest backup, verifies its SHA-256, decrypts it, restores into `mern_restore_drill` and matches document counts → `Restore drill PASSED`
failover	`playbooks/drill-failover.yml`	standbys asleep → GCP's gateway scaled to 0 → shop fails → standbys woken (automatically or `-e wake_via=manual`) → shop answers again → GCP repaired → standbys back to sleep → RTO ≤ 20 min

6. Your public IP changed mid-session

Symptoms: `kubectl` times out (`dial tcp …:443: i/o timeout`), Atlas answers `IP_ADDRESS_NOT_ON_ACCESS_LIST`. Three places need the new IP:

1. The six `terraform.tfvars` (`admin_cidr`) — PowerShell:

   ```
   $R="C:\Users\dell\Documents\HeroVired\Capstone\repos"; $old="<old-ip>/32"; $new="<new-ip>/32"
   foreach ($c in "gcp","azure","aws") { foreach ($s in "cluster","addons") {
     $f="$R\$c-platform\environments\prod\$s\terraform.tfvars"
     [IO.File]::WriteAllText($f, (Get-Content $f -Raw).Replace($old,$new)) } }
   ```

2. Atlas → Organization → Access Manager → Applications → API Keys → your key → *Edit* → API access list → *Use current IP*.

3. Re-run `up.yml` (or, before a teardown, `terraform apply` in the three `cluster` stacks — each plan must show `1 to change` on the cluster's API allow-list).

After a network change also re-test WSL downloads (Troubleshooting).

7. Remove a leftover AWS load balancer by hand

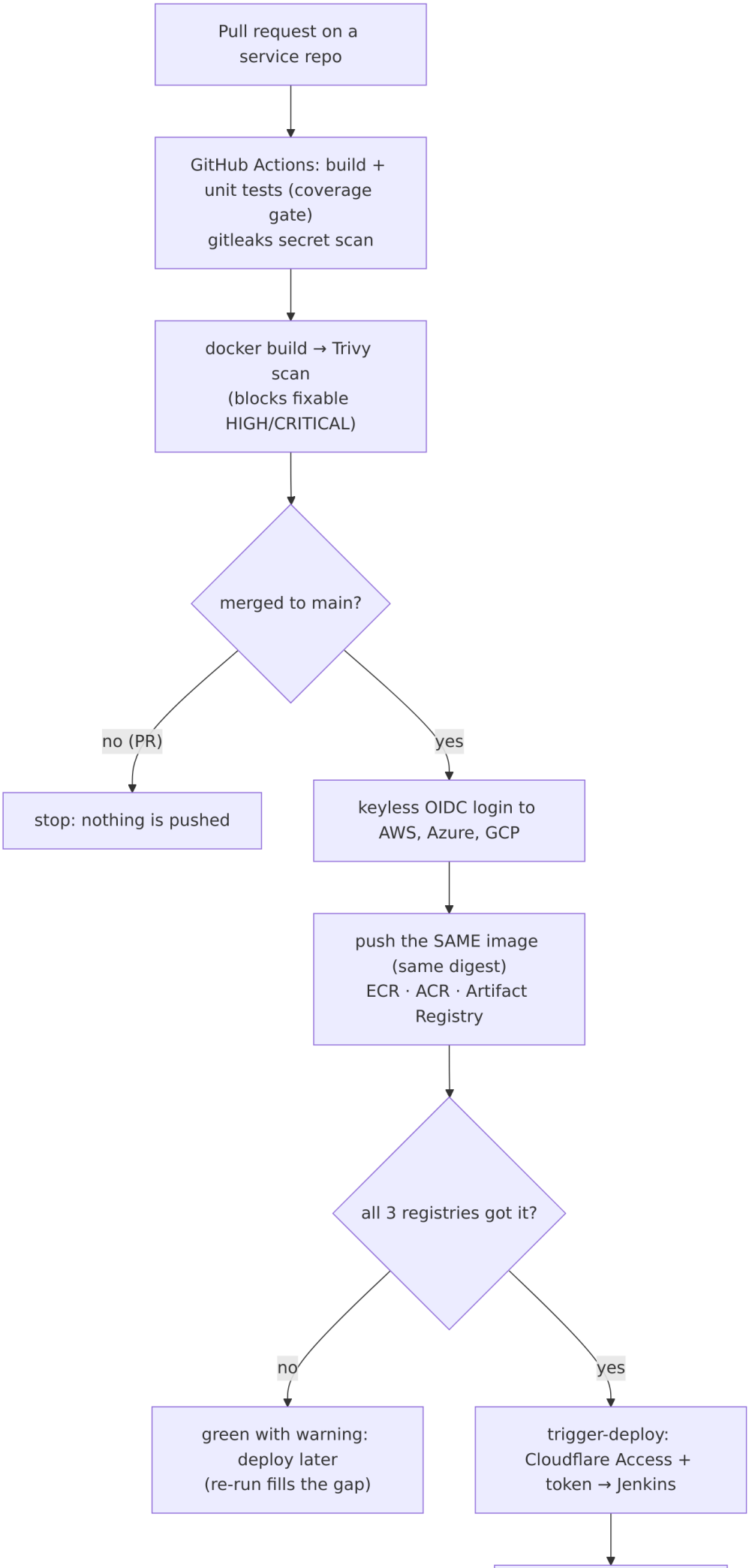
```
A="--region ap-south-1 --profile HeroviredProfile"
LB=$(aws elbv2 describe-load-balancers $A --query "LoadBalancers[?starts_with(LoadBalancerName,'k8s-
')].LoadBalancerArn" --output text)
TGS=$(aws elbv2 describe-target-groups $A --load-balancer-arn $LB --query
"TargetGroups[].TargetGroupArn" --output text)
aws elbv2 delete-load-balancer $A --load-balancer-arn $LB && aws elbv2 wait load-balancers-deleted $A -
-load-balancer-arns $LB
for tg in $TGS; do aws elbv2 delete-target-group $A --target-group-arn $tg; done
# then the controller's security groups (k8s-traffic-*, k8s-envoygat-*): remove rules that reference
them, then delete them
```

8. Rotate a secret

Secret	How
Atlas DB passwords, JWT secret, seed admin password	`terraform apply -replace=random_password.<name>` (or `-replace` the Atlas user) in `global-platform`; External Secrets refreshes within 1 h — restart the deployments to

Secret	How
	pick it up at once
Slack webhook	new value in GCP Secret Manager `mern-slack-webhook-url`, then `up.yml` / `global-platform` apply copies it
Cloudflare / Atlas provider keys	create new ones in the provider console, revoke the old ones; they are typed in per session and never stored
failover GitHub token	new fine-grained PAT → `gcloud secrets versions add failover-github-token` → apply `global-platform`
backup age key	new key pair; keep the **old private key** until every backup encrypted with it has expired

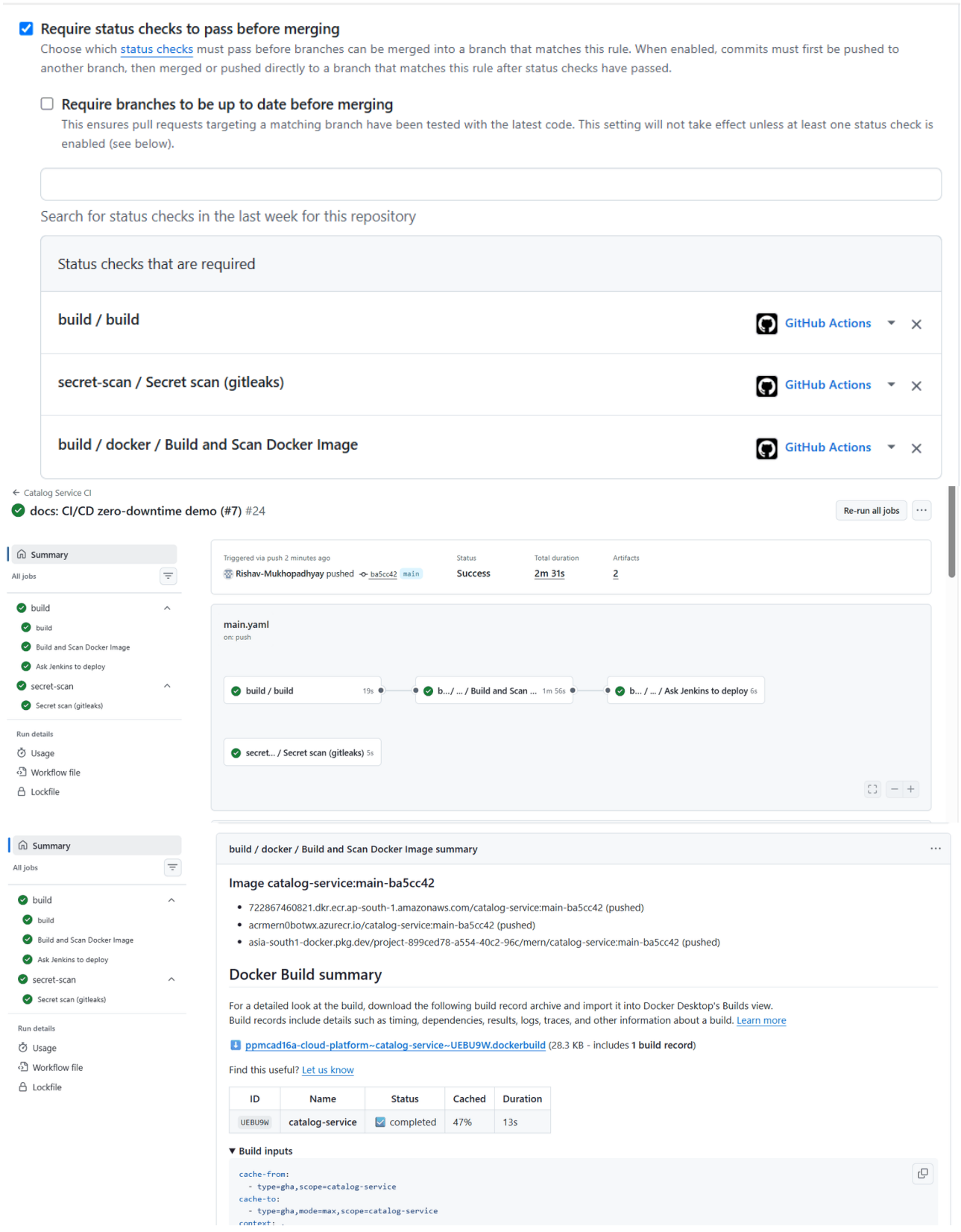
06 — CI/CD: from a pull request to three clouds


Pull request on a service repo
GitHub Actions: build + unit tests (coverage gate)
gitleaks secret scan
docker build → Trivy scan
(blocks fixable HIGH/CRITICAL)
merged to main?
no (PR)
yes
stop: nothing is pushed
keyless OIDC login to AWS, Azure, GCP
push the SAME image (same digest)
ECR · ACR · Artifact Registry
all 3 registries got it?
no
yes
green with warning: deploy later
(re-run fills the gap)
trigger-deploy:
Cloudflare Access + token → Jenkins

1. Continuous integration (GitHub Actions)

Each service repository contains only a small `main.yaml` that calls a shared, versioned workflow in `core-cloud-platform` (`node-app.yaml`, `java-app.yaml` or `python-app.yaml` `@v2`). The shared workflow does:

Step	Pull request	Push to `main`
install + unit tests with coverage gate	✅	✅
gitleaks over the whole git history	✅	✅
`docker build` + Trivy (blocks fixable HIGH/CRITICAL)	✅	✅
keyless login to AWS, Azure, GCP	—	✅
push the **same image** (tag `main-<sha>`) to ECR, ACR, Artifact Registry	—	✅
ask Jenkins to deploy	—	✅ (if all three registries have the image)



Registry outage tolerance (v2.2.0). Each cloud's login and push stand on their own. If one registry is down, the image still goes to the others and the run ends green with a warning naming the missing cloud — but the automatic deploy waits, because Jenkins would fail on the cloud without the image. Re-running the workflow later skips registries that already have the tag, fills the gap and then deploys.

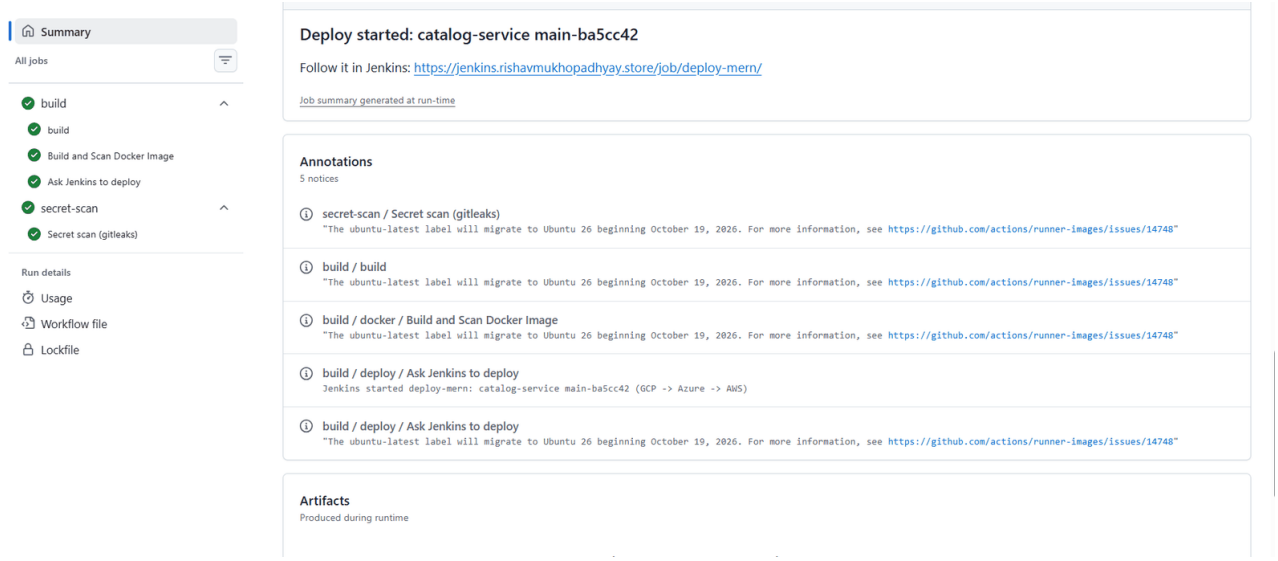
Registries that got the image	Run	Automatic deploy
3	green	yes

Registries that got the image	Run	Automatic deploy
1–2	green + warning	no — re-run later, or deploy now with `TARGETS=`
0	red	no

Versioning of the shared workflows: `v1` (build + scan), `v2.0.0` (push to 3 registries), `v2.1.1` (deploy trigger), `v2.2.0` (registry-outage tolerance); `v2` always points to the newest compatible 2.x. Release order: merge → pull → check `git log -1` → tag → move `v2`.

2. The hand-off to Jenkins

`trigger-deploy.yaml` reads two secrets from GCP Secret Manager (with the keyless CI identity), then calls `https://jenkins.<domain>/generic-webhook-trigger/invoke` with `{"service": …, "image_tag": …}`. The request must pass **two locks**: Cloudflare Access (service token) and the Jenkins trigger token. When the clusters are off, the job ends green with *"Jenkins is offline … deploy later with SERVICE=x IMAGE_TAG=y"*.



Jenkins triggered from GitHub

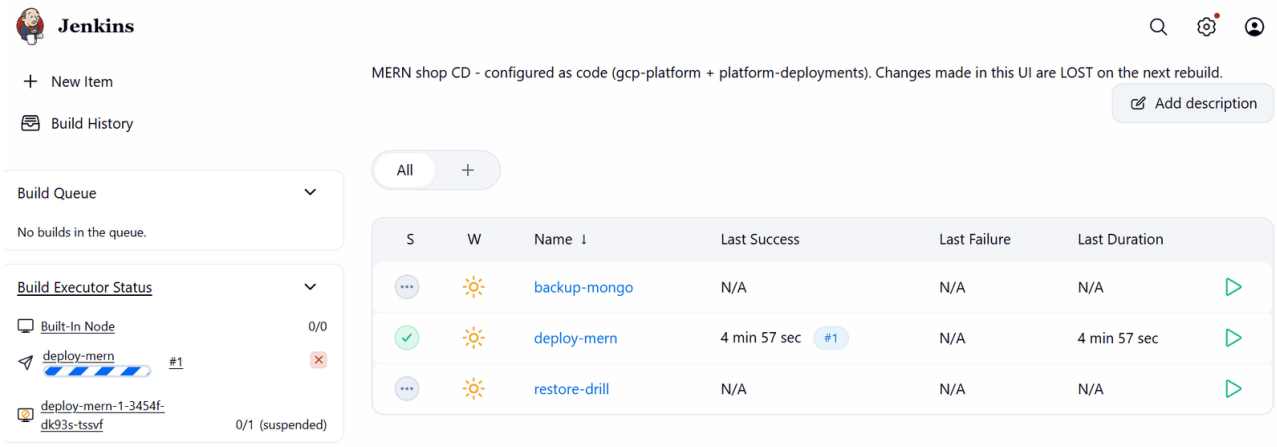
3. Continuous delivery (Jenkins in GKE)

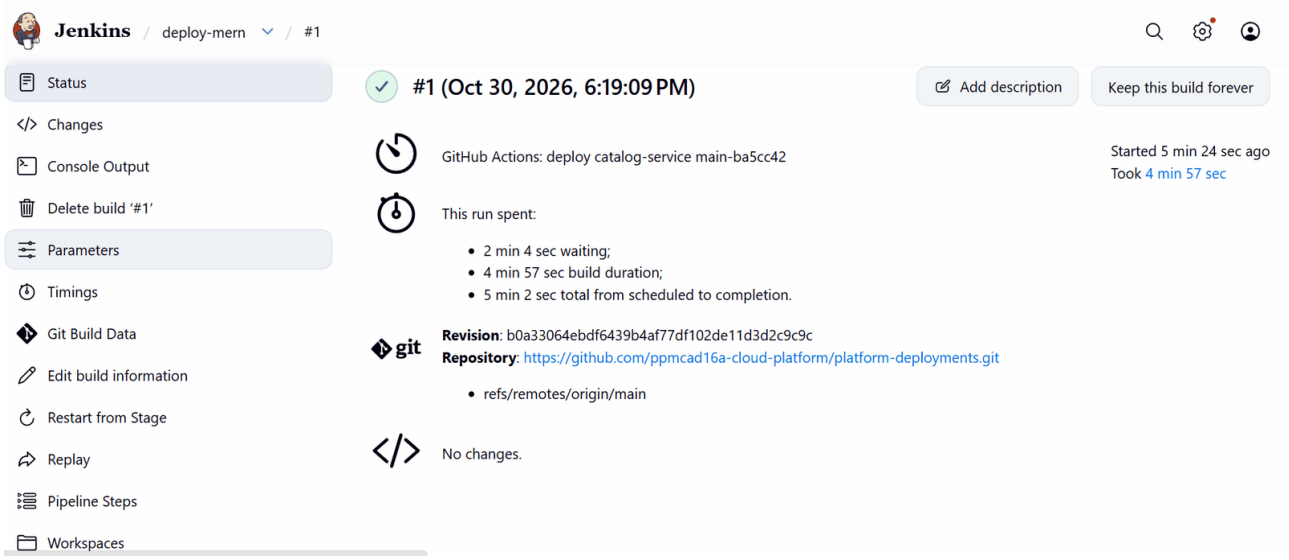
Jenkins is installed by Terraform (Helm chart 5.9.68) with Configuration-as-Code and Job DSL, so it is rebuilt identically every session. Jobs: `deploy-mern`, `backup-mongo`, `restore-drill`.

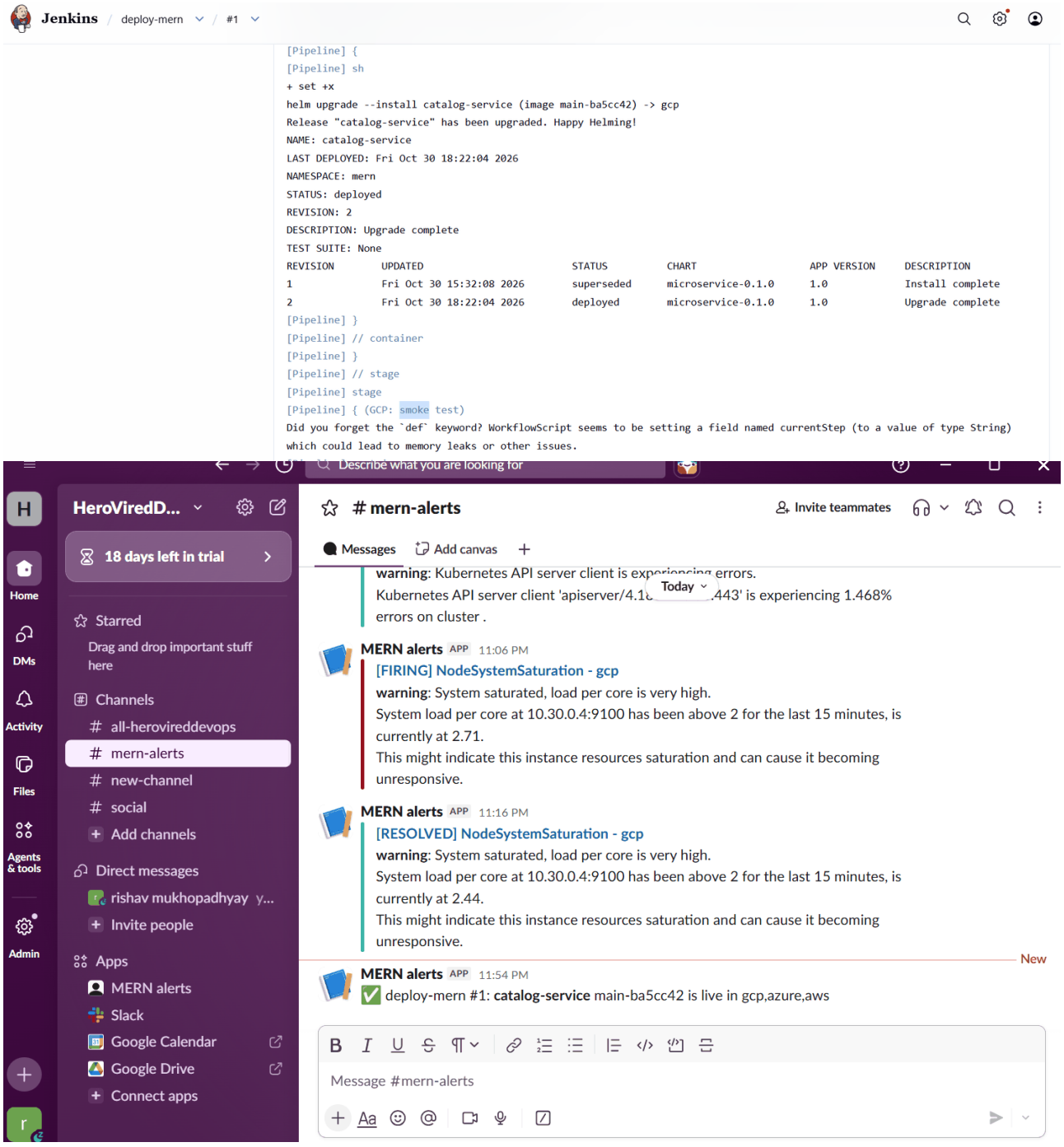
`deploy-mern` parameters: `SERVICE`, `IMAGE_TAG`, `TARGETS` (default `gcp,azure,aws`). Stages:

Stage	What happens
Validate	service name, tag format, target list
GCP: deploy → smoke test	`helm upgrade --install --rollback-on-failure --wait --timeout 8m`, then `https://gcp.<domain>/api/health` must answer 200
Azure: awake?	reads AKS power state; if *Stopped* → "sleeping standby, skipped"
Azure: credentials → deploy → smoke test	federated login (no stored key), same Helm command
AWS: awake? → credentials → deploy → smoke test	node group size 0 → skipped; else the same
Slack	☑ `deploy-mern #N: <service> <tag> is live in gcp,azure,aws (asleep, skipped: …)` or a failure message

Clouds are deployed **one after another**. A failure rolls back that cloud (Helm restores the previous revision) and **stops** — later clouds keep the old version, earlier ones already run the new one, so releases must be backward compatible.







4. Why a deploy causes no downtime

1. Kubernetes rolling update with `maxUnavailable: 0`: a new pod starts **next to** the old ones.
2. The new pod receives traffic only after its **readiness probe** (`/ready`, checks MongoDB) passes.
3. The old pod gets `SIGTERM`, the `preStop` hook gives Envoy time to stop routing to it, the service finishes in-flight requests and exits (graceful shutdown in every language).
4. PodDisruptionBudgets keep at least one copy during node maintenance; HPA keeps 2–4 replicas.
5. Only one cloud changes at a time, and Cloudflare keeps serving from the primary.

Measured: on 29 Oct 2026 a probe that requested all three clouds continuously during a CI-triggered deploy recorded **0 failed requests** (catalog-service PR #6, run “CD test 2 (zero-downtime probe)”). In the final test (30 Oct) the probe was stopped before the deploy started, so that run is **not** counted as evidence; the Grafana *Failed requests (5xx)* panel read 0.00 % in every cloud afterwards. The probe script and how to repeat the measurement are in Test report §D.

5. Failure and edge cases

Case	Behaviour
tests, gitleaks or Trivy fail	nothing is pushed or deployed
a registry is down	push to the others, warning, no auto-deploy (§1)
Jenkins offline	green run with “deploy later”
smoke test fails in Azure	Azure rolled back, AWS untouched, Slack alert (proven with a tag that exists only in GCP: Azure failed after up to 8 min, `Rollback to 2`, AWS got no new revision)
standby asleep	skipped, caught up on wake
GCP (Jenkins) down	break-glass `ansible-playbook playbooks/deploy.yml -e service=… -e tag=…`

6. Where the pieces live

Piece	Repo / file	Purpose
Push identities	`aws/azure/gcp-platform/environments/prod/foundation/ci-push.tf`	keyless identities that may only **add** images
Language pipelines	`core-cloud-platform/.github/workflows/{node,java,python}-app.yaml`	build + test; inputs `push-images`, `deploy-after-push`
Image workflow	`core-cloud-platform/.github/workflows/docker-build.yaml`	build, scan, push to 3 registries (main only)
Deploy trigger	`core-cloud-platform/.github/workflows/trigger-deploy.yaml`	reads the Access and trigger tokens from GCP Secret Manager, calls Jenkins
Service opt-in	`<service>/.github/workflows/main.yaml`	`push-images: true`, `deploy-after-push: true` (seed-db: image only)
Jenkins door	`global-platform/jenkins-access.tf`	DNS (only while GKE runs), Access application + policies, service token, CI secrets
Jenkins itself	`gcp-platform/environments/prod/addons/jenkins.tf` + `values/jenkins.yaml`	Helm release, Configuration-as-Code, Job DSL, Generic Webhook Trigger
Pipelines	`platform-deployments/pipelines/*.Jenkinsfile`	deploy, backup, restore drill

7. Manual operations

- **Open Jenkins:** `https://jenkins.<domain>` → Cloudflare e-mails a PIN → user `admin`, password `terraform output -raw jenkins_admin_password` in `gcp-platform/environments/prod/addons`.
- **Roll back:** run `deploy-mern` with the previous `IMAGE_TAG`.
- **Deploy to one cloud only:** set `TARGETS` (e.g. `gcp`).
- **Failure test:** an `IMAGE_TAG` that exists only in Artifact Registry deploys to GCP, fails in Azure after up to 8 minutes (Helm `--timeout 8m`), rolls Azure back and skips AWS; old pods serve throughout.

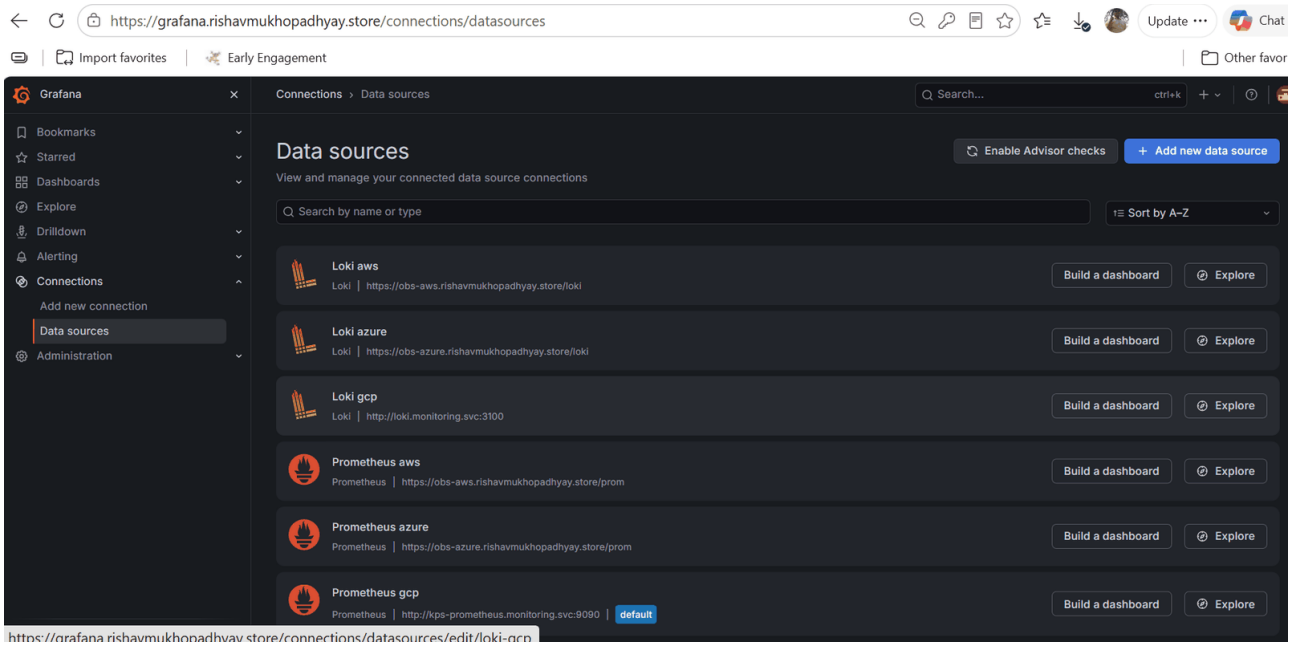
Verified on 29 Oct 2026: 7 services built, scanned and pushed with an identical digest in ECR, ACR and Artifact Registry; the offline path ended green with “deploy later”; a merge (catalog-service PR #5 → `main-c33cefa`) started Jenkins and reached all clouds; the failure path rolled Azure back (`Rollback to 2`) and gave AWS no new revision.

07 — Monitoring, alerts and cost

1. What runs where

Component	In every cluster	Central (GKE only)
Prometheus + Alertmanager (kube-prometheus-stack 92.6.0)	✅ scrapes nodes, pods and every service's `/metrics`	—
Loki 7.3.0 + Alloy 1.13.1	✅ collects every container's JSON logs, adds `cloud`, `app`, `pod` labels	—
OpenCost 2.5.32	✅ cost per node / namespace from list prices	—
Alert rules + dashboards	✅ from module `observability-platform` **v1.1.0** (identical in every cloud)	—
Grafana	—	✅ `https://grafana.<domain>` behind Cloudflare Access; data sources: 3 × Prometheus, 3 × Loki

Azure and AWS expose their Prometheus/Loki to the central Grafana only through `obs-azure.` / `obs-aws.` hostnames — `GET` only, Cloudflare Access service token required.



Grafana data sources

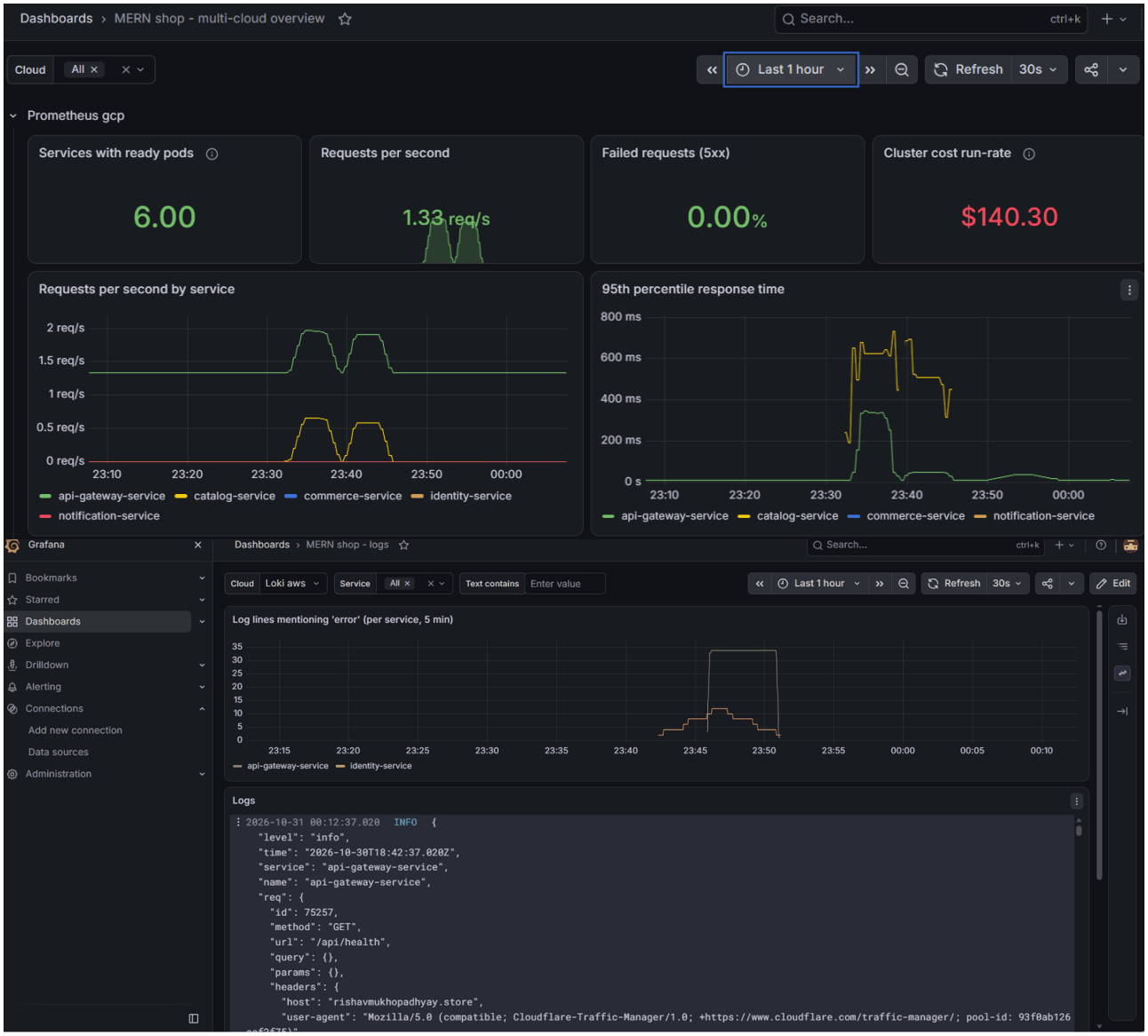
2. Dashboards

MERN shop — multi-cloud overview (one row per cloud, `cloud` selector at the top):

Panel	Meaning
Services with ready pods	how many of the 6 services can serve right now (healthy = 6)
Requests per second	traffic through the gateway
Failed requests (5xx)	share of server errors
Cluster cost run-rate	OpenCost: this cluster's nodes, per 30-day month at the current size
Requests per second by service · 95th percentile response	per-service RED metrics (rate, errors, duration)

Panel	Meaning
time · 5xx per second · container restarts	

MERN shop — logs: log lines mentioning “error” per service, and the raw JSON logs, filterable by cloud and service. In *Explore*, `{namespace="mern", app="catalog-service"}` shows one service’s logs in one cloud.

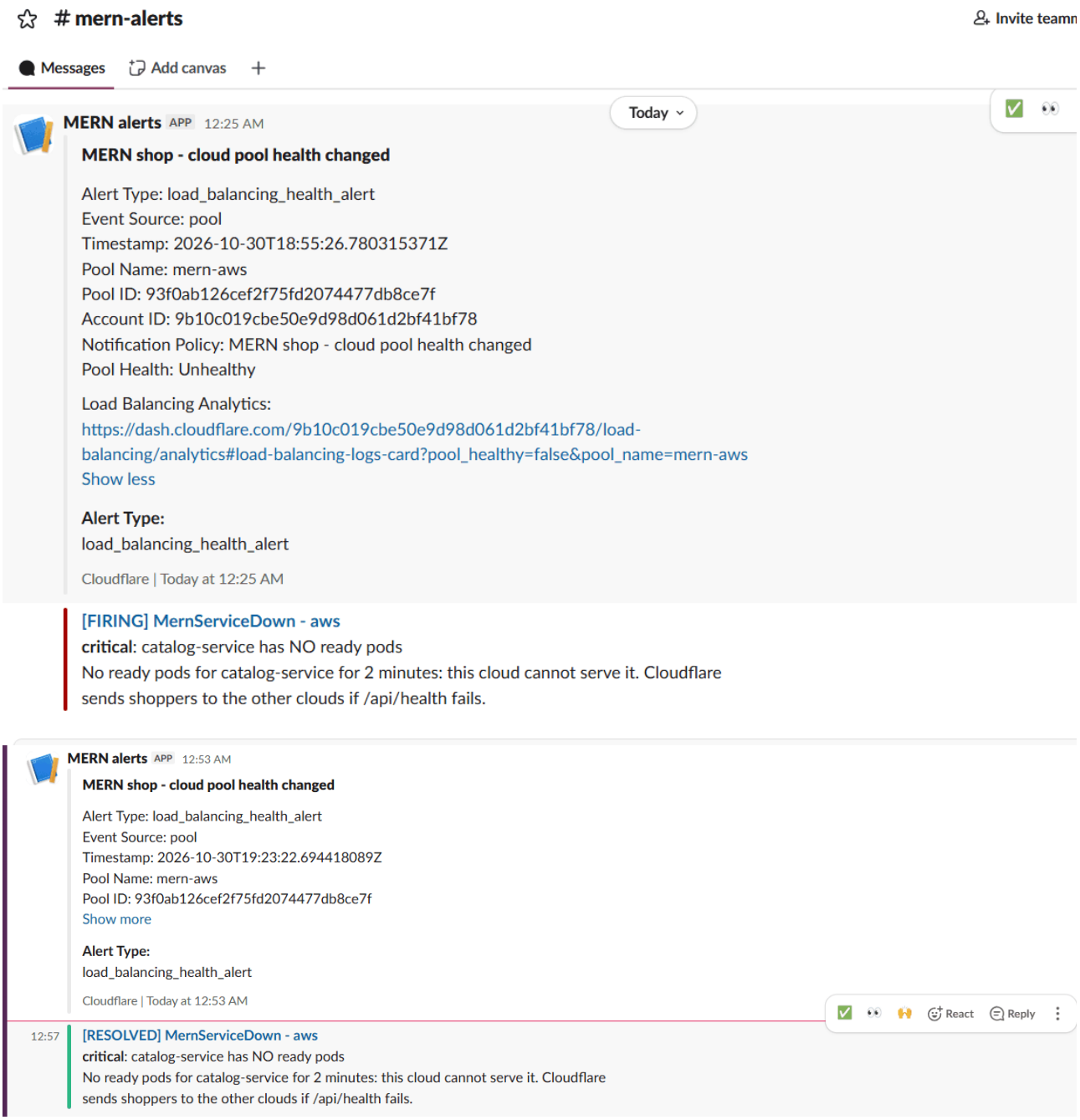


3. Alerts → Slack (`#mern-alerts`)

Alert	Fires when	Severity
MernServiceDown	a deployment in `mern` has **no ready pod for 2 min** — this cloud cannot serve it	critical
MernHighErrorRate	> 5 % of a service’s requests fail with 5xx for 5 min (health-check traffic excluded since v1.1.0)	critical
MernServiceDegraded	fewer ready pods than wanted for 10 min (crash loop, image pull, no room)	warning
MernSlowResponses	p95 response time > 1 s for 10 min	warning
MernPodRestarting	a container restarted > 2 times in 15 min	warning
kube-prometheus-stack defaults	e.g. `NodeSystemSaturation`, API-server errors	warning/critical

Alert	Fires when	Severity
Cloudflare pool health	a cloud's pool turns unhealthy / healthy	notification
Jenkins	deploy succeeded / failed / standby skipped; backup and restore results	info

Each cloud's Alertmanager posts to Slack directly (webhook from its own vault), so alerts keep arriving when GCP — and the central Grafana — is down.



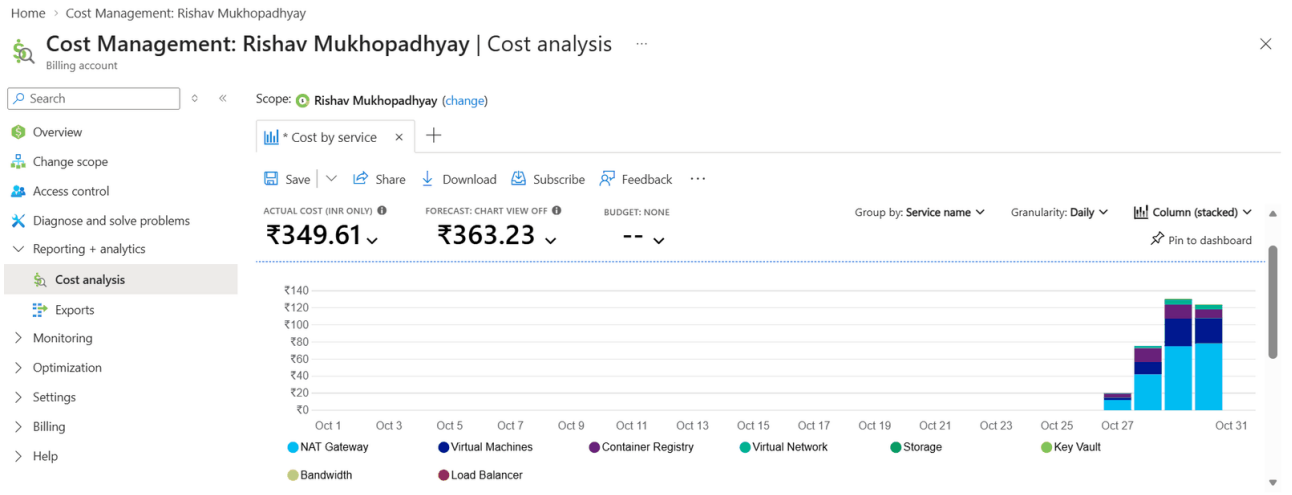
4. Cost

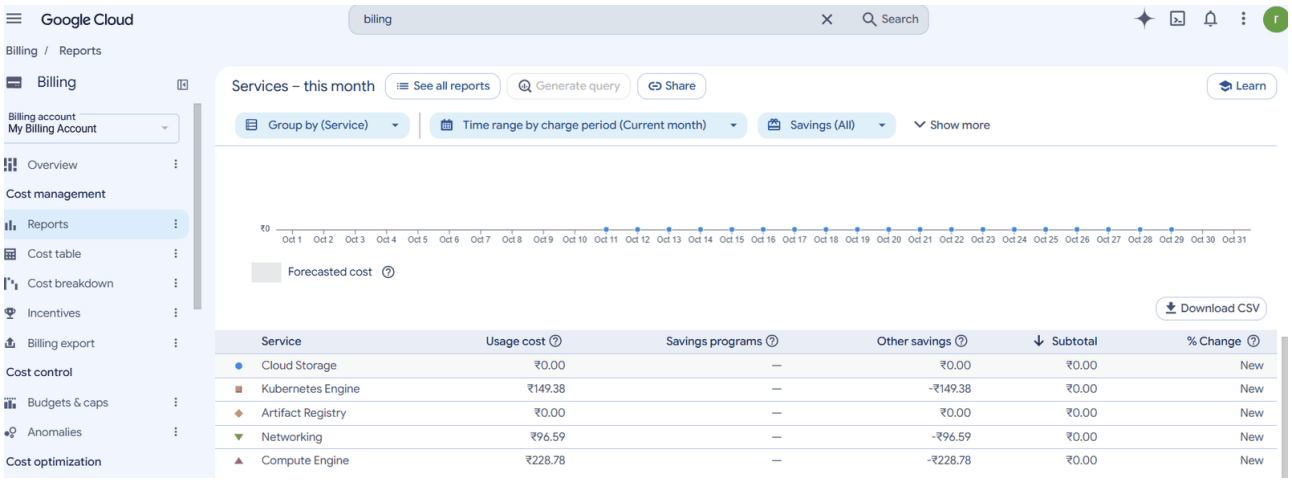
4.1 What was actually billed (1–30 October 2026)

Cloud	Billed	Main items
AWS	**US $6.76**	EKS control plane, EC2 nodes, NAT gateway, load balancer

Cloud	Billed	Main items
Azure	**₹349.61** (≈ US $4.2)	NAT gateway, VMs, container registry, load balancer, storage
GCP	**₹474.75** gross (Compute ₹228.78 · Kubernetes Engine ₹149.38 · Networking ₹96.59), **₹0 payable** (free credits)	
MongoDB Atlas	US $0	M0 free tier
Total	**≈ US $17 for a month of building and testing three clouds**	plus the Cloudflare Load Balancing add-on (US $5/month) and the domain







4.2 What it would cost to leave running

OpenCost run-rates measured with all three clusters awake (nodes only, list prices, per 30-day month):

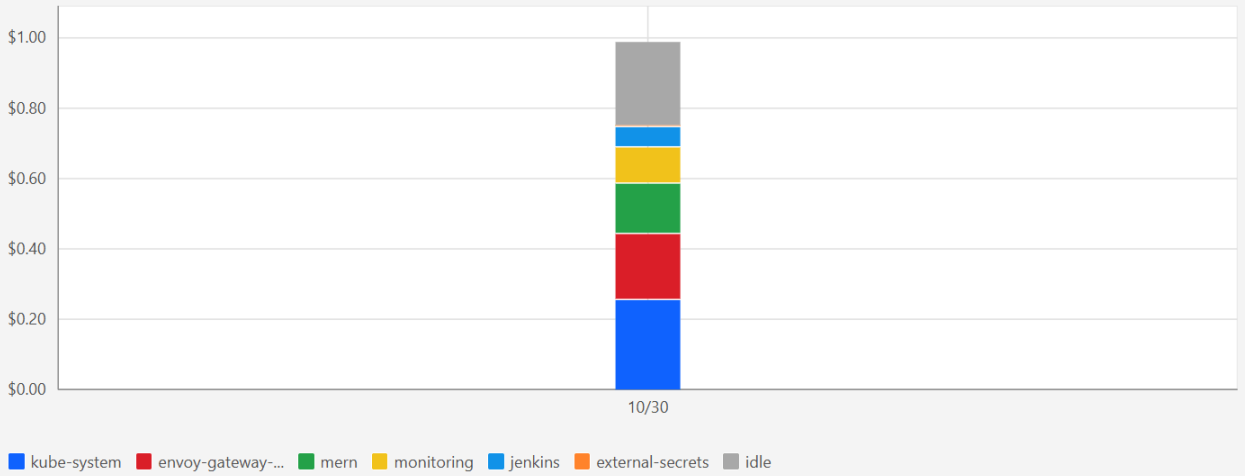
Cluster	Run-rate
GCP (2 × e2-standard-2 + Jenkins/Grafana load)	**US $140.30**
AWS (2 × t3.large)	**US $130.82**
Azure (2 × D2as_v5)	**US $81.17**

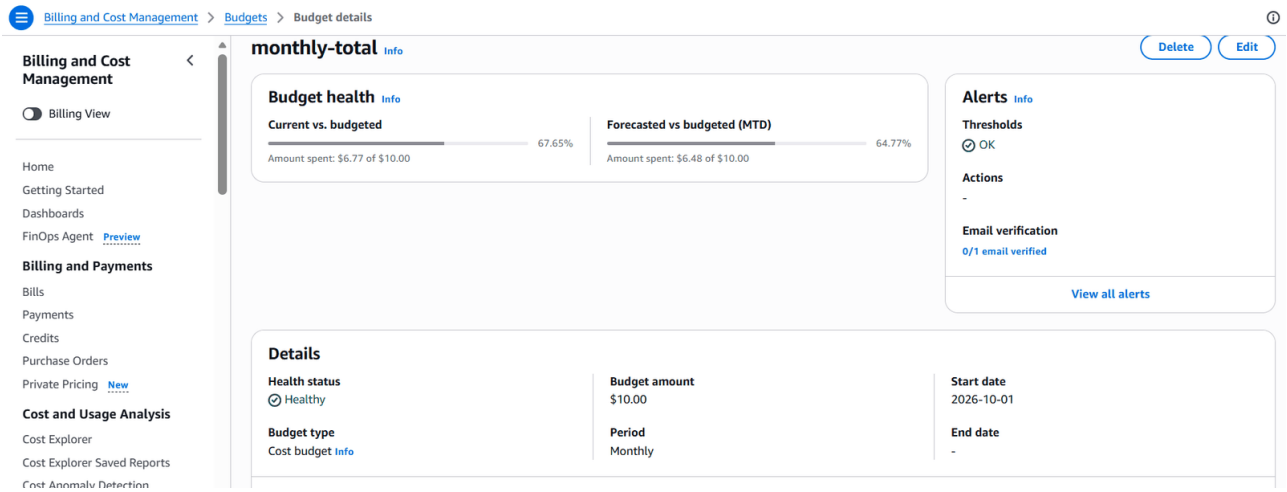
Operating model	Node cost / month	Comment
Active-active, 24/7	≈ US $352	plus 3 × NAT, LB, EKS control plane (US $73)
Primary + stopped standbys, 24/7	≈ US $140	standby nodes cost **$0** while asleep → **−60 %** (≈ US $212 saved); the EKS control plane, NAT gateways, IPs and disks still cost money
Sessions only (what we run)	pay per hour of use	≈ US $0.8–1 per working hour with all three clusters awake (nodes ≈ $0.48/h + NAT, load balancers, EKS control plane); nothing per-hour between sessions

4.3 How cost is kept low

Measure	Effect
Clusters, add-ons and the global layer exist **only during sessions** (`up.yml` / `down.yml`)	no node, NAT or load-balancer cost between sessions
Stopped standbys	Azure/AWS nodes off unless GCP fails
Small, equal node shapes (2 vCPU / 8 GB), autoscaling 2–3	fits free-trial quotas
GKE **zonal** Standard cluster	no GKE management fee for one zonal cluster
AKS **Free** tier, Atlas **M0**	$0 control plane / database
Artifact Registry keeps only the newest 10 images; Azure backup storage has a lifecycle policy	storage stays small
`down.yml` **leftover check**	catches orphaned load balancers and disks — the classic silent cost
Budgets (AWS $10, Azure $10, GCP ₹850 ≈ $10) with e-mail alerts	warning long before credits run out
OpenCost per namespace	shows what each part of the platform costs

Last 7 days by namespace (Daily)



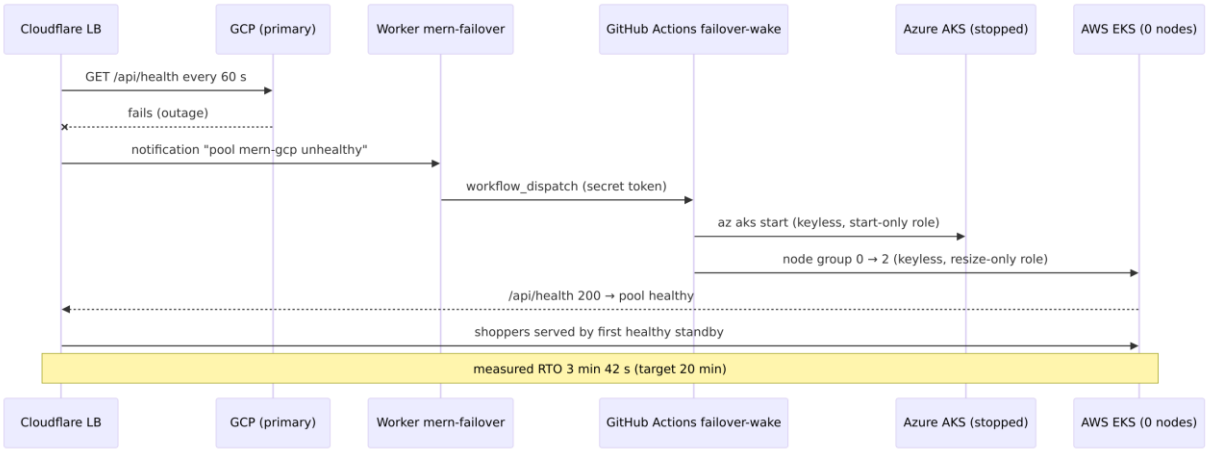


08 — Disaster recovery and failover

1. What can fail, and what happens

Failure	Detection	Recovery	Time	Data loss
a pod crashes	probes	Kubernetes restarts / replaces it; other replicas keep serving	seconds	none
a node disappears	Kubernetes	pods rescheduled; PDB keeps one copy	1–2 min	none
a bad release	smoke test per cloud	Helm rolls that cloud back, pipeline stops, Slack	≤ 8 min	none
a **standby** cloud is lost	Cloudflare pool alert	nothing urgent — GCP serves everyone; rebuild with `up.yml`	—	none
the **primary (GCP)** is lost	Cloudflare monitor (60 s)	**automatic**: Cloudflare → Worker → GitHub Actions wakes Azure and AWS; the first healthy pool takes the traffic	**RTO 3 min 42 s** (measured)	none (shared database)
the database (Atlas) is lost	`/api/health` fails in every cloud, MernServiceDown	restore the newest encrypted backup into a new cluster (runbook §4.3)	≈ 1 h (manual)	**RPO ≤ 24 h** — since the last nightly backup (between sessions no cluster runs, so nothing is written)
one backup copy is lost or corrupt	restore drill per source	use one of the other two copies	minutes	none
a registry is down	CI warning	other registries still get the image; re-run fills the gap	—	none

2. Failover: the stopped-standby model



Failover sequence

- Cloudflare's load balancer has three pools in failover order **gcp → azure → aws** and checks `GET /api/health` on each every 60 s. `/api/health` is a *deep* check: the API gateway asks every backend's `/ready`, which checks MongoDB — so "healthy" means "can really serve a shopper".

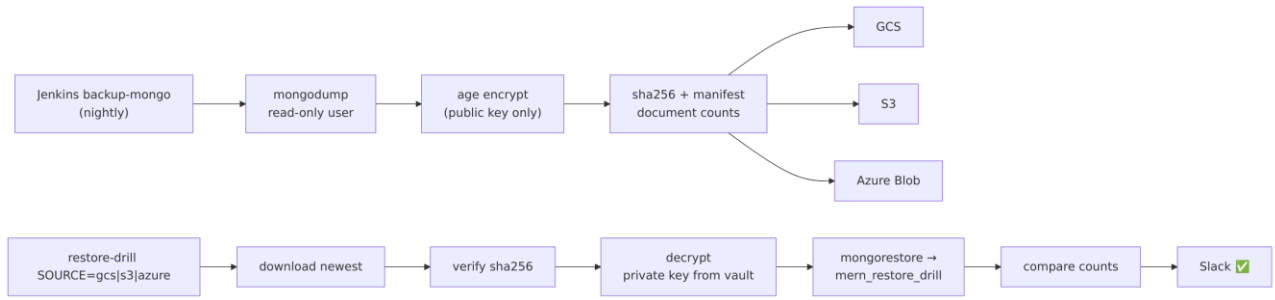
- Azure and AWS normally **sleep** (AKS stopped, EKS node group at 0). All their configuration — cluster, add-ons, shop, secrets, DNS, pool — stays in place, like a stopped EC2 instance.
- When the **gcp** pool turns unhealthy, a Cloudflare notification policy calls the Worker `mern-failover` (`https://mern-failover.<subdomain>.workers.dev`). The Worker checks a shared secret and starts the GitHub workflow `platform-automation/.github/workflows/failover-wake.yml` with a fine-grained token stored in GCP Secret Manager → Worker secret binding.
- The workflow logs in **keylessly** with identities that may only *start* AKS or *resize* the node group, wakes both standbys in parallel, and stops if there is nothing to wake.
- The first standby that answers `/api/health` with 200 takes the traffic (AWS woke in 2 min 48 s, Azure in ≈ 6 min in the drill).

Principle: the failover control plane (Cloudflare + GitHub Actions) does **not** live in the thing that fails (GCP).

What survives a GCP outage

Capability	During a GCP outage
Shop for customers	✅ served by Azure/AWS after the automatic wake-up
Image builds (CI)	✅ GitHub Actions; ECR/ACR still receive images (v2.2.0 tolerates Artifact Registry being down)
Deployments (CD)	⚠️ Jenkins runs in GKE → use break-glass `ansible-playbook playbooks/deploy.yml`
Alerts	✅ Azure/AWS Alertmanagers post to Slack directly; Cloudflare pool alerts
Central Grafana	❌ lives in GKE → use each cloud's Prometheus/Loki directly until GCP is back
Nightly backup	❌ job runs in Jenkins → take a manual backup if the outage is long
Secrets	✅ each cloud has its own vault (copies written by `global-platform`)

3. Backup and restore



Backup and restore

	Backup (`backup-mongo`, nightly while the platform runs, or on demand)	Restore drill (`restore-drill SOURCE=gcs|s3|azure`)
Database access	Atlas user `backup-ro` (read-only)	Atlas user `restore-drill` (only database `mern_restore_drill`)
Encryption	`age` with the **public** key only — the job can encrypt but never decrypt	private key read from the vault, deleted after the job
Integrity	SHA-256 + manifest with document count per collection	checksum must match; every collection's count must match
Storage	GCS, S3, Azure Blob — the job **fails unless all three** uploads succeed	newest copy from the chosen cloud
Identity	write-only per bucket (Azure: Put Blob over REST because the identity may not even read)	read-only per bucket

	Backup (`backup-mongo`, nightly while the platform runs, or on demand)	Restore drill (`restore-drill SOURCE=gcs|s3|azure`)
Result	Slack 💾 `backup-mongo #N: <name> stored in GCS + S3 + Azure Blob`	Slack ☑ `restore-drill #N: <name> from <source> restored and verified`

The private key exists in GCP Secret Manager, Azure Key Vault and AWS SSM (outside `/mern/`, so no cluster can read it) and in a password manager — a backup can be restored even if GCP is gone completely.

Example manifest (final test): `mern_ecommerce`, 14,711 bytes, `products 100, orders 13, carts 13, brands 10, categories 10, contacts 8, users 5, reviews 2, merchants 1, wishlists 1`.

4. Runbooks

4.1 GCP is down (real outage)

1. Check Cloudflare → Load Balancing: `mern-gcp` critical; Slack shows "pool unhealthy".
2. Watch GitHub → `platform-automation` → Actions → **Failover – wake standbys**. If it did not start within 5 minutes, wake by hand: `ansible-playbook playbooks/wake.yml -e cloud=azure` (and `aws`).
3. Confirm `https://<domain>` answers and `azure./aws.<domain>/api/health` is `ok`.
4. Releases during the outage: `ansible-playbook playbooks/deploy.yml -e service=… -e tag=….`
5. When GCP is back and healthy, Cloudflare returns traffic to it automatically (failover order). Put the standbys back to sleep: `ansible-playbook playbooks/standby.yml`.

4.2 A whole cloud must be rebuilt

The cloud's foundation (registry, vault, backups) is permanent and in Terraform state; clusters are rebuilt every session anyway. Run `ansible-playbook playbooks/up.yml` — the guarded plan shows exactly what will be created.

4.3 The database is lost

1. Create a new Atlas cluster (or project) and update `atlas_project_id`, `atlas_cluster_name`, `mongo_host` in `global-platform/variables.tf`.
2. `terraform apply` in `global-platform` → new per-cloud users and connection strings in all three vaults; External Secrets refreshes them (restart the deployments to apply at once).
3. Restore the newest backup into the **real** database: run the restore steps by hand with a user that may write `mern_ecommerce` — download the newest archive (`scripts/dr/download-latest.sh <gcs|s3|azure>`), verify its SHA-256 against the manifest, decrypt with the private key from the vault (`age -d -i key.txt`), then `mongorestore --uri "<uri>" --gzip --archive=<file> --nsInclude 'mern_ecommerce.*'`.
4. Check the document counts against the manifest, then `/api/health` in every cloud.

4.4 Prove it regularly

```
ansible-playbook playbooks/drill-restore.yml      # all three backup copies restore and match
ansible-playbook playbooks/drill-failover.yml     # primary fails, standbys wake, RTO measured
```

5. Drill results (final test, 30 Oct 2026)

Drill	Result
Backup `mern-20261030T212438Z`	stored in GCS + S3 + Azure Blob
Restore drill	**gcs: SUCCESS · s3: SUCCESS · azure: SUCCESS** — "Restore drill PASSED"
Failover drill (automatic wake-up)	shop down after 15 s → standbys woken by Cloudflare → Worker → GitHub Actions → shop back; **RTO 3 min 42 s** (target 20 min); MernServiceDown raised in GCP

```
TASK [Drill report] ******************************************************
ok: [gcp] =>
    msg:
    - 'gcs: SUCCESS'
    - 's3: SUCCESS'
    - 'azure: SUCCESS'

TASK [All three backup copies restored and verified] *************************
ok: [gcp] =>
    changed: false
    msg: 'Restore drill PASSED: GCS, S3 and Azure Blob copies all restore and match.'

PLAY [7. Drill report] ***************************************************

TASK [Report] ************************************************************
ok: [localhost] =>
    msg:
    - 'Wake-up: auto'
    - 'Shop down after: 15 s (Cloudflare noticed the primary failing)'
    - 'Shop back (served by a standby): True'
    - 'RTO (front door closed -> shop answering again): 3 min 42 s  (target 20 min)'
    - 'MernServiceDown alert raised in GCP: True'

TASK [The drill passed] **************************************************
ok: [localhost] =>
    changed: false
    msg: Failover drill PASSED - RTO 3 min 42 s
```

09 — Final end-to-end test report

When: 30 October 2026 14:55 UTC → 31 October 2026 (teardown) **Who:** Rishav Mukhopadhyay (operator) — commands from WSL Ubuntu / PowerShell, evidence by screenshot **Scope:** everything at once — start from code, run the shop in three clouds, release through CI/CD, monitor, sleep the standbys, back up and restore, fail over, tear down.

Summary

Part	Test	Result
A	Permanent foundations: repos, CI, registries, backups, state, identities, budgets	✅ all present
B	Session start with `up.yml` (3 clusters in parallel)	✅ `failed=0` everywhere, 23 min 13 s [1]
C	Shop, clusters, global layer and database in all three clouds	✅ 6/6 services ready ×3, `/api/health` ok ×3
D	One merge → CI → same image in 3 registries → Jenkins → 3 clouds	✅ live in gcp, azure, aws in 4 min 57 s [2]
E	Dashboards, logs, alert → Slack → resolved, cost	✅
F	Standbys to sleep; Jenkins skips them	✅ (AWS sleep slow — issue 4)
G	Backup to 3 clouds · restore from 3 clouds · automatic failover	✅ 3/3 restores · **RTO 3 min 42 s**
H	Teardown with `down.yml`	✅ after manual fixes (issues 5–6)

[1] GKE and AKS already existed from the first attempt (see issue 1); EKS was built from scratch. [2] Zero downtime during this deploy was **not** measured — the probe was stopped before the deploy began (see §D).

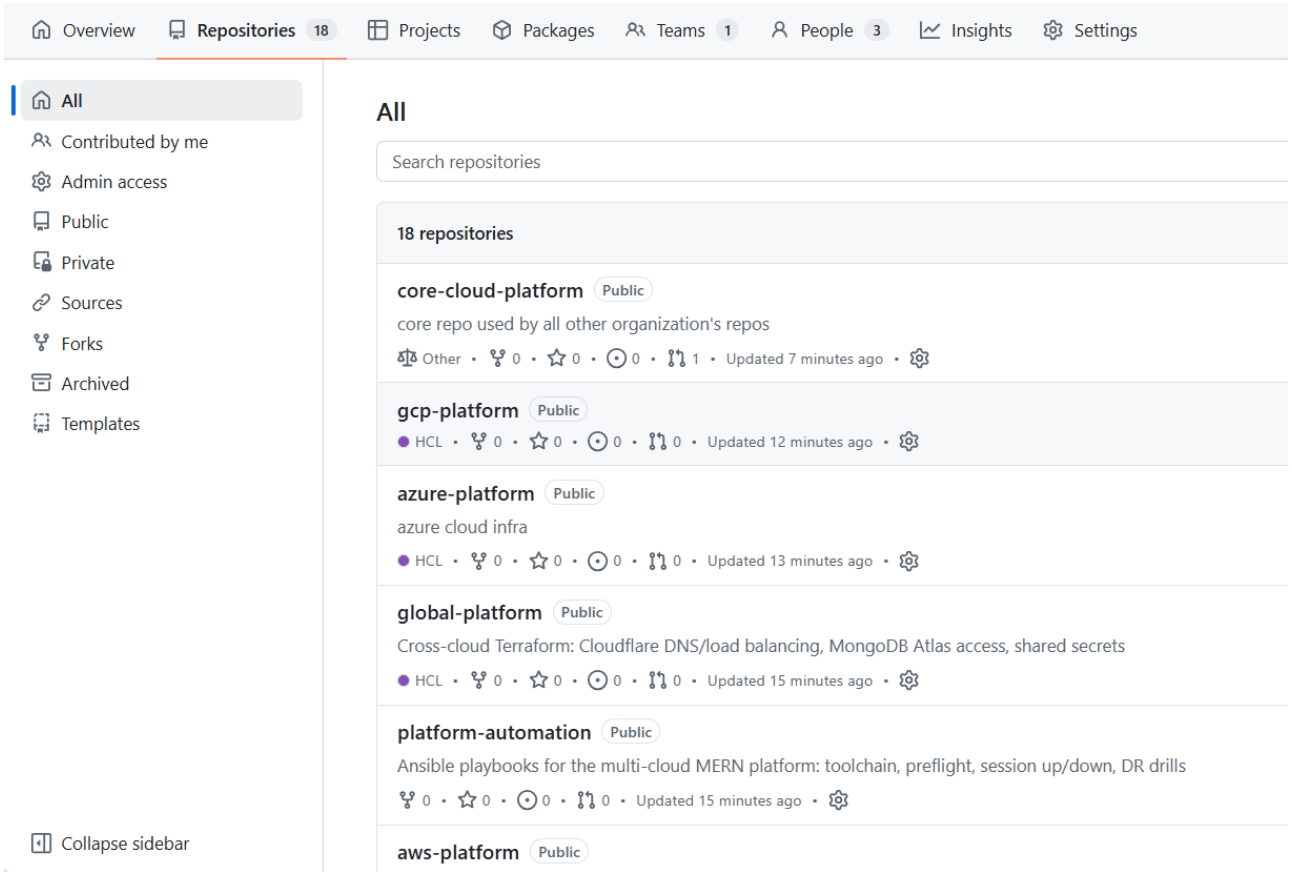
Issues found during the test

#	Issue	Handled
1	AWS SSO session expired → AWS dropped out of the first `up.yml`, then the global layer failed	`aws sso login`, re-run — Troubleshooting
2	Operator's public IP changed twice (home ↔ phone hotspot) → cluster APIs and Atlas API refused	IP updated in tfvars, Atlas key, clusters re-applied — Runbook §6
3	Helm chart downloads timed out — one GitHub CDN address unreachable from the ISP	address pinned — Troubleshooting
4	AWS standby sleep stuck ~30 min (PDB drain); Azure stop refused during an in-progress start	manual drain / wait — see Limitations
5	AWS NLB left behind → `down.yml` safety stop kept EKS	NLB, target group, security groups deleted by hand, re-run
6	Atlas IP access list contained a manual `0.0.0.0/0`	removed

A. Permanent foundations

These exist between sessions and cost (almost) nothing.

GitHub organisation and branch protection



Branch protection rule

Level up your branch protections with Repository Rules

Rulesets are the recommended replacement for classic branch protections. They support rule layering, evaluate mode, and organization-wide policies. Learn more about Rulesets.

Branch name pattern *

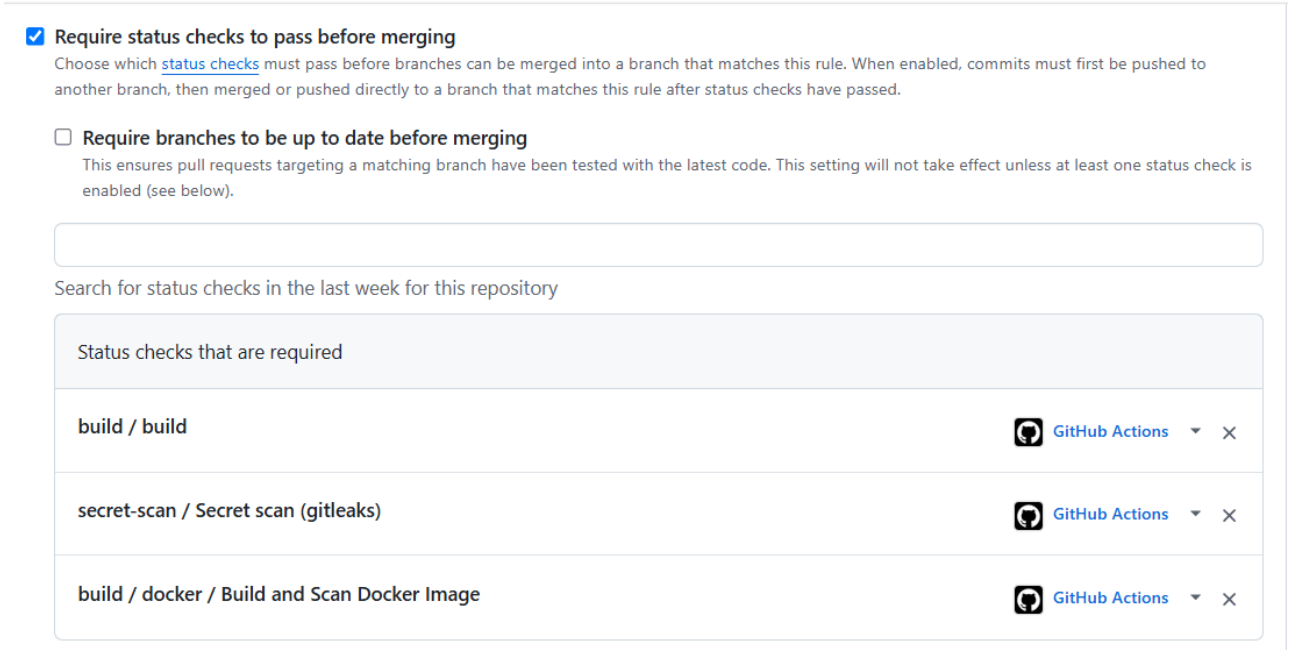
main

Applies to 1 branch

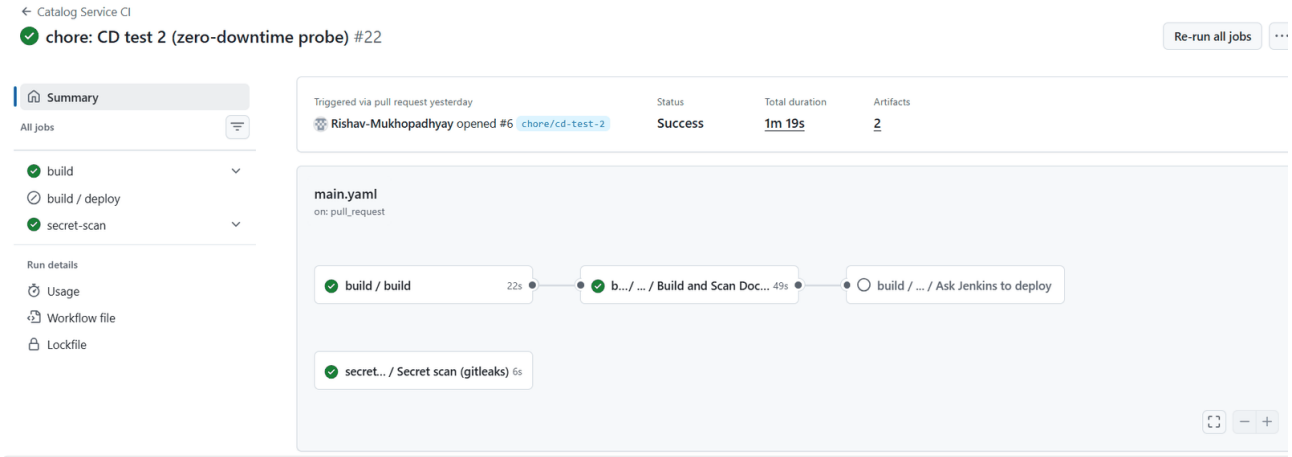
`main`

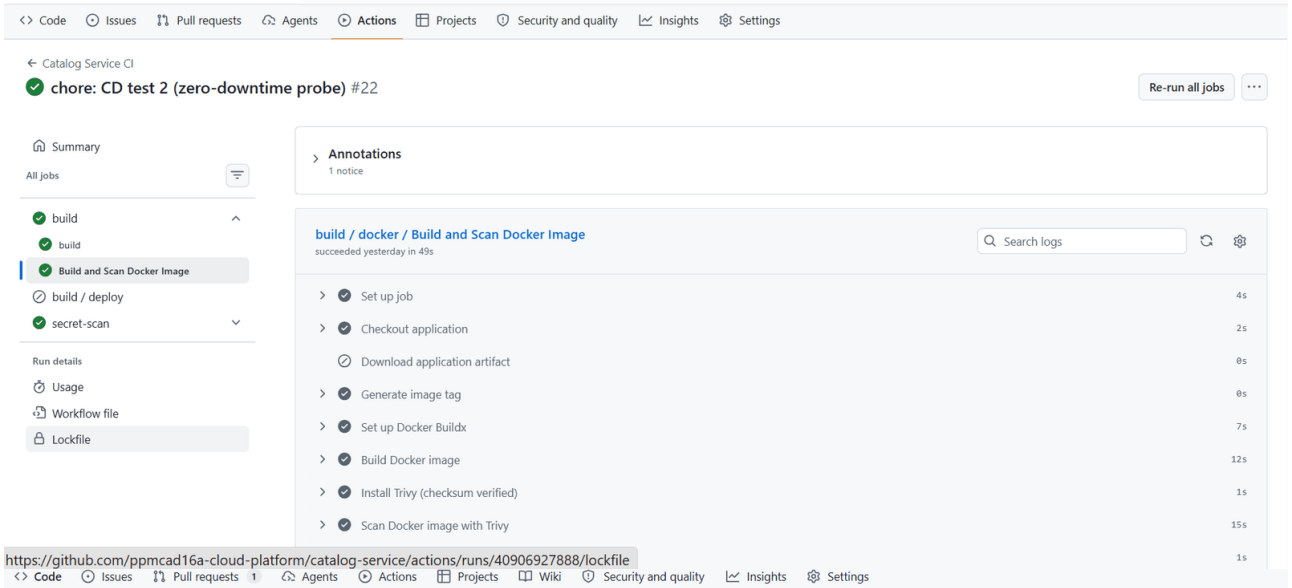
Protect matching branches

- [x] **Require a pull request before merging**
  When enabled, all commits must be made to a non-protected branch and submitted via a pull request before they can be merged into a branch that matches this rule.
  - [x] **Require approvals**
    When enabled, pull requests targeting a matching branch require a number of approvals and no changes requested before they can be merged.
    Required number of approvals before merging: 1

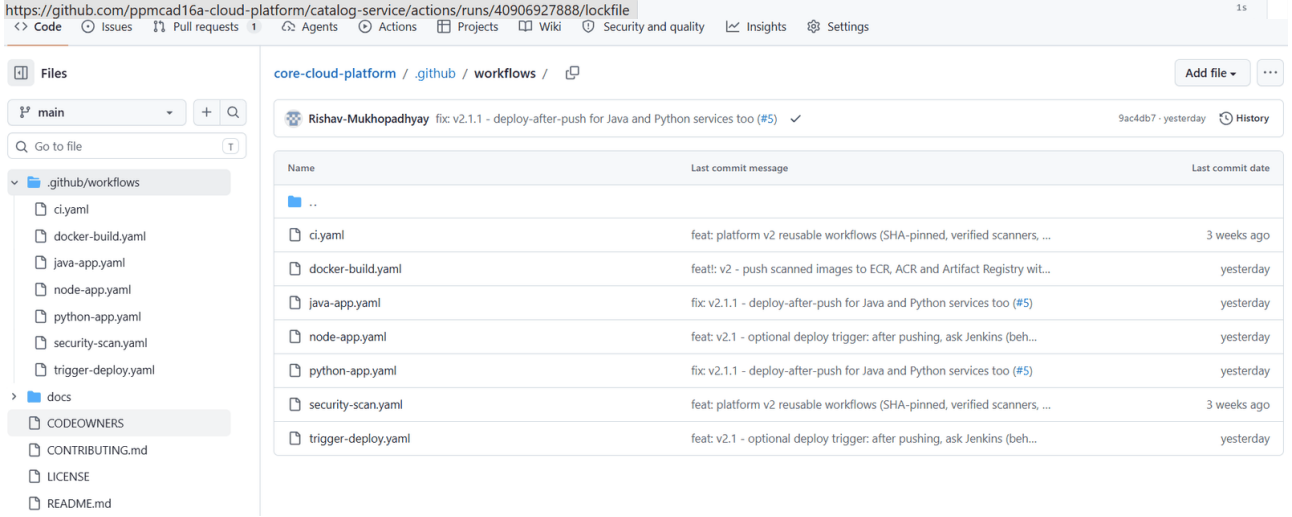


CI run, security scan and shared workflows

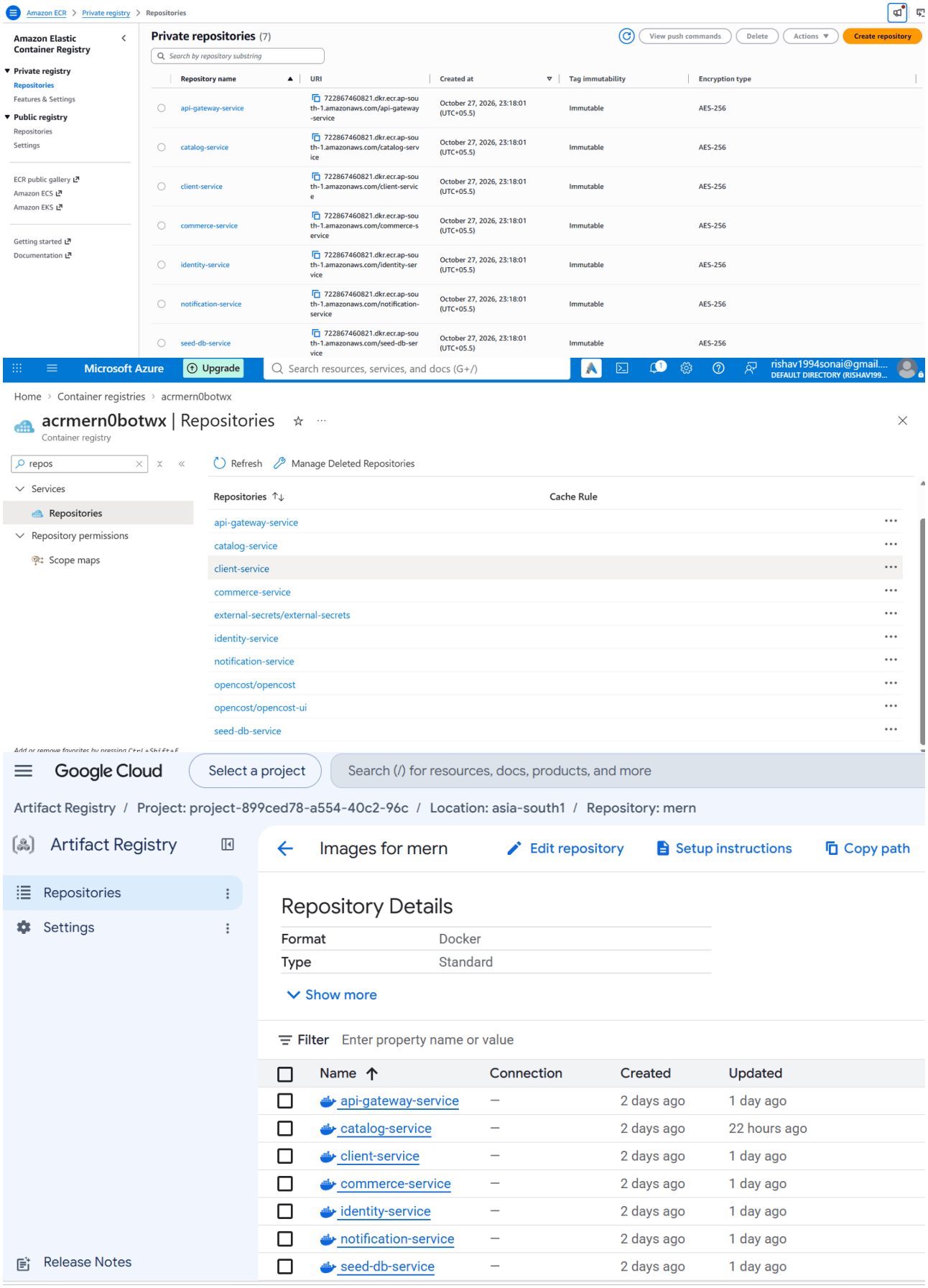




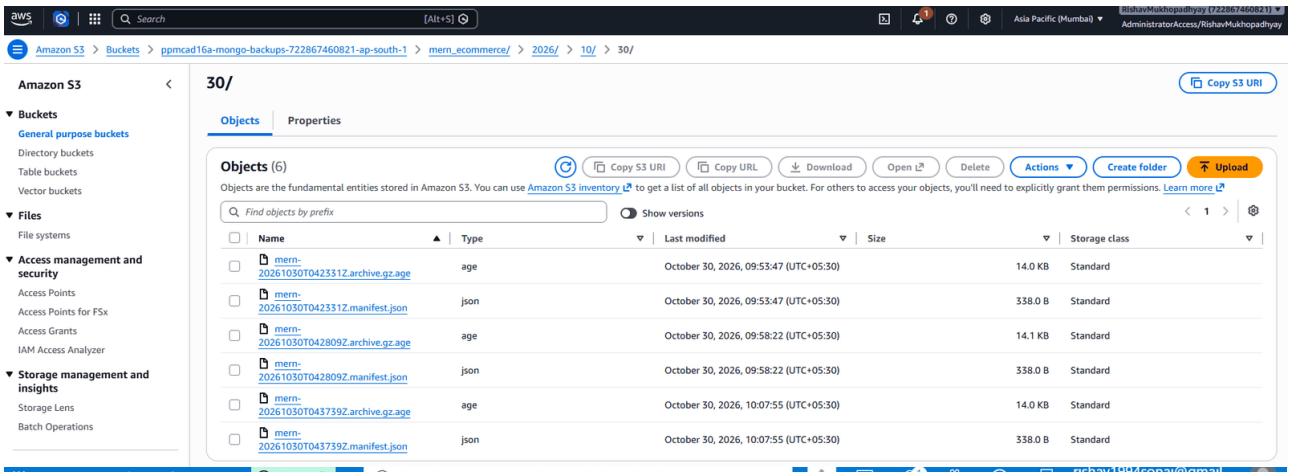
https://github.com/ppmcad16a-cloud-platform/catalog-service/actions/runs/40906927888/lockfile

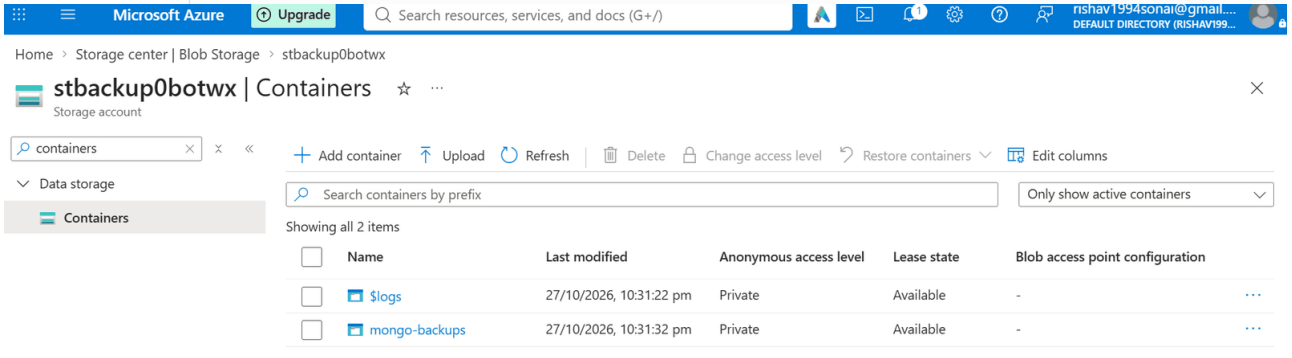


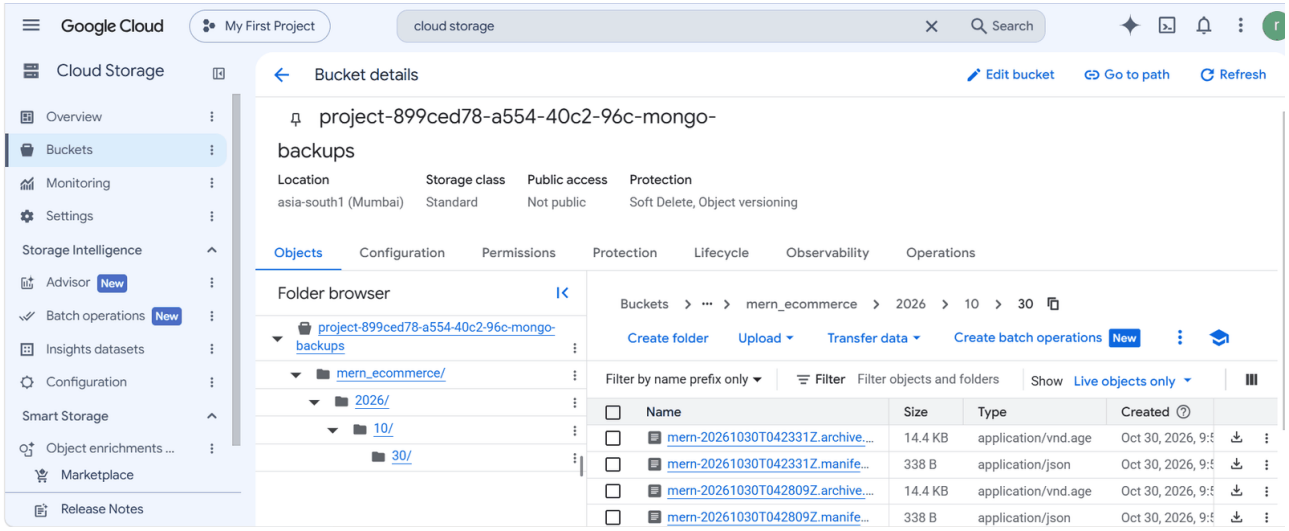
The same images in three registries



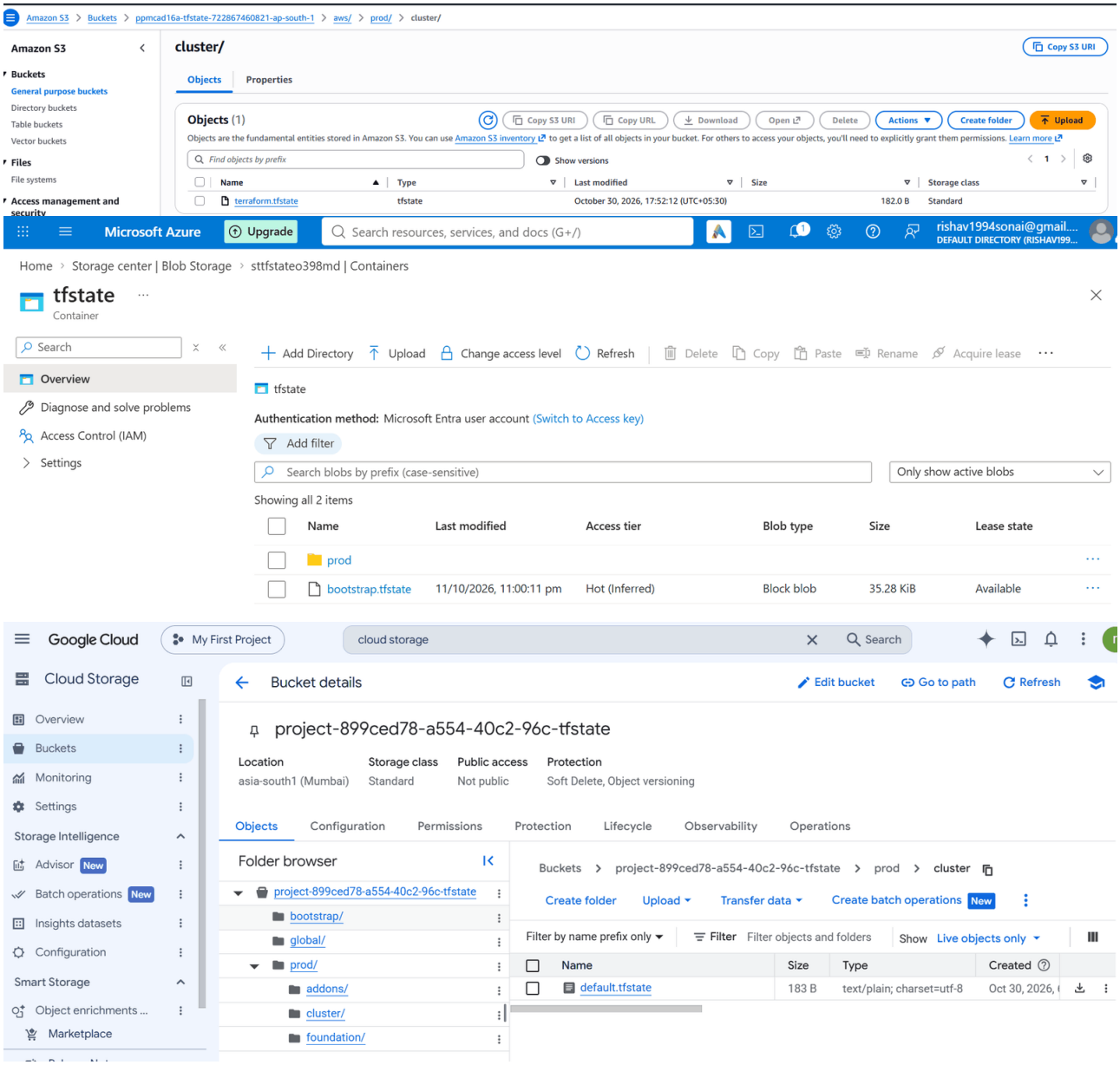
Encrypted backups in three clouds



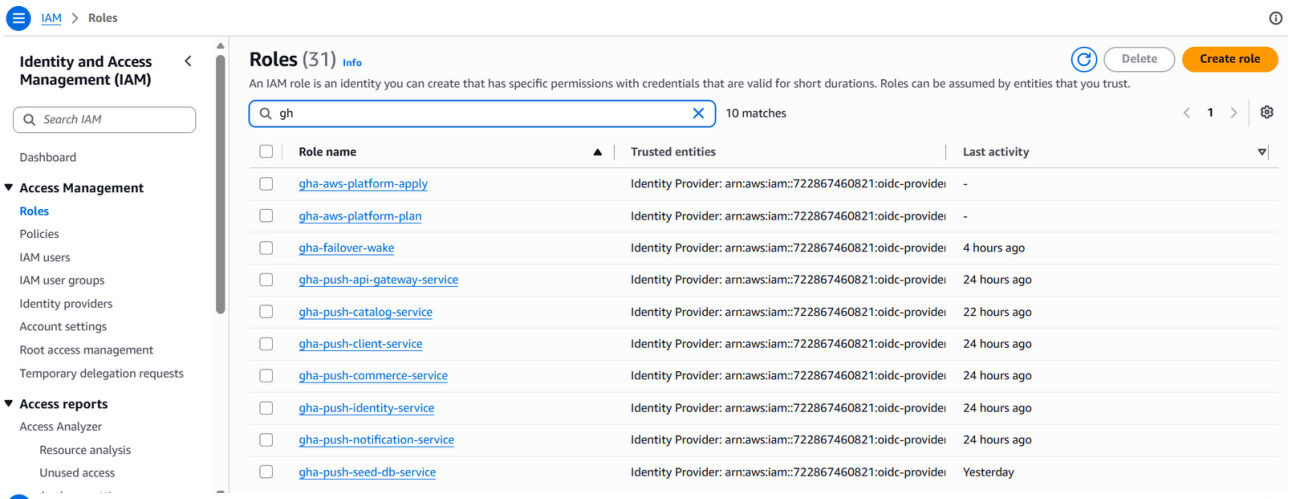


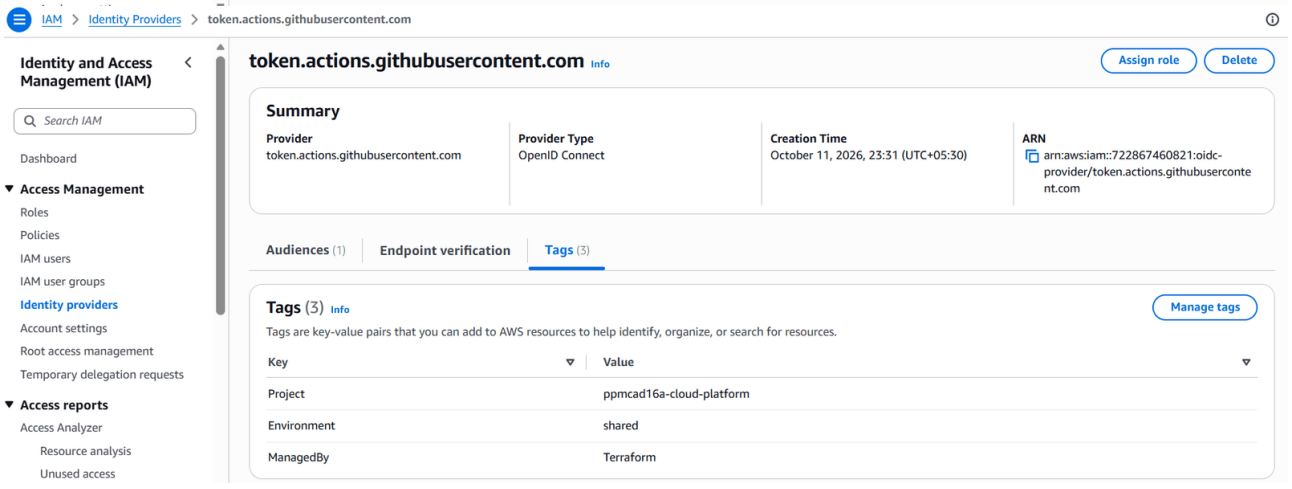


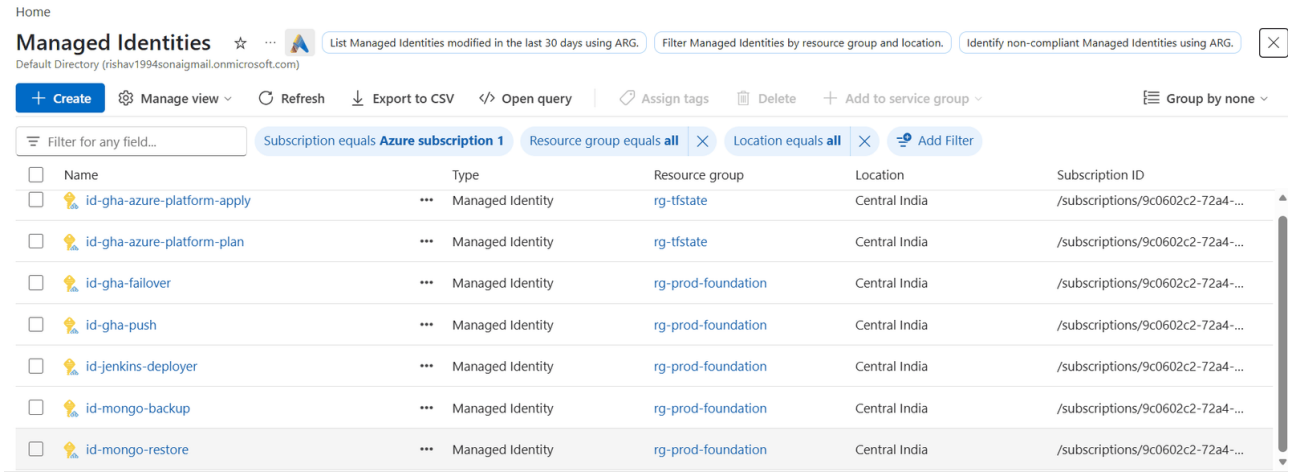
Terraform state (versioned)

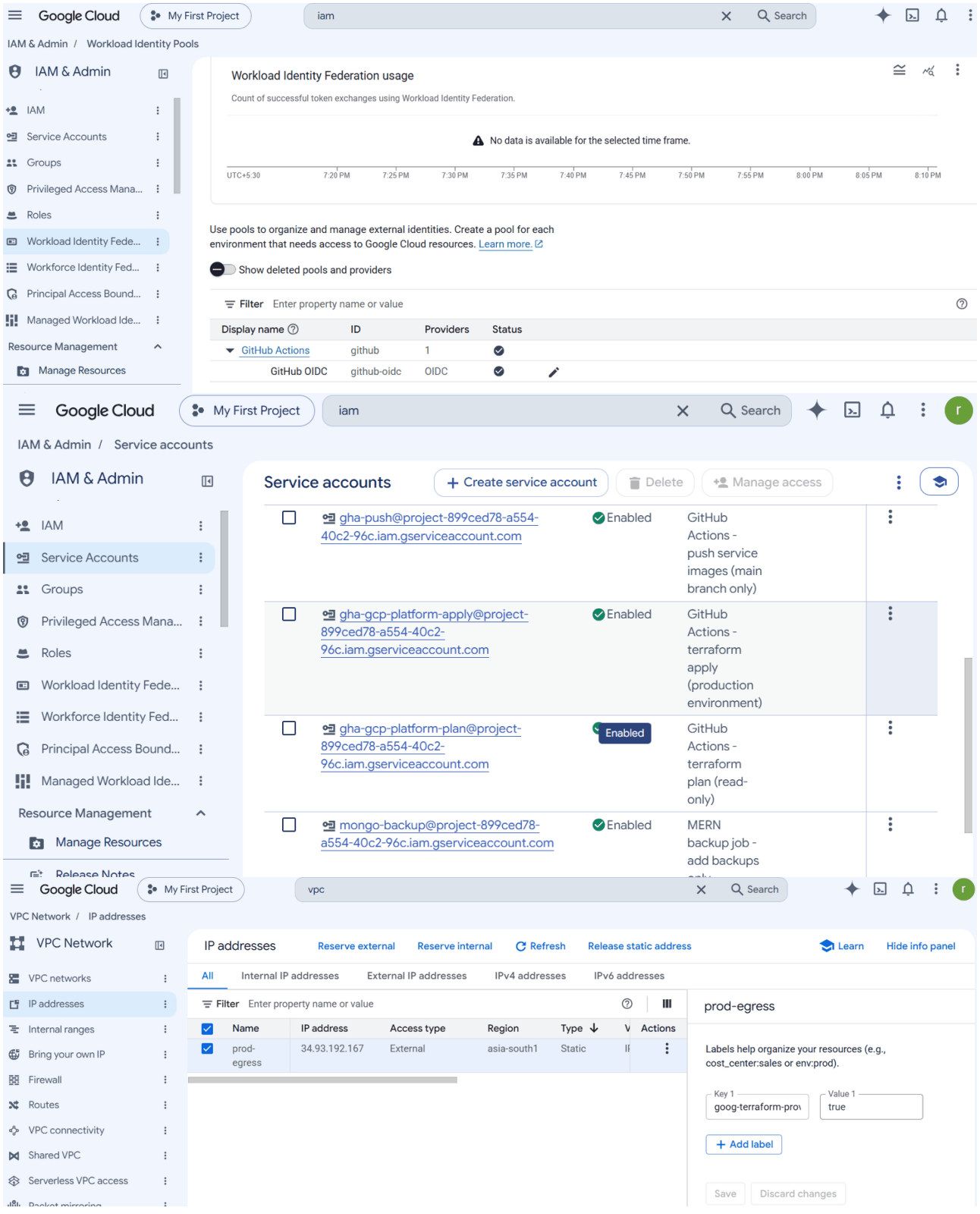


Keyless identities

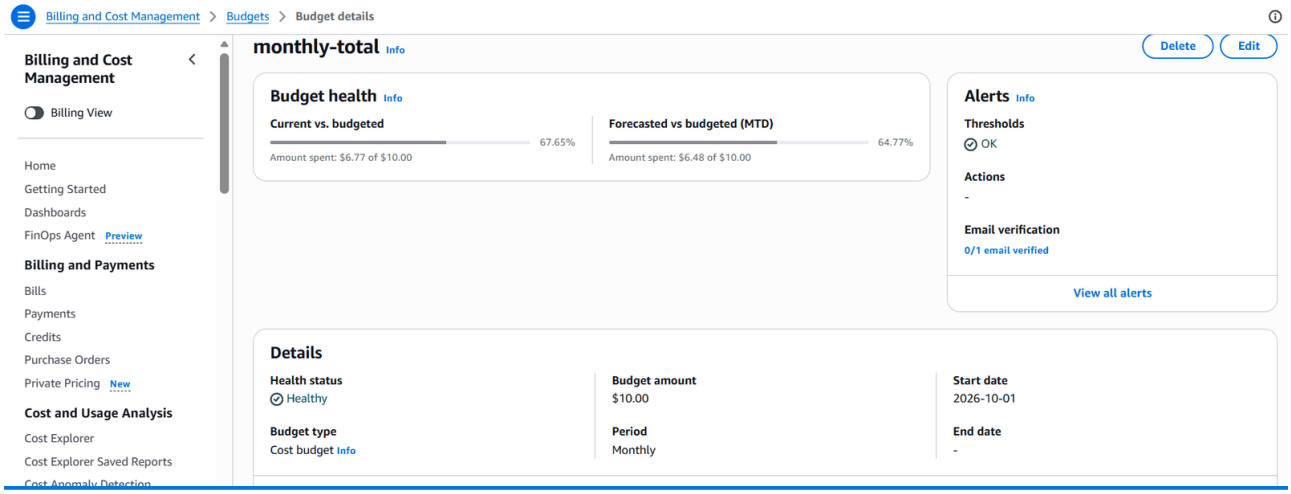

IAM > Roles
Identity and Access Management (IAM)
Search IAM
Dashboard
Access Management
Roles
Policies
IAM users
IAM user groups
Identity providers
Account settings
Root access management
Temporary delegation requests
Access reports
Access Analyzer
Resource analysis
Unused access
Roles (31) Info
An IAM role is an identity you can create that has specific permissions with credentials that are valid for short durations. Roles can be assumed by entities that you trust.
Delete
Create role
gh
10 matches
1
Role name
Trusted entities
Last activity
gha-aws-platform-apply
Identity Provider: arn:aws:iam::722867460821:oidc-provide
-
gha-aws-platform-plan
Identity Provider: arn:aws:iam::722867460821:oidc-provide
-
gha-failover-wake
Identity Provider: arn:aws:iam::722867460821:oidc-provide
4 hours ago
gha-push-api-gateway-service
Identity Provider: arn:aws:iam::722867460821:oidc-provide
24 hours ago
gha-push-catalog-service
Identity Provider: arn:aws:iam::722867460821:oidc-provide
22 hours ago
gha-push-client-service
Identity Provider: arn:aws:iam::722867460821:oidc-provide
24 hours ago
gha-push-commerce-service
Identity Provider: arn:aws:iam::722867460821:oidc-provide
24 hours ago
gha-push-identity-service
Identity Provider: arn:aws:iam::722867460821:oidc-provide
24 hours ago
gha-push-notification-service
Identity Provider: arn:aws:iam::722867460821:oidc-provide
24 hours ago
gha-push-seed-db-service
Identity Provider: arn:aws:iam::722867460821:oidc-provide
Yesterday

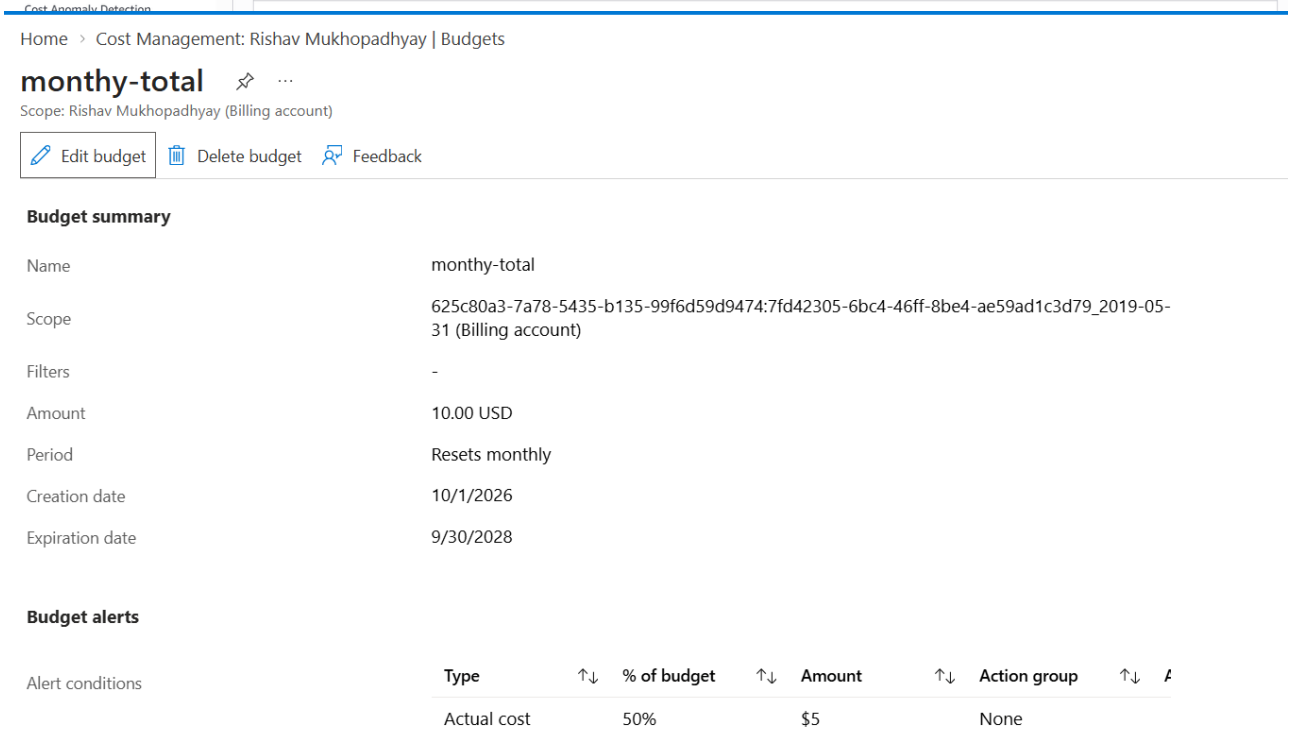

IAM > Identity Providers > token.actions.githubusercontent.com
Identity and Access Management (IAM)
Search IAM
Dashboard
Access Management
Roles
Policies
IAM users
IAM user groups
Identity providers
Account settings
Root access management
Temporary delegation requests
Access reports
Access Analyzer
Resource analysis
Unused access
token.actions.githubusercontent.com Info
Assign role
Delete
Summary
Provider
token.actions.githubusercontent.com
Provider Type
OpenID Connect
Creation Time
October 11, 2026, 23:31 (UTC+05:30)
ARN
arn:aws:iam::722867460821:oidc-provider/token.actions.githubusercontent.com
Audiences (1)
Endpoint verification
Tags (3)
Tags (3) Info
Manage tags
Tags are key-value pairs that you can add to AWS resources to help identify, organize, or search for resources.
Key
Value
Project
ppmcad16a-cloud-platform
Environment
shared
ManagedBy
Terraform

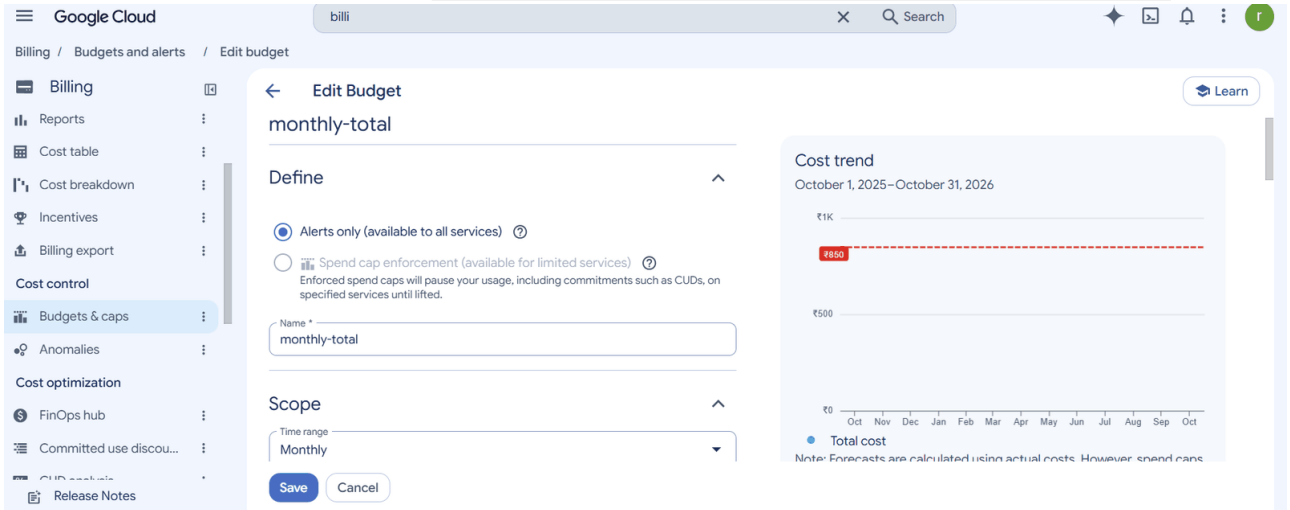

Home
Managed Identities
Default Directory (rishav1994sonaigmail.onmicrosoft.com)
List Managed Identities modified in the last 30 days using ARG.
Filter Managed Identities by resource group and location.
Identify non-compliant Managed Identities using ARG.
Create
Manage view
Refresh
Export to CSV
Open query
Assign tags
Delete
Add to service group
Group by none
Filter for any field...
Subscription equals Azure subscription 1
Resource group equals all
Location equals all
Add Filter
Name
Type
Resource group
Location
Subscription ID
id-gha-azure-platform-apply
Managed Identity
rg-tfstate
Central India
/subscriptions/9c0602c2-72a4-...
id-gha-azure-platform-plan
Managed Identity
rg-tfstate
Central India
/subscriptions/9c0602c2-72a4-...
id-gha-failover
Managed Identity
rg-prod-foundation
Central India
/subscriptions/9c0602c2-72a4-...
id-gha-push
Managed Identity
rg-prod-foundation
Central India
/subscriptions/9c0602c2-72a4-...
id-jenkins-deployer
Managed Identity
rg-prod-foundation
Central India
/subscriptions/9c0602c2-72a4-...
id-mongo-backup
Managed Identity
rg-prod-foundation
Central India
/subscriptions/9c0602c2-72a4-...
id-mongo-restore
Managed Identity
rg-prod-foundation
Central India
/subscriptions/9c0602c2-72a4-...


Google Cloud
My First Project
iam
Search
IAM & Admin / Workload Identity Pools
IAM & Admin
IAM
Service Accounts
Groups
Privileged Access Mana...
Roles
Workload Identity Fede...
Workforce Identity Fed...
Principal Access Bound...
Managed Workload Ide...
Resource Management
Manage Resources
Workload Identity Federation usage
Count of successful token exchanges using Workload Identity Federation.
No data is available for the selected time frame.
UTC+5:30
7:20 PM
7:25 PM
7:30 PM
7:35 PM
7:40 PM
7:45 PM
7:50 PM
7:55 PM
8:00 PM
8:05 PM
8:10 PM
Use pools to organize and manage external identities. Create a pool for each environment that needs access to Google Cloud resources. Learn more.
Show deleted pools and providers
Filter Enter property name or value
Display name
ID
Providers
Status
GitHub Actions
github
1
GitHub OIDC
github-oidc
OIDC
IAM & Admin / Service accounts
Service accounts
+ Create service account
Delete
Manage access
gha-push@project-899ced78-a554-40c2-96c.iam.gserviceaccount.com
Enabled
GitHub Actions - push service images (main branch only)
gha-gcp-platform-apply@project-899ced78-a554-40c2-96c.iam.gserviceaccount.com
Enabled
GitHub Actions - terraform apply (production environment)
gha-gcp-platform-plan@project-899ced78-a554-40c2-96c.iam.gserviceaccount.com
Enabled
GitHub Actions - terraform plan (read-only)
mongo-backup@project-899ced78-a554-40c2-96c.iam.gserviceaccount.com
Enabled
MERN backup job - add backups
vpc
VPC Network / IP addresses
VPC Network
VPC networks
IP addresses
Internal ranges
Bring your own IP
Firewall
Routes
VPC connectivity
Shared VPC
Serverless VPC access
IP addresses
Reserve external
Reserve internal
Refresh
Release static address
Learn
Hide info panel
All
Internal IP addresses
External IP addresses
IPv4 addresses
IPv6 addresses
Filter Enter property name or value
Name
IP address
Access type
Region
Type
Actions
prod-egress
34.93.192.167
External
asia-south1
Static
prod-egress
Labels help organize your resources (e.g., cost_center:sales or env:prod).
Key 1
goog-terraform-pro
Value 1
true
+ Add label
Save
Discard changes

Budgets







B. Session start (`up.yml`)

`ansible-playbook playbooks/up.yml -e keep_awake=true` (standbys kept awake for the photos).

```
(ansible) dell@DESKTOP-LSIMSKL:/mnt/c/Users/dell/Documents/HeroVired/Capstone/repos/platform-automation$ time ansible-playbook playbooks/up.yml

PLAY [Preflight — is everything ready?] ************************************

TASK [Terraform is the pinned version] *************************************
ok: [localhost]

TASK [A folder for Terraform's provider cache (faster init in every stack)] ****
ok: [localhost]

TASK [Logged in to Google Cloud?] ******************************************
ok: [localhost]

TASK [Logged in to Azure?] *************************************************
ok: [localhost]

TASK [Logged in to AWS?] ***************************************************
ok: [localhost]

TASK [All three logins are valid] ******************************************
ok: [localhost] =>
    changed: false
    msg: Logged in to GCP, Azure (subscription 9c0602c2-72a4-4432-a192-7abf998e2b60) and
        AWS (722867460821)

TASK [Your public IP] ******************************************************
ok: [localhost]

TASK [Read admin_cidr from every cluster/add-ons stack] ********************
ok: [localhost] => (item=['gcp', 'cluster'])
ok: [localhost] => (item=['gcp', 'addons'])
ok: [localhost] => (item=['azure', 'cluster'])
ok: [localhost] => (item=['azure', 'addons'])
ok: [localhost] => (item=['aws', 'cluster'])
ok: [localhost] => (item=['aws', 'addons'])

TASK [admin_cidr everywhere = your IP] *************************************
ok: [localhost] => (item=gcp/cluster)
ok: [localhost] => (item=gcp/addons)
ok: [localhost] => (item=azure/cluster)
ok: [localhost] => (item=azure/addons)
ok: [localhost] => (item=aws/cluster)
ok: [localhost] => (item=aws/addons)

TASK [Ask for the Cloudflare token (input hidden) — unless CLOUDFLARE_API_TOKEN is set] ***
skipping: [localhost]

TASK [Ask for the Atlas public key — unless MONGODB_ATLAS_PUBLIC_KEY is set] ***
skipping: [localhost]

TASK [Where everything is] *************************************************
ok: [localhost] =>
    msg:
    - 'Shop:    https://rishavmukhopadhyay.store  (gcp. / azure. / aws. for one cloud)'
    - 'Jenkins: https://jenkins.rishavmukhopadhyay.store   password: cd gcp-platform/environments/prod/addons;
        terraform output -raw jenkins_admin_password'
    - 'Grafana: https://grafana.rishavmukhopadhyay.store   password: cd gcp-platform/environments/prod/addons;
        terraform output -raw grafana_admin_password'
    - 'End of session: ansible-playbook playbooks/down.yml'

PLAY RECAP *****************************************************************
aws                        : ok=27   changed=7    unreachable=0    failed=0    skipped=5    rescued=0    ignored=0
azure                      : ok=27   changed=7    unreachable=0    failed=0    skipped=5    rescued=0    ignored=0
gcp                        : ok=24   changed=6    unreachable=0    failed=0    skipped=3    rescued=0    ignored=0
localhost                  : ok=21   changed=2    unreachable=0    failed=0    skipped=4    rescued=0    ignored=0


real    23m13.006s
user    6m18.811s
sys     1m0.538s
```

C. Everything running

Inside the clusters

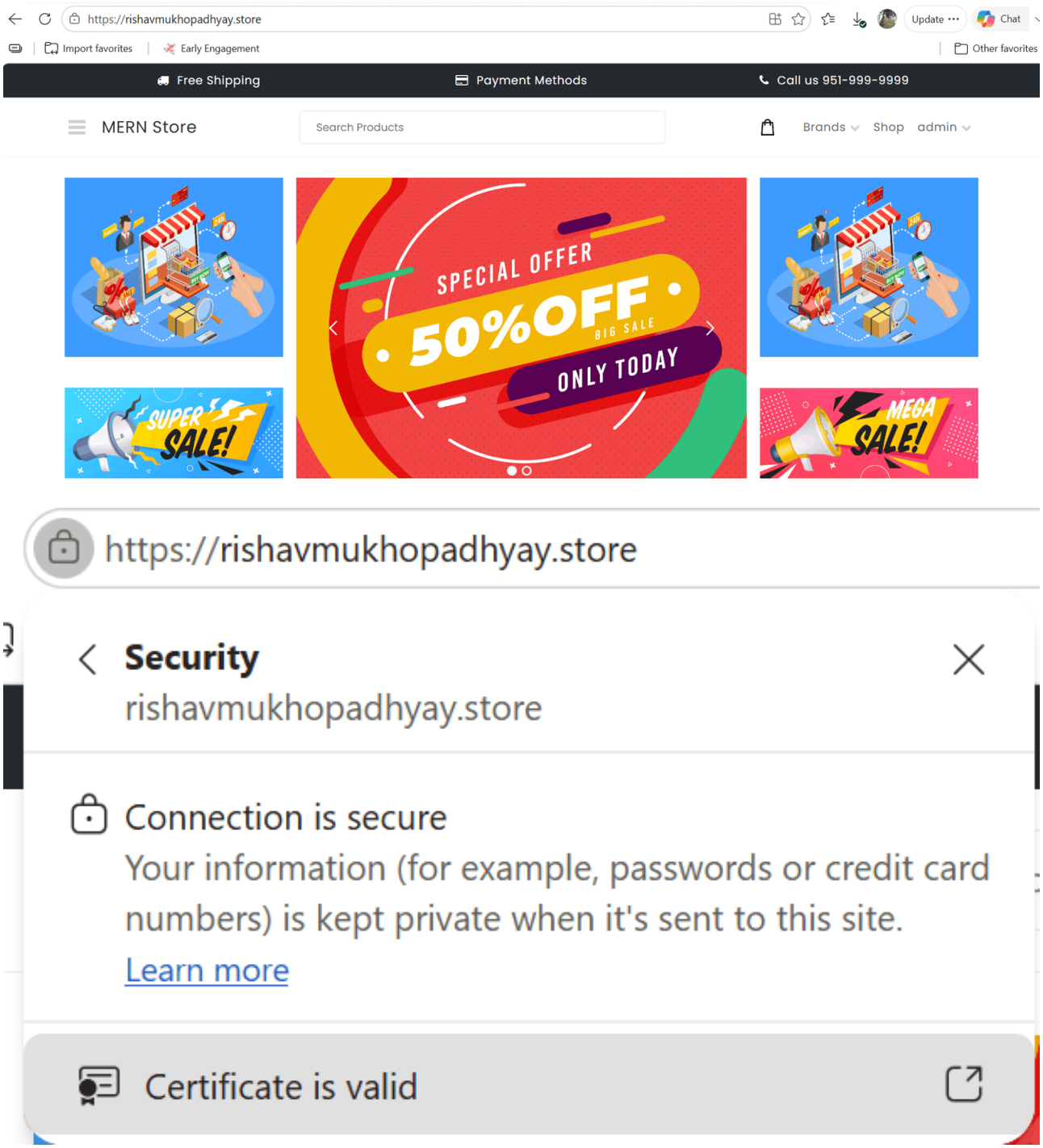
```
(ansible) dell@DESKTOP-LSIMSKL:/mnt/c/Users/dell/Documents/HeroVired/Capstone/repos/platform-automation$ kubectl --context $CTX get nodes -o wide
NAME                                  STATUS   ROLES    AGE   VERSION               INTERNAL-IP   EXTERNAL-IP   OS-IMAGE                             KERNEL-VERSION      CONTAINER-RUNTIME
gke-prod-gke-default-2ed5f0fd-dzb9    Ready    <none>   33m   v1.36.4-gke.2046003   10.30.0.5     <none>        Container-Optimized OS from Google   6.12.105+ (amd64)   containerd://2.2.7
gke-prod-gke-default-2ed5f0fd-gvvg    Ready    <none>   33m   v1.36.4-gke.2046003   10.30.0.4     <none>        Container-Optimized OS from Google   6.12.105+ (amd64)   containerd://2.2.7
(ansible) dell@DESKTOP-LSIMSKL:/mnt/c/Users/dell/Documents/HeroVired/Capstone/repos/platform-automation$ kubectl --context $CTX get pods -n mern -o wide
NAME                                  READY   STATUS    RESTARTS   AGE     IP             NODE                                  NOMINATED NODE   READINESS GATES
api-gateway-service-97fddb575-276dk   1/1     Running   0          7m16s   10.30.128.22   gke-prod-gke-default-2ed5f0fd-gvvg    <none>           <none>
api-gateway-service-97fddb575-6qnlr   1/1     Running   0          7m1s    10.30.129.25   gke-prod-gke-default-2ed5f0fd-dzb9    <none>           <none>
api-gateway-service-97fddb575-hmckj   1/1     Running   0          7m58s   10.30.129.22   gke-prod-gke-default-2ed5f0fd-dzb9    <none>           <none>
api-gateway-service-97fddb575-xmb4l   1/1     Running   0          7m43s   10.30.128.21   gke-prod-gke-default-2ed5f0fd-gvvg    <none>           <none>
catalog-service-7bbbc89fcf-97v6l      1/1     Running   0          8m3s    10.30.128.18   gke-prod-gke-default-2ed5f0fd-gvvg    <none>           <none>
catalog-service-7bbbc89fcf-kdtnc      1/1     Running   0          7m48s   10.30.129.24   gke-prod-gke-default-2ed5f0fd-dzb9    <none>           <none>
client-service-86b4c758d4-n5nlk       1/1     Running   0          7m46s   10.30.128.20   gke-prod-gke-default-2ed5f0fd-gvvg    <none>           <none>
client-service-86b4c758d4-zs4qj       1/1     Running   0          8m1s    10.30.129.21   gke-prod-gke-default-2ed5f0fd-dzb9    <none>           <none>
commerce-service-56675b6f7f-jtd6n     1/1     Running   0          8m3s    10.30.128.17   gke-prod-gke-default-2ed5f0fd-gvvg    <none>           <none>
commerce-service-56675b6f7f-qqb94     1/1     Running   0          7m48s   10.30.129.23   gke-prod-gke-default-2ed5f0fd-dzb9    <none>           <none>
identity-service-684846b4f7-8gs8p     1/1     Running   0          8m4s    10.30.129.20   gke-prod-gke-default-2ed5f0fd-dzb9    <none>           <none>
identity-service-684846b4f7-sjvbr     1/1     Running   0          6m46s   10.30.128.23   gke-prod-gke-default-2ed5f0fd-gvvg    <none>           <none>
identity-service-684846b4f7-z5kz2     1/1     Running   0          7m48s   10.30.128.19   gke-prod-gke-default-2ed5f0fd-gvvg    <none>           <none>
notification-service-d7bbbd64f-wnvnl  1/1     Running   0          8m4s    10.30.129.19   gke-prod-gke-default-2ed5f0fd-dzb9    <none>           <none>
(ansible) dell@DESKTOP-LSIMSKL:/mnt/c/Users/dell/Documents/HeroVired/Capstone/repos/platform-automation$ CTX=$AKS
(ansible) dell@DESKTOP-LSIMSKL:/mnt/c/Users/dell/Documents/HeroVired/Capstone/repos/platform-automation$ kubectl --context $CTX get nodes -o wide
NAME                                STATUS   ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION           CONTAINER-RUNTIME
aks-system-18156796-vmss000000      Ready    <none>   43m   v1.36.5   10.20.0.4     <none>        Ubuntu 24.04.5 LTS   6.8.0-1071-azure (amd64)   containerd://2.3.7-3
aks-system-18156796-vmss000001      Ready    <none>   43m   v1.36.5   10.20.0.5     <none>        Ubuntu 24.04.5 LTS   6.8.0-1071-azure (amd64)   containerd://2.3.7-3
(ansible) dell@DESKTOP-LSIMSKL:/mnt/c/Users/dell/Documents/HeroVired/Capstone/repos/platform-automation$ kubectl --context $CTX get pods -n mern -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP             NODE                                NOMINATED NODE   READINESS GATES
api-gateway-service-5cfdffb86f-gttv6   1/1     Running   0          13m   10.244.1.167   aks-system-18156796-vmss000000   <none>           <none>
api-gateway-service-5cfdffb86f-v4drh   1/1     Running   0          12m   10.244.0.61    aks-system-18156796-vmss000001   <none>           <none>
catalog-service-5ddcbbc4b5-2lctv    1/1     Running   0          13m   10.244.1.237   aks-system-18156796-vmss000000   <none>           <none>
catalog-service-5ddcbbc4b5-7t68g    1/1     Running   0          12m   10.244.0.135   aks-system-18156796-vmss000001   <none>           <none>
client-service-598f96b9f8-pbxst     1/1     Running   0          12m   10.244.0.64    aks-system-18156796-vmss000001   <none>           <none>
client-service-598f96b9f8-z9czq     1/1     Running   0          13m   10.244.1.219   aks-system-18156796-vmss000000   <none>           <none>
commerce-service-6f95587f5-fxtmf    1/1     Running   0          12m   10.244.1.215   aks-system-18156796-vmss000000   <none>           <none>
commerce-service-6f95587f5-w4bk6    1/1     Running   0          13m   10.244.0.244   aks-system-18156796-vmss000001   <none>           <none>
identity-service-54749dddf9-4hr9h   1/1     Running   0          12m   10.244.1.25    aks-system-18156796-vmss000000   <none>           <none>
identity-service-54749dddf9-9z8pc   1/1     Running   0          13m   10.244.0.6     aks-system-18156796-vmss000001   <none>           <none>
notification-service-7596c4f48c-rpgcv   1/1     Running   0          13m   10.244.1.64    aks-system-18156796-vmss000000   <none>           <none>
(ansible) dell@DESKTOP-LSIMSKL:/mnt/c/Users/dell/Documents/HeroVired/Capstone/repos/platform-automation$ CTX=$EKS
(ansible) dell@DESKTOP-LSIMSKL:/mnt/c/Users/dell/Documents/HeroVired/Capstone/repos/platform-automation$ kubectl --context $CTX get nodes -o wide
NAME                                          STATUS   ROLES    AGE   VERSION               INTERNAL-IP    EXTERNAL-IP   OS-IMAGE                         KERNEL-VERSION                      CONTAINER-RUN
TIME
ip-10-10-20-146.ap-south-1.compute.internal   Ready    <none>   23m   v1.37.1-eks-577a51e   10.10.20.146   <none>        Amazon Linux 2023.12.20261026   6.18.56-125.164.amzn2023.x86_64 (amd64)   containerd://
2.3.7+unknown
ip-10-10-32-84.ap-south-1.compute.internal    Ready    <none>   23m   v1.37.1-eks-577a51e   10.10.32.84    <none>        Amazon Linux 2023.12.20261026   6.18.56-125.164.amzn2023.x86_64 (amd64)   containerd://
2.3.7+unknown
(ansible) dell@DESKTOP-LSIMSKL:/mnt/c/Users/dell/Documents/HeroVired/Capstone/repos/platform-automation$ kubectl --context $CTX get pods -n mern -o wide
NAME                                  READY   STATUS    RESTARTS   AGE   IP             NODE                                          NOMINATED NODE   READINESS GATES
api-gateway-service-6fd8dd6455-dxldw  1/1     Running   0          14m   10.10.33.142   ip-10-10-32-84.ap-south-1.compute.internal    <none>           <none>
api-gateway-service-6fd8dd6455-pxzcf  1/1     Running   0          14m   10.10.27.201   ip-10-10-20-146.ap-south-1.compute.internal   <none>           <none>
catalog-service-c88d54b99-hz6hr       1/1     Running   0          14m   10.10.19.248   ip-10-10-20-146.ap-south-1.compute.internal   <none>           <none>
catalog-service-c88d54b99-xjgqk       1/1     Running   0          14m   10.10.41.24    ip-10-10-32-84.ap-south-1.compute.internal    <none>           <none>
client-service-d8dddd44b-2g2lr        1/1     Running   0          14m   10.10.29.102   ip-10-10-20-146.ap-south-1.compute.internal   <none>           <none>
client-service-d8dddd44b-x7kj5        1/1     Running   0          14m   10.10.45.227   ip-10-10-32-84.ap-south-1.compute.internal    <none>           <none>
commerce-service-5969d48df7-f4j7m     1/1     Running   0          14m   10.10.23.129   ip-10-10-20-146.ap-south-1.compute.internal   <none>           <none>
commerce-service-5969d48df7-ftvsv     1/1     Running   0          14m   10.10.34.136   ip-10-10-32-84.ap-south-1.compute.internal    <none>           <none>
identity-service-678f8488f-lsm5d      1/1     Running   0          14m   10.10.43.237   ip-10-10-32-84.ap-south-1.compute.internal    <none>           <none>
identity-service-678f8488f-zlmq5      1/1     Running   0          14m   10.10.24.12    ip-10-10-20-146.ap-south-1.compute.internal   <none>           <none>
notification-service-69f4458c89-d9xc9 1/1     Running   0          14m   10.10.29.169   ip-10-10-20-146.ap-south-1.compute.internal   <none>           <none>
(ansible) dell@DESKTOP-LSIMSKL:/mnt/c/Users/dell/Documents/HeroVired/Capstone/repos/platform-automation$ kubectl --context $GKE get deploy,hpa,pdb -n mern
NAME                                      READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/api-gateway-service       4/4     4            4           8m34s
deployment.apps/catalog-service           2/2     2            2           8m39s
deployment.apps/client-service            2/2     2            2           8m37s
deployment.apps/commerce-service          2/2     2            2           8m40s
deployment.apps/identity-service          3/3     3            3           8m40s
deployment.apps/notification-service      1/1     1            1           8m40s

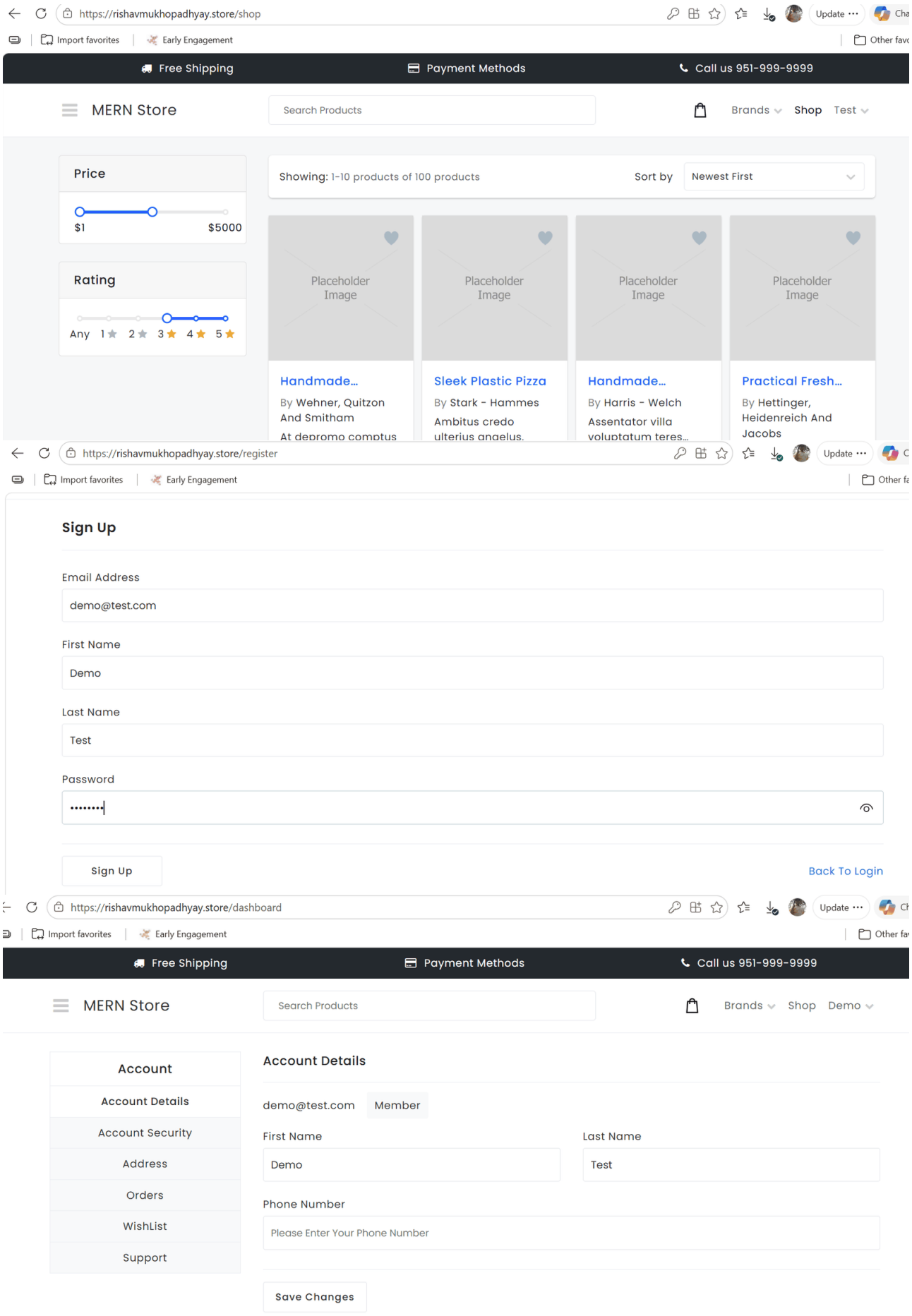
NAME                                                                    REFERENCE                                 TARGETS          MINPODS   MAXPODS   REPLICAS   AGE
horizontalpodautoscaler.autoscaling/api-gateway-service   Deployment/api-gateway-service   cpu: 84%/70%   2         4         4          8m34s
horizontalpodautoscaler.autoscaling/catalog-service       Deployment/catalog-service       cpu: 39%/70%   2         4         2          8m39s
horizontalpodautoscaler.autoscaling/client-service        Deployment/client-service        cpu: 0%/70%    2         4         2          8m37s
horizontalpodautoscaler.autoscaling/commerce-service      Deployment/commerce-service      cpu: 57%/70%   2         4         2          8m39s
horizontalpodautoscaler.autoscaling/identity-service      Deployment/identity-service      cpu: 35%/70%   2         3         3          8m40s

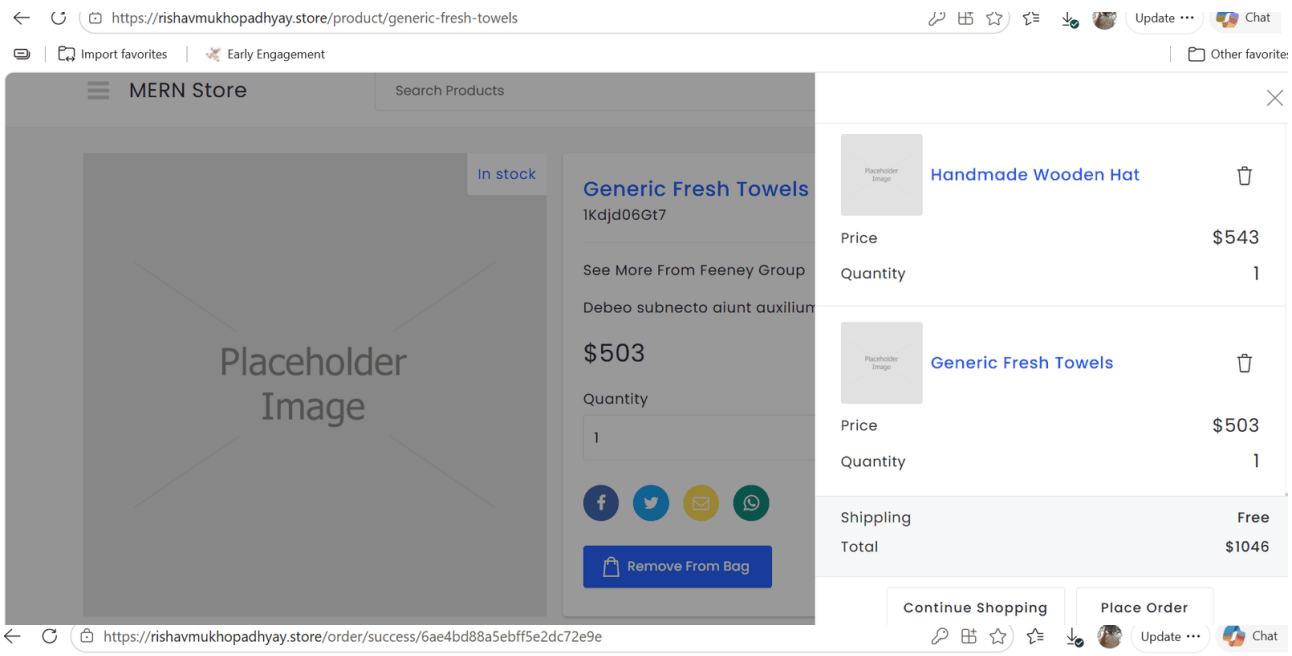
NAME                                                  MIN AVAILABLE   MAX UNAVAILABLE   ALLOWED DISRUPTIONS   AGE
poddisruptionbudget.policy/api-gateway-service   1               N/A               3                     8m34s
poddisruptionbudget.policy/catalog-service       1               N/A               1                     8m40s
poddisruptionbudget.policy/client-service        1               N/A               1                     8m38s
poddisruptionbudget.policy/commerce-service      1               N/A               1                     8m40s
poddisruptionbudget.policy/identity-service      1               N/A               2                     8m40s
(ansible) dell@DESKTOP-LSIMSKL:/mnt/c/Users/dell/Documents/HeroVired/Capstone/repos/platform-automation$ kubectl --context $GKE get externalsecret -n mern
NAME                        STORETYPE            STORE          REFRESH INTERVAL   STATUS         READY   LAST SYNC
catalog-service-secrets     ClusterSecretStore   cloud-vault    1h                 SecretSynced   True    9m2s
commerce-service-secrets    ClusterSecretStore   cloud-vault    1h                 SecretSynced   True    9m2s
identity-service-secrets    ClusterSecretStore   cloud-vault    1h                 SecretSynced   True    9m4s
notification-service-secrets   ClusterSecretStore   cloud-vault    1h                 SecretSynced   True    9m5s
(ansible) dell@DESKTOP-LSIMSKL:/mnt/c/Users/dell/Documents/HeroVired/Capstone/repos/platform-automation$ kubectl --context $GKE get gateway,httproute -A
NAMESPACE            NAME                                                CLASS   ADDRESS          PROGRAMMED   AGE
envoy-gateway-system   gateway.gateway.networking.k8s.io/public   envoy   34.14.173.232   True         15m

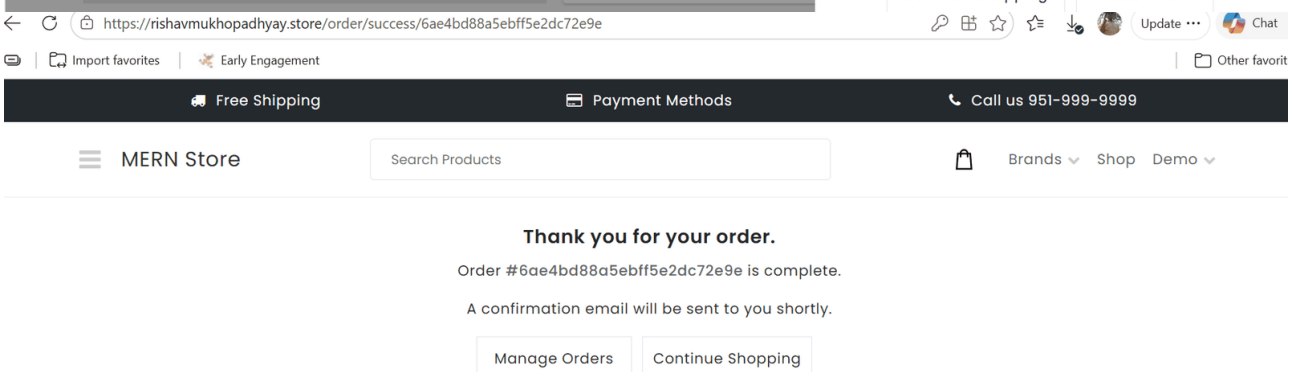
NAMESPACE    NAME                                                HOSTNAMES                                                                AGE
jenkins      httproute.gateway.networking.k8s.io/jenkins                ["jenkins.rishavmukhopadhyay.store"]                                     15m
mern         httproute.gateway.networking.k8s.io/api-gateway-service    ["rishavmukhopadhyay.store","gcp.rishavmukhopadhyay.store"]              9m40s
mern         httproute.gateway.networking.k8s.io/client-service         ["rishavmukhopadhyay.store","gcp.rishavmukhopadhyay.store"]              9m43s
monitoring   httproute.gateway.networking.k8s.io/grafana                ["grafana.rishavmukhopadhyay.store"]                                     15m
(ansible) dell@DESKTOP-LSIMSKL:/mnt/c/Users/dell/Documents/HeroVired/Capstone/repos/platform-automation$ helm --kube-context $GKE list -A
NAME                    NAMESPACE               REVISION        UPDATED                                 STATUS          CHART                           APP VERSION
alloy                   monitoring              1               2026-10-30 15:31:26.675329972 +0000 UTC deployed        alloy-1.13.1                    v1.20.1
api-gateway-service     mern                    1               2026-10-30 15:32:13.266490383 +0000 UTC deployed        microservice-0.1.0              1.0
catalog-service         mern                    1               2026-10-30 15:32:08.793649133 +0000 UTC deployed        microservice-0.1.0              1.0
client-service          mern                    1               2026-10-30 15:32:13.891985777 +0000 UTC deployed        microservice-0.1.0              1.0
commerce-service        mern                    1               2026-10-30 15:32:10.213994663 +0000 UTC deployed        microservice-0.1.0              1.0
envoy-gateway           envoy-gateway-system    1               2026-10-30 15:25:00.626937841 +0000 UTC deployed        gateway-helm-v1.9.2             v1.9.2
external-secrets        external-secrets        1               2026-10-30 15:25:08.398592358 +0000 UTC deployed        external-secrets-2.12.0         v2.12.0
identity-service        mern                    1               2026-10-30 15:32:08.264355155 +0000 UTC deployed        microservice-0.1.0              1.0
jenkins                 jenkins                 1               2026-10-30 15:26:38.385972098 +0000 UTC deployed        jenkins-5.9.68                  2.580.1
kube-prometheus-stack   monitoring              1               2026-10-30 15:26:50.48335956 +0000 UTC  deployed        kube-prometheus-stack-92.6.0    v0.94.1
loki                    monitoring              1               2026-10-30 15:30:03.295397672 +0000 UTC deployed        loki-7.3.0                      3.6.12
mern-observability      monitoring              1               2026-10-30 15:30:00.220248294 +0000 UTC deployed        mern-observability-1.0.0
notification-service    mern                    1               2026-10-30 15:32:11.617843979 +0000 UTC deployed        microservice-0.1.0              1.0
opencost                monitoring              1               2026-10-30 15:30:04.687389746 +0000 UTC deployed        opencost-2.5.32                 1.121.3
platform-config         envoy-gateway-system    1               2026-10-30 15:26:20.34693014 +0000 UTC  deployed        platform-config-0.1.0
(ansible) dell@DESKTOP-LSIMSKL:/mnt/c/Users/dell/Documents/HeroVired/Capstone/repos/platform-automation$ for c in gcp azure aws; do echo "== $c"; curl -s https://$c.rishavmukhopadhyay.store/api/health; echo;
done
== gcp
{"status":"ok","services":{"identity":"up","catalog":"up","commerce":"up","notification":"up"}}
== azure
{"status":"ok","services":{"identity":"up","catalog":"up","commerce":"up","notification":"up"}}
== aws
{"status":"ok","services":{"identity":"up","catalog":"up","commerce":"up","notification":"up"}}
```

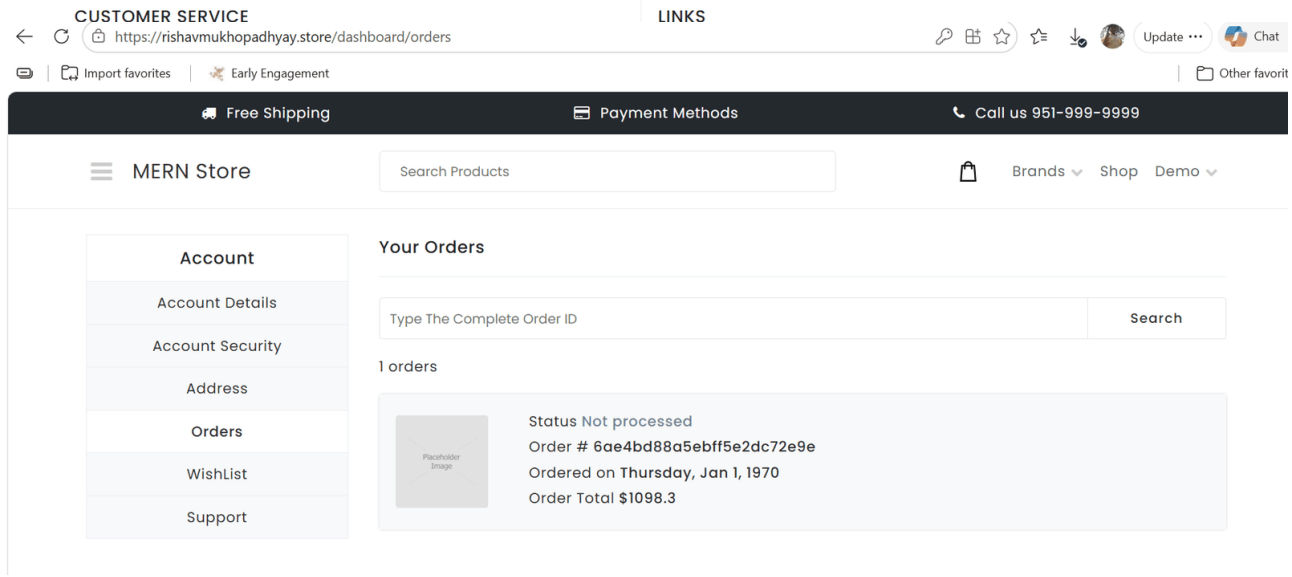
The shop — sign up, browse, cart, order; the same user and order are visible from every cloud (one shared database).

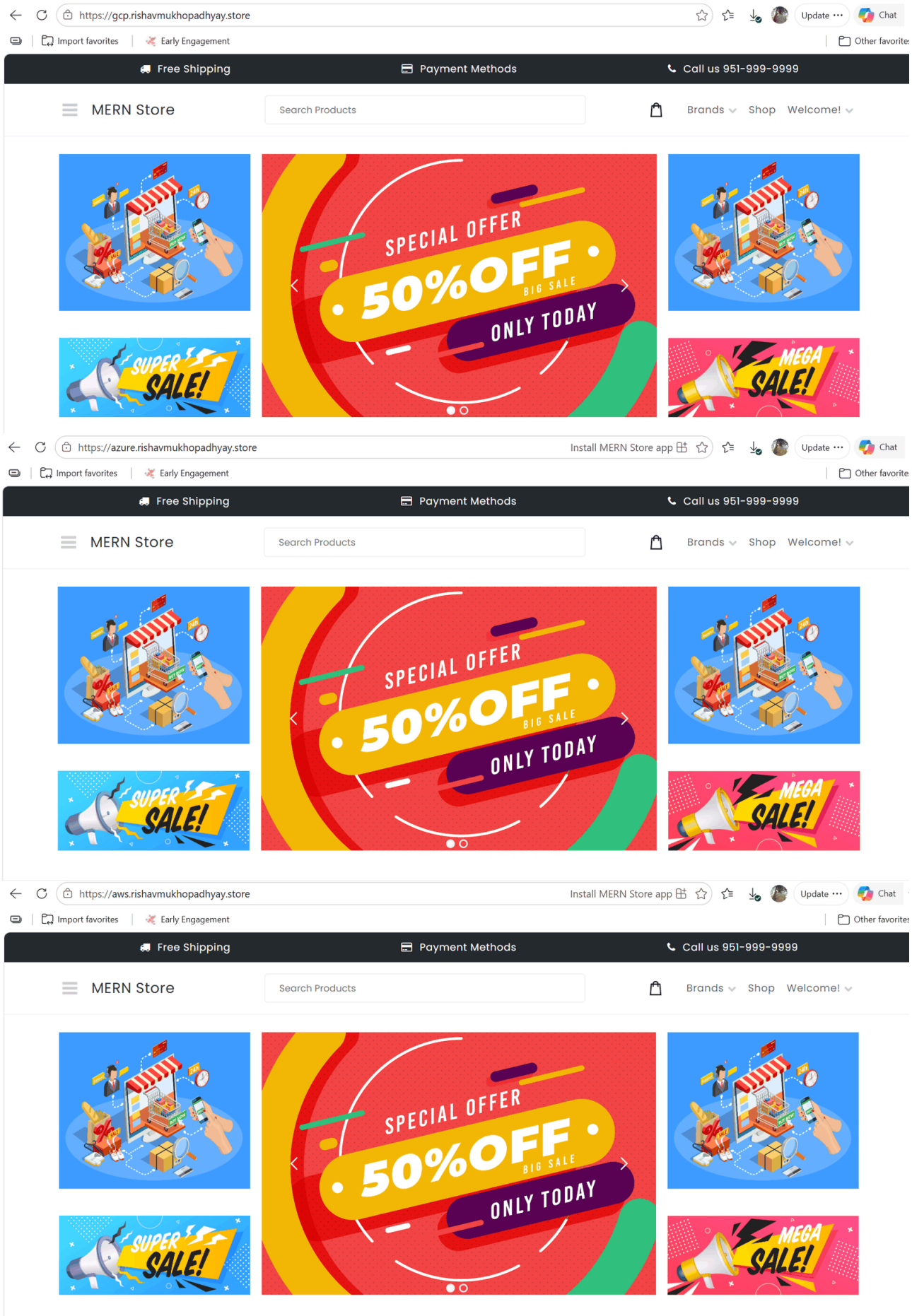



https://rishavmukhopadhyay.store/shop
Import favorites
Early Engagement
Update
Free Shipping
Payment Methods
Call us 951-999-9999
MERN Store
Search Products
Brands
Shop
Test
Price
$1
$5000
Rating
Any 1 2 3 4 5
Showing: 1-10 products of 100 products
Sort by
Newest First
Placeholder Image
Handmade...
By Wehner, Quitzon And Smitham
Sleek Plastic Pizza
By Stark - Hammes
Ambitus credo ulterius angelus.
Handmade...
By Harris - Welch
Assentator villa voluptatum teres...
Practical Fresh...
By Hettinger, Heidenreich And Jacobs
https://rishavmukhopadhyay.store/register
Sign Up
Email Address
demo@test.com
First Name
Demo
Last Name
Test
Password
Sign Up
Back To Login
https://rishavmukhopadhyay.store/dashboard
Free Shipping
Payment Methods
Call us 951-999-9999
MERN Store
Search Products
Brands
Shop
Demo
Account
Account Details
Account Security
Address
Orders
WishList
Support
Account Details
demo@test.com
Member
First Name
Demo
Last Name
Test
Phone Number
Please Enter Your Phone Number
Save Changes

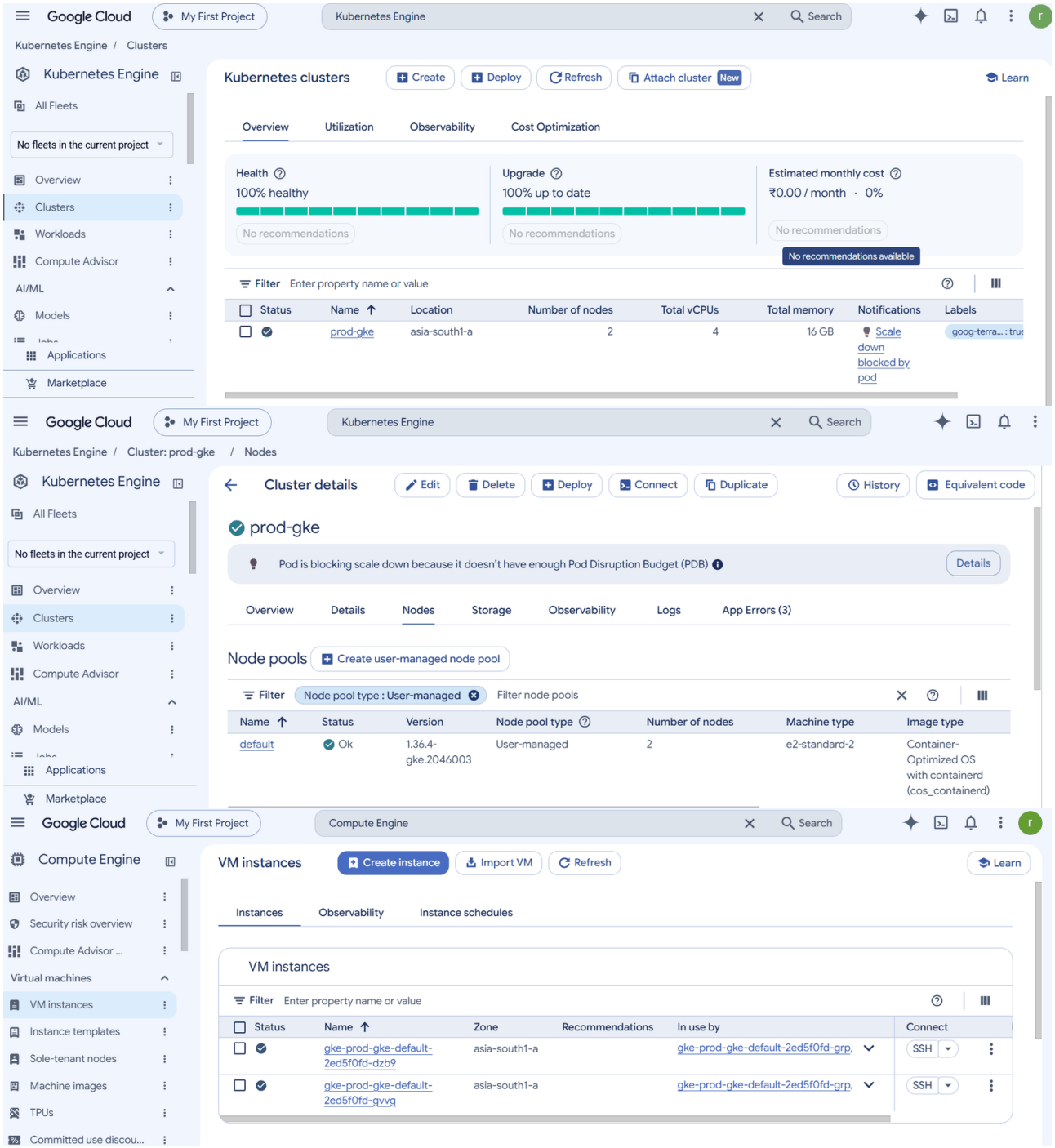

https://rishavmukhopadhyay.store/product/generic-fresh-towels
Update
Chat
Import favorites
Early Engagement
Other favorites
MERN Store
Search Products
In stock
Placeholder Image
Generic Fresh Towels
1Kdjd06Gt7
See More From Feeney Group
Debeo subnecto aiunt auxilium
$503
Quantity
1
Remove From Bag
Handmade Wooden Hat
Price
$543
Quantity
1
Generic Fresh Towels
Price
$503
Quantity
1
Shipping
Free
Total
$1046
Continue Shopping
Place Order

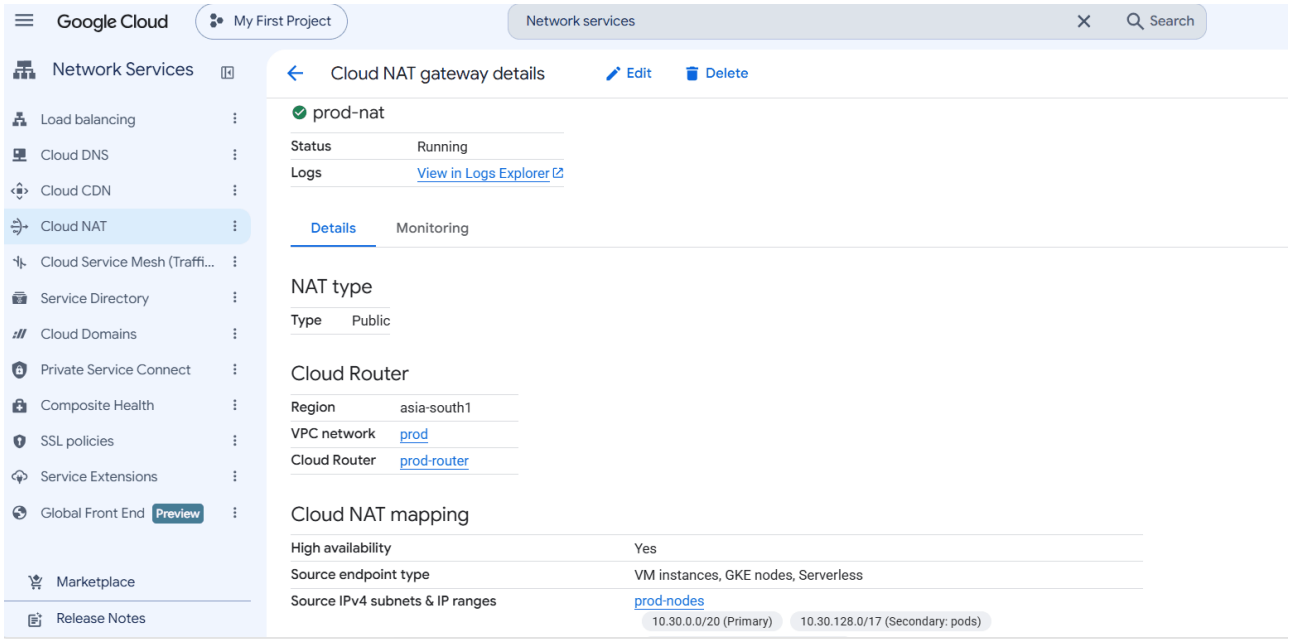

https://rishavmukhopadhyay.store/order/success/6ae4bd88a5ebff5e2dc72e9e
Update
Chat
Import favorites
Early Engagement
Other favorit
Free Shipping
Payment Methods
Call us 951-999-9999
MERN Store
Search Products
Brands
Shop
Demo
Thank you for your order.
Order #6ae4bd88a5ebff5e2dc72e9e is complete.
A confirmation email will be sent to you shortly.
Manage Orders
Continue Shopping

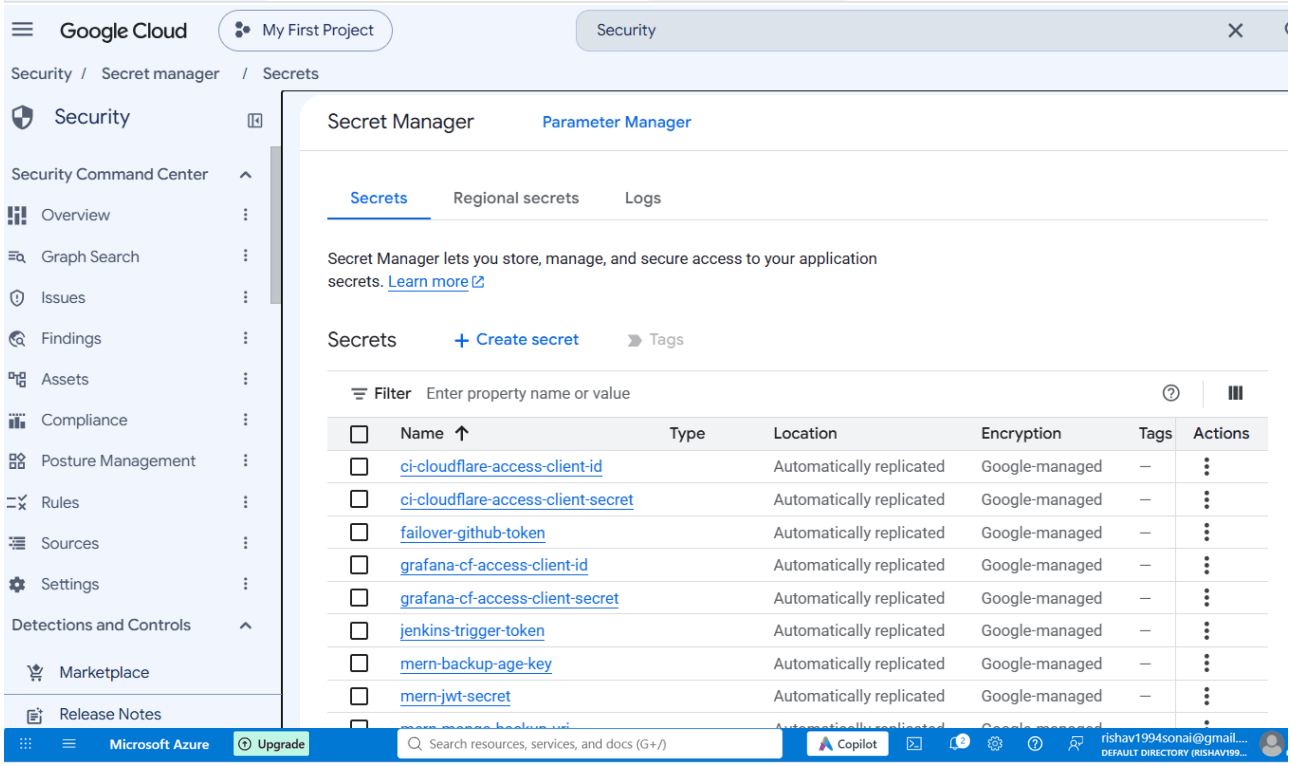

CUSTOMER SERVICE
LINKS
https://rishavmukhopadhyay.store/dashboard/orders
Update
Chat
Import favorites
Early Engagement
Other favorit
Free Shipping
Payment Methods
Call us 951-999-9999
MERN Store
Search Products
Brands
Shop
Demo
Account
Account Details
Account Security
Address
Orders
WishList
Support
Your Orders
Type The Complete Order ID
Search
1 orders
Placeholder Image
Status Not processed
Order # 6ae4bd88a5ebff5e2dc72e9e
Ordered on Thursday, Jan 1, 1970
Order Total $1098.3

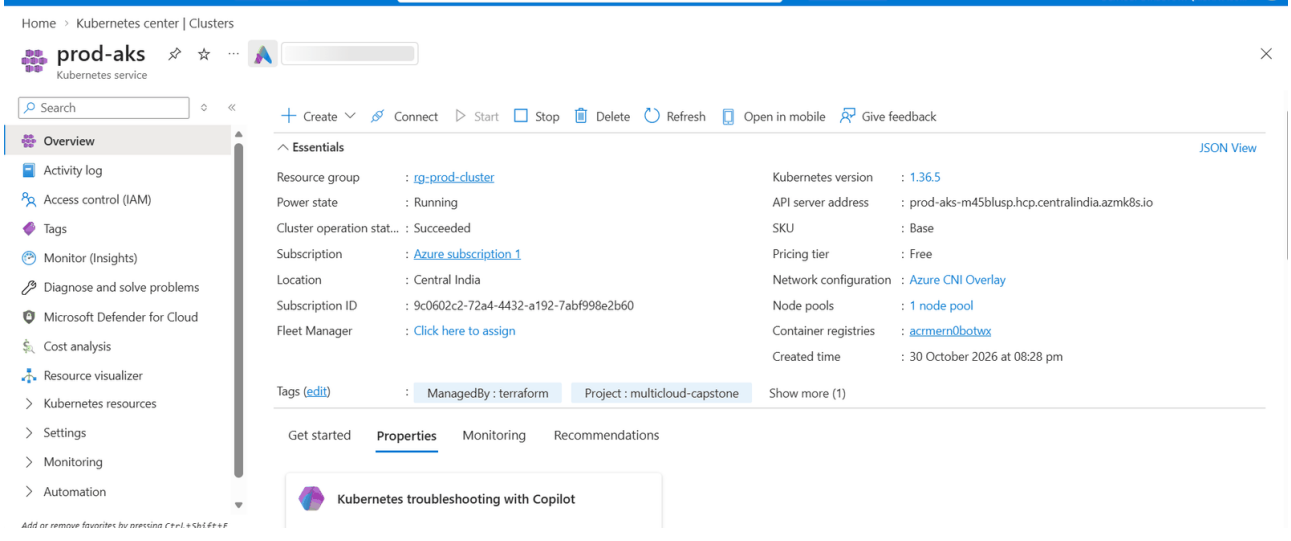


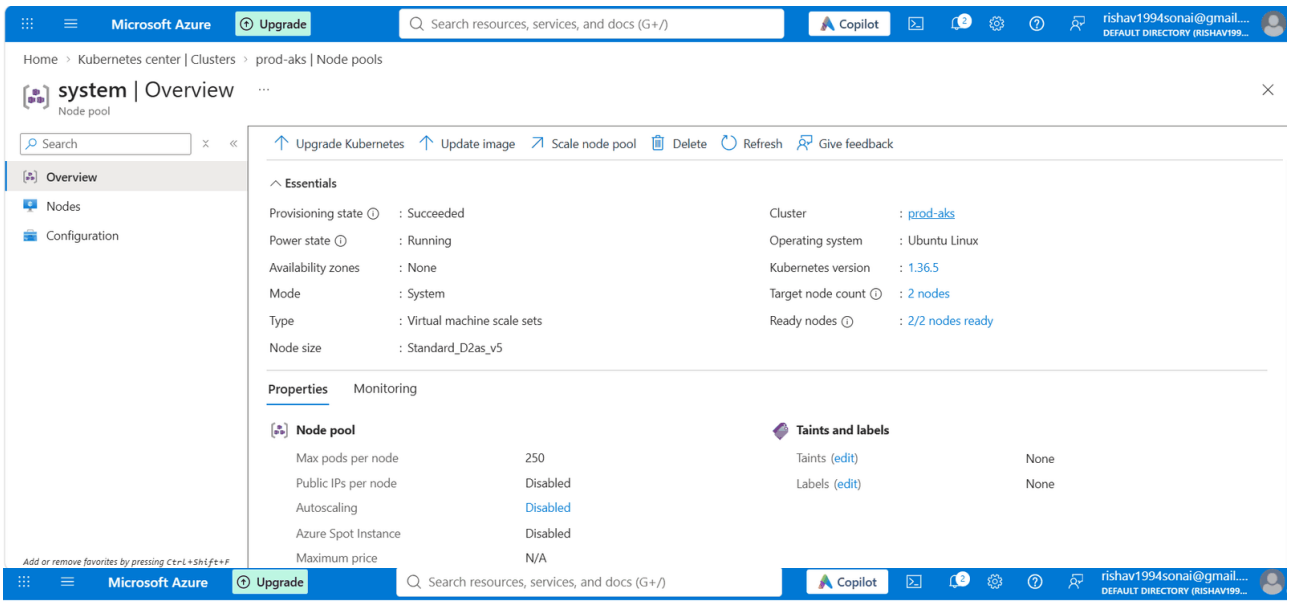
Cloud consoles

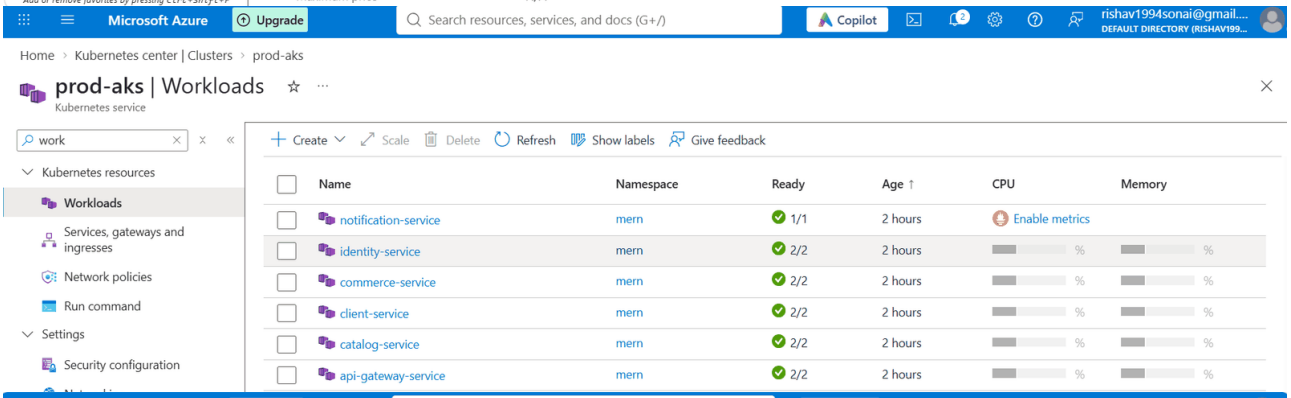

Google Cloud
My First Project
Kubernetes Engine
Search
Kubernetes Engine / Clusters
Kubernetes Engine
All Fleets
No fleets in the current project
Overview
Clusters
Workloads
Compute Advisor
AI/ML
Models
Applications
Marketplace
Kubernetes clusters
Create
Deploy
Refresh
Attach cluster
New
Learn
Overview
Utilization
Observability
Cost Optimization
Health
100% healthy
No recommendations
Upgrade
100% up to date
No recommendations
Estimated monthly cost
₹0.00 / month · 0%
No recommendations
No recommendations available
Filter
Enter property name or value
Status
Name
Location
Number of nodes
Total vCPUs
Total memory
Notifications
Labels
prod-gke
asia-south1-a
2
4
16 GB
Scale down blocked by pod
Google Cloud
My First Project
Kubernetes Engine
Search
Kubernetes Engine / Cluster: prod-gke / Nodes
Kubernetes Engine
All Fleets
No fleets in the current project
Overview
Clusters
Workloads
Compute Advisor
AI/ML
Models
Applications
Marketplace
Cluster details
Edit
Delete
Deploy
Connect
Duplicate
History
Equivalent code
prod-gke
Pod is blocking scale down because it doesn't have enough Pod Disruption Budget (PDB)
Details
Overview
Details
Nodes
Storage
Observability
Logs
App Errors (3)
Node pools
Create user-managed node pool
Filter
Node pool type : User-managed
Filter node pools
Name
Status
Version
Node pool type
Number of nodes
Machine type
Image type
default
Ok
1.36.4-gke.2046003
User-managed
2
e2-standard-2
Container-Optimized OS with containerd (cos_containerd)
Google Cloud
My First Project
Compute Engine
Search
Compute Engine
Overview
Security risk overview
Compute Advisor ...
Virtual machines
VM instances
Instance templates
Sole-tenant nodes
Machine images
TPUs
Committed use discou...
VM instances
Create instance
Import VM
Refresh
Learn
Instances
Observability
Instance schedules
VM instances
Filter
Enter property name or value
Status
Name
Zone
Recommendations
In use by
Connect
gke-prod-gke-default-2ed5f0fd-dzb9
asia-south1-a
gke-prod-gke-default-2ed5f0fd-grp
SSH
gke-prod-gke-default-2ed5f0fd-gvvg
asia-south1-a
gke-prod-gke-default-2ed5f0fd-grp
SSH

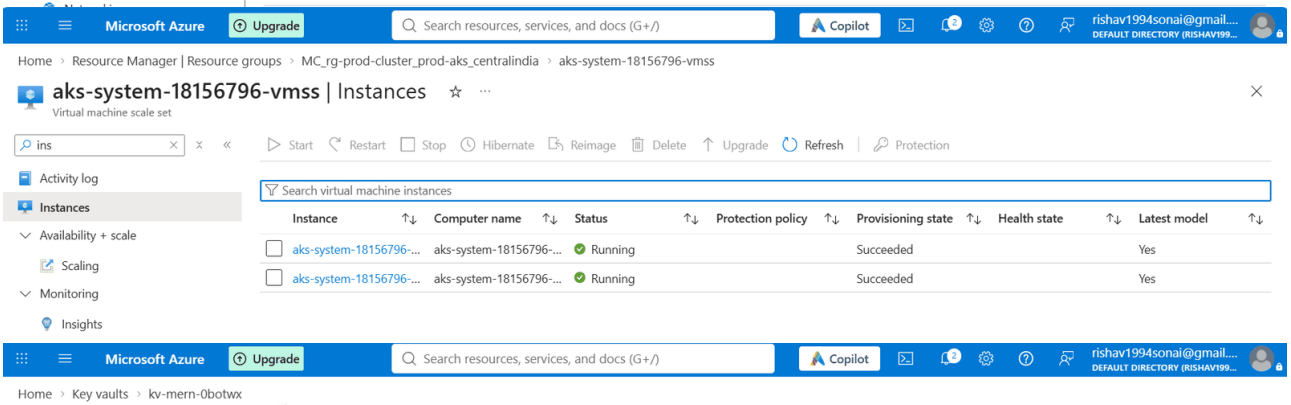

Google Cloud
My First Project
Network services
Search
Network Services
Load balancing
Cloud DNS
Cloud CDN
Cloud NAT
Cloud Service Mesh (Traffi...
Service Directory
Cloud Domains
Private Service Connect
Composite Health
SSL policies
Service Extensions
Global Front End
Preview
Marketplace
Release Notes
Cloud NAT gateway details
Edit
Delete
prod-nat
Status
Running
Logs
View in Logs Explorer
Details
Monitoring
NAT type
Type
Public
Cloud Router
Region
asia-south1
VPC network
prod
Cloud Router
prod-router
Cloud NAT mapping
High availability
Yes
Source endpoint type
VM instances, GKE nodes, Serverless
Source IPv4 subnets & IP ranges
prod-nodes
10.30.0.0/20 (Primary)
10.30.128.0/17 (Secondary: pods)

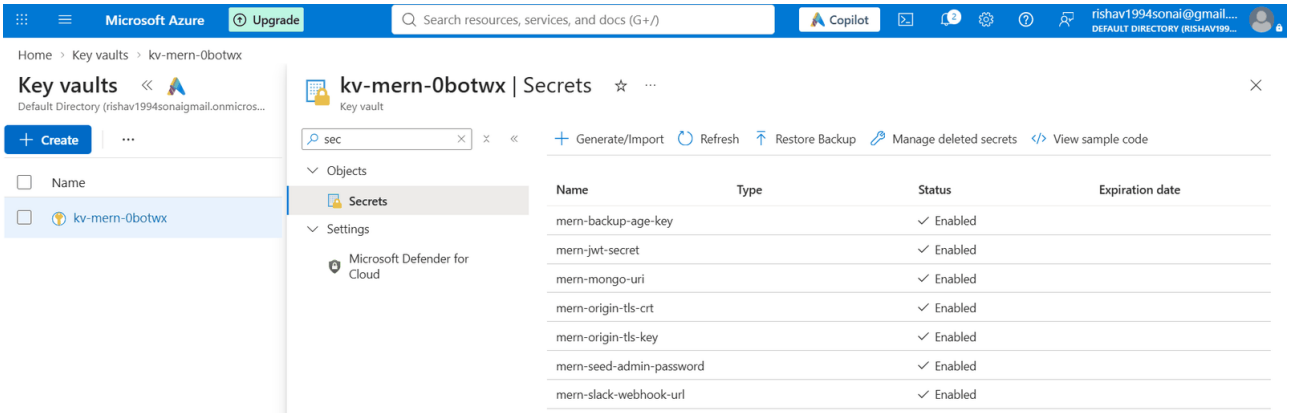

Google Cloud
My First Project
Security
Security / Secret manager / Secrets
Security
Security Command Center
Overview
Graph Search
Issues
Findings
Assets
Compliance
Posture Management
Rules
Sources
Settings
Detections and Controls
Marketplace
Release Notes
Secret Manager
Parameter Manager
Secrets
Regional secrets
Logs
Secret Manager lets you store, manage, and secure access to your application secrets. Learn more
Secrets
+ Create secret
Tags
Filter
Enter property name or value
Name
Type
Location
Encryption
Tags
Actions
ci-cloudflare-access-client-id
ci-cloudflare-access-client-secret
failover-github-token
grafana-cf-access-client-id
grafana-cf-access-client-secret
jenkins-trigger-token
mern-backup-age-key
mern-jwt-secret
Automatically replicated
Google-managed

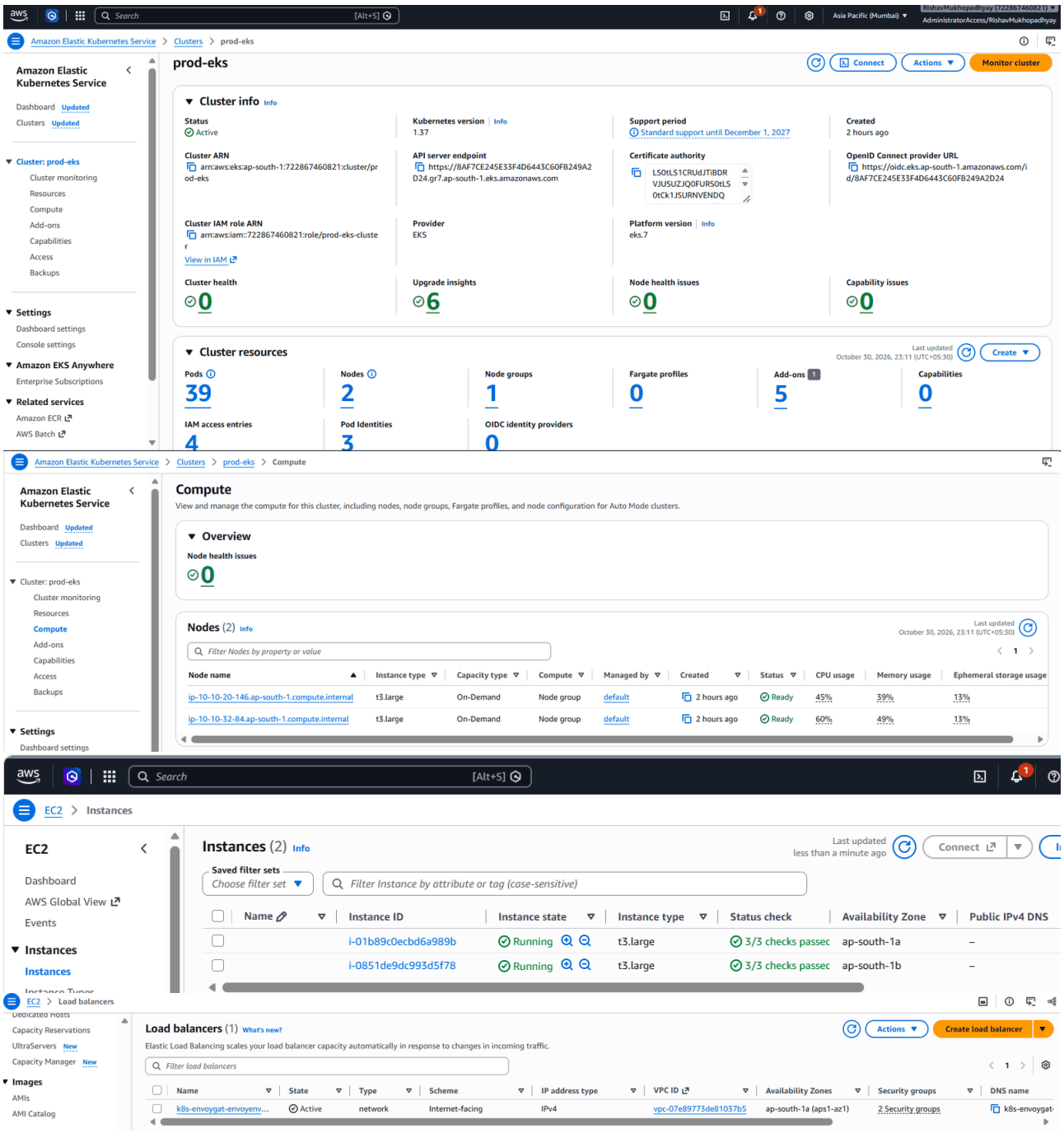

Microsoft Azure
Upgrade
Search resources, services, and docs (G+/)
Copilot
rishav1994sonai@gmail....
DEFAULT DIRECTORY (RISHAV199...
Home > Kubernetes center | Clusters
prod-aks
Kubernetes service
Search
Overview
Activity log
Access control (IAM)
Tags
Monitor (Insights)
Diagnose and solve problems
Microsoft Defender for Cloud
Cost analysis
Resource visualizer
Kubernetes resources
Settings
Monitoring
Automation
Create
Connect
Start
Stop
Delete
Refresh
Open in mobile
Give feedback
Essentials
JSON View
Resource group : rg-prod-cluster
Power state : Running
Cluster operation stat... : Succeeded
Subscription : Azure subscription 1
Location : Central India
Subscription ID : 9c0602c2-72a4-4432-a192-7abf998e2b60
Fleet Manager : Click here to assign
Kubernetes version : 1.36.5
API server address : prod-aks-m45blusp.hcp.centralindia.azmk8s.io
SKU : Base
Pricing tier : Free
Network configuration : Azure CNI Overlay
Node pools : 1 node pool
Container registries : acrmern0botwx
Created time : 30 October 2026 at 08:28 pm
Tags (edit) : ManagedBy : terraform
Project : multicloud-capstone
Show more (1)
Get started
Properties
Monitoring
Recommendations
Kubernetes troubleshooting with Copilot


Microsoft Azure
Upgrade
Search resources, services, and docs (G+/)
Copilot
rishav1994sonai@gmail....
DEFAULT DIRECTORY (RISHAV199...
Home › Kubernetes center | Clusters › prod-aks | Node pools
system | Overview
Node pool
Search
Overview
Nodes
Configuration
Upgrade Kubernetes
Update image
Scale node pool
Delete
Refresh
Give feedback
Essentials
Provisioning state : Succeeded
Power state : Running
Availability zones : None
Mode : System
Type : Virtual machine scale sets
Node size : Standard_D2as_v5
Cluster : prod-aks
Operating system : Ubuntu Linux
Kubernetes version : 1.36.5
Target node count : 2 nodes
Ready nodes : 2/2 nodes ready
Properties
Monitoring
Node pool
Max pods per node 250
Public IPs per node Disabled
Autoscaling Disabled
Azure Spot Instance Disabled
Maximum price N/A
Taints and labels
Taints (edit) None
Labels (edit) None
Add or remove favorites by pressing Ctrl+Shift+F

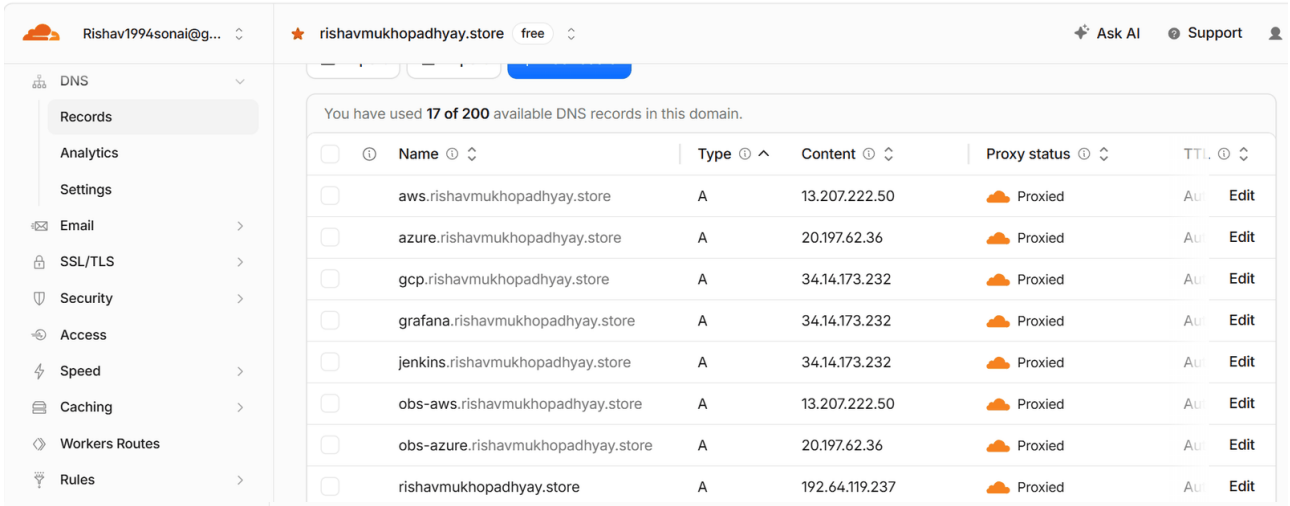

Microsoft Azure
Upgrade
Search resources, services, and docs (G+/)
Copilot
rishav1994sonai@gmail....
DEFAULT DIRECTORY (RISHAV199...
Home › Kubernetes center | Clusters › prod-aks
prod-aks | Workloads
Kubernetes service
work
Kubernetes resources
Workloads
Services, gateways and ingresses
Network policies
Run command
Settings
Security configuration
Create
Scale
Delete
Refresh
Show labels
Give feedback
Name
Namespace
Ready
Age
CPU
Memory
notification-service mern 1/1 2 hours Enable metrics
identity-service mern 2/2 2 hours
commerce-service mern 2/2 2 hours
client-service mern 2/2 2 hours
catalog-service mern 2/2 2 hours
api-gateway-service mern 2/2 2 hours

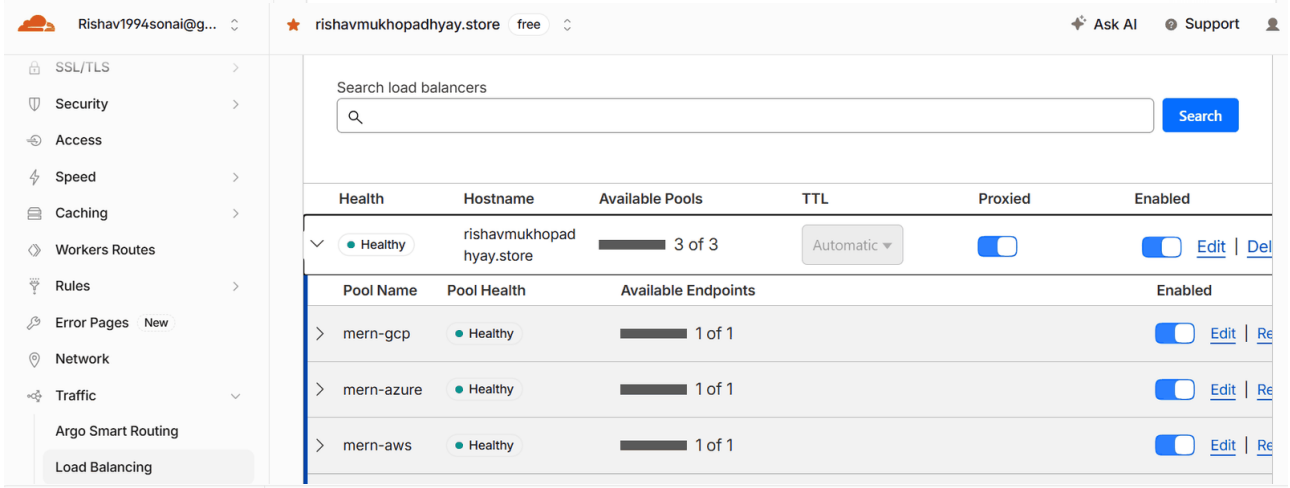

Microsoft Azure
Upgrade
Search resources, services, and docs (G+/)
Copilot
rishav1994sonai@gmail....
DEFAULT DIRECTORY (RISHAV199...
Home › Resource Manager | Resource groups › MC_rg-prod-cluster_prod-aks_centralindia › aks-system-18156796-vmss
aks-system-18156796-vmss | Instances
Virtual machine scale set
ins
Activity log
Instances
Availability + scale
Scaling
Monitoring
Insights
Start
Restart
Stop
Hibernate
Reimage
Delete
Upgrade
Refresh
Protection
Search virtual machine instances
Instance
Computer name
Status
Protection policy
Provisioning state
Health state
Latest model
aks-system-18156796-... aks-system-18156796-... Running Succeeded Yes
aks-system-18156796-... aks-system-18156796-... Running Succeeded Yes

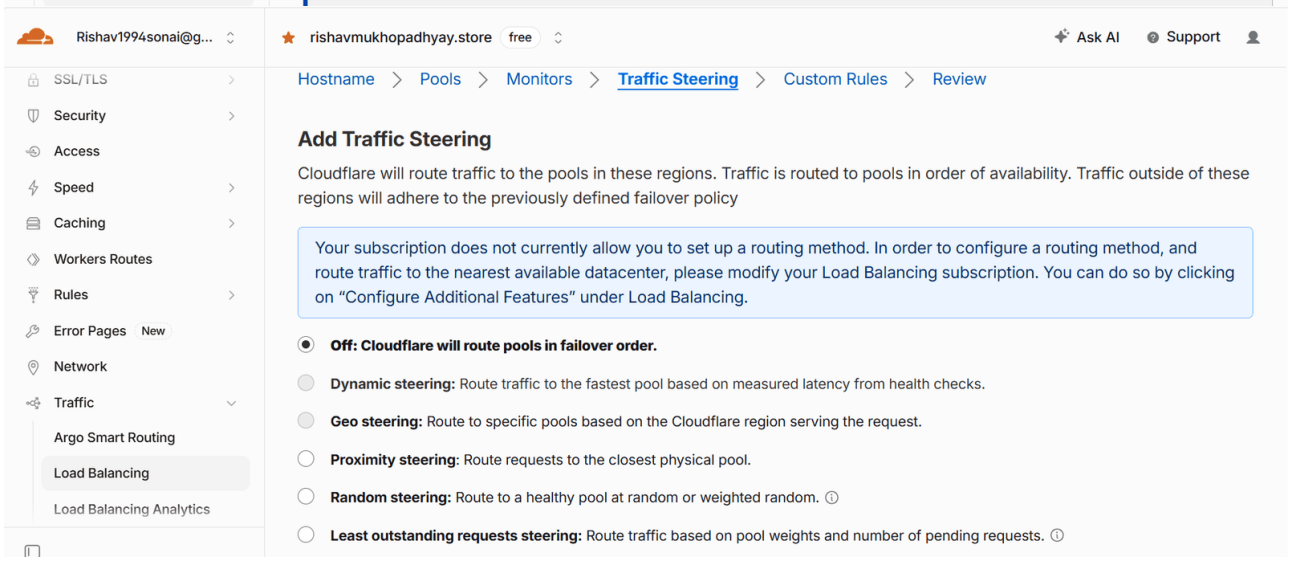

Microsoft Azure
Upgrade
Search resources, services, and docs (G+/)
Copilot
rishav1994sonai@gmail....
DEFAULT DIRECTORY (RISHAV199...
Home › Key vaults › kv-mern-0botwx
Key vaults
Default Directory (rishav1994sonaigmail.onmicros...
Create
Name
kv-mern-0botwx
kv-mern-0botwx | Secrets
Key vault
sec
Objects
Secrets
Settings
Microsoft Defender for Cloud
Generate/Import
Refresh
Restore Backup
Manage deleted secrets
View sample code
Name
Type
Status
Expiration date
mern-backup-age-key Enabled
mern-jwt-secret Enabled
mern-mongo-uri Enabled
mern-origin-tls-crt Enabled
mern-origin-tls-key Enabled
mern-seed-admin-password Enabled
mern-slack-webhook-url Enabled

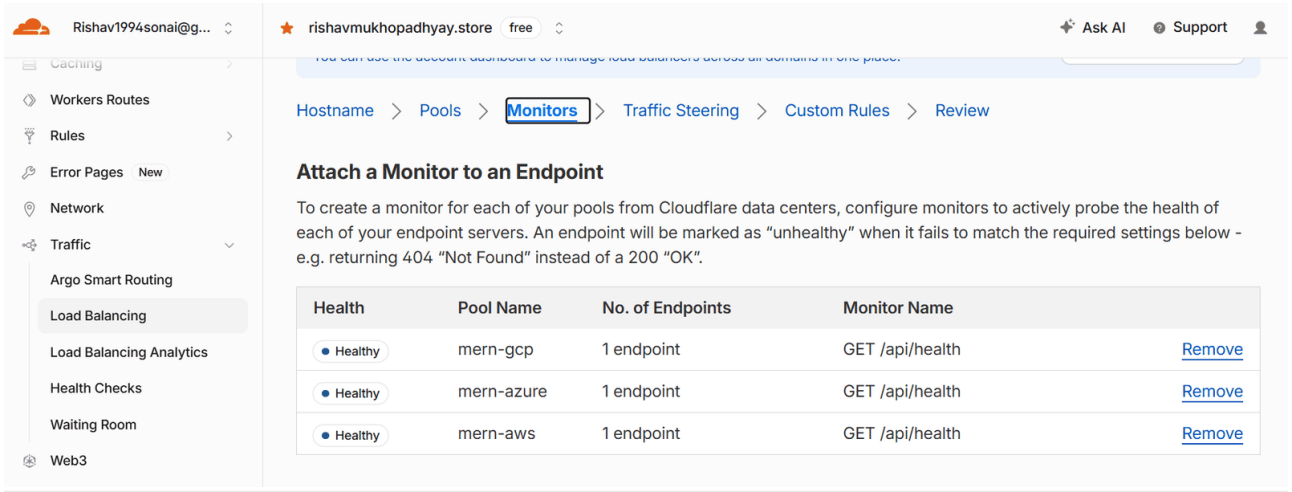


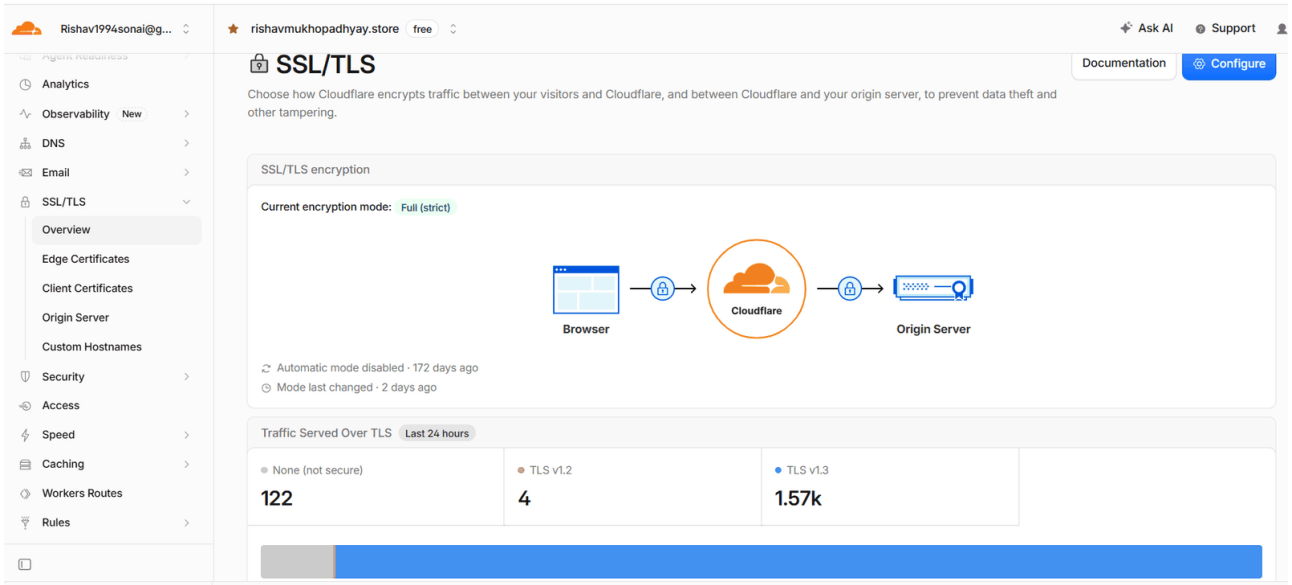
Global layer and database

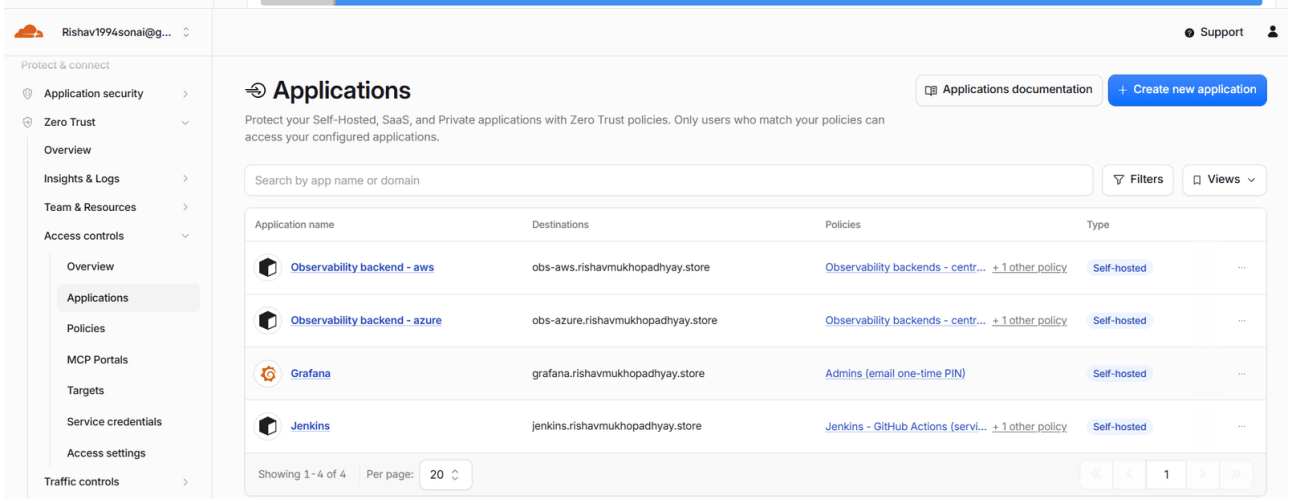

Rishav1994sonai@g...
rishavmukhopadhyay.store
free
Ask AI
Support
DNS
Records
Analytics
Settings
Email
SSL/TLS
Security
Access
Speed
Caching
Workers Routes
Rules
You have used 17 of 200 available DNS records in this domain.
Name
Type
Content
Proxy status
TTL
aws.rishavmukhopadhyay.store A 13.207.222.50 Proxied Edit
azure.rishavmukhopadhyay.store A 20.197.62.36 Proxied Edit
gcp.rishavmukhopadhyay.store A 34.14.173.232 Proxied Edit
grafana.rishavmukhopadhyay.store A 34.14.173.232 Proxied Edit
jenkins.rishavmukhopadhyay.store A 34.14.173.232 Proxied Edit
obs-aws.rishavmukhopadhyay.store A 13.207.222.50 Proxied Edit
obs-azure.rishavmukhopadhyay.store A 20.197.62.36 Proxied Edit
rishavmukhopadhyay.store A 192.64.119.237 Proxied Edit

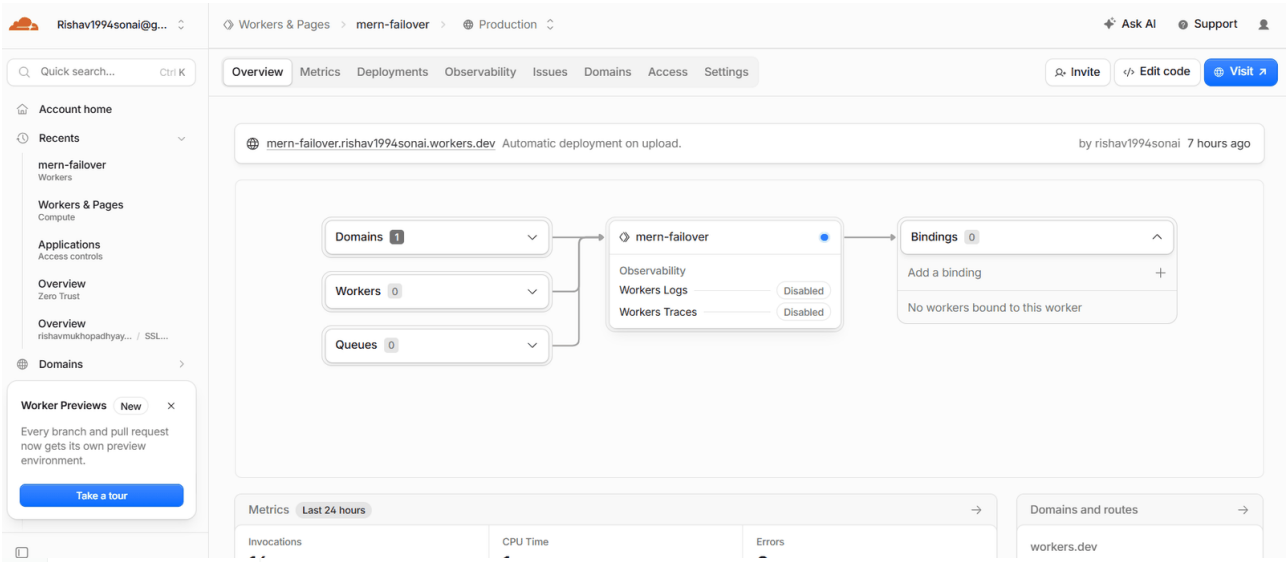

Rishav1994sonai@g...
rishavmukhopadhyay.store
free
Ask AI
Support
SSL/TLS
Security
Access
Speed
Caching
Workers Routes
Rules
Error Pages New
Network
Traffic
Argo Smart Routing
Load Balancing
Search load balancers
Search
Health
Hostname
Available Pools
TTL
Proxied
Enabled
Healthy
rishavmukhopadhyay.store
3 of 3
Automatic
Edit | Del
Pool Name
Pool Health
Available Endpoints
Enabled
mern-gcp Healthy 1 of 1 Edit | Re
mern-azure Healthy 1 of 1 Edit | Re
mern-aws Healthy 1 of 1 Edit | Re

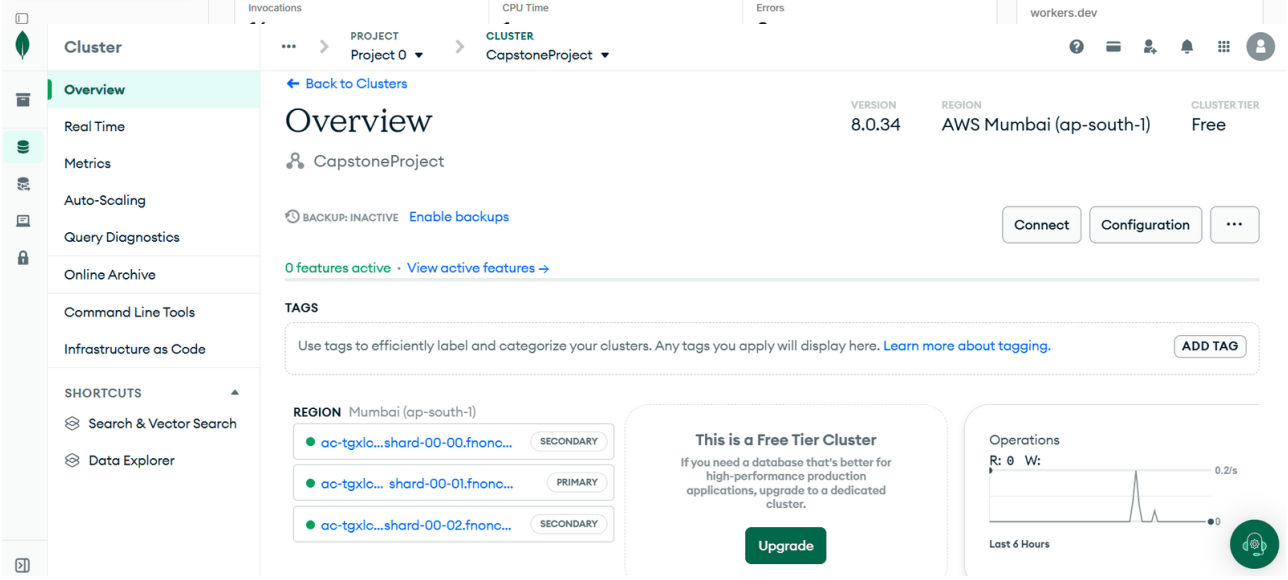

Rishav1994sonai@g...
rishavmukhopadhyay.store
free
Ask AI
Support
SSL/TLS
Security
Access
Speed
Caching
Workers Routes
Rules
Error Pages New
Network
Traffic
Argo Smart Routing
Load Balancing
Load Balancing Analytics
Hostname > Pools > Monitors > Traffic Steering > Custom Rules > Review
Add Traffic Steering
Cloudflare will route traffic to the pools in these regions. Traffic is routed to pools in order of availability. Traffic outside of these regions will adhere to the previously defined failover policy
Your subscription does not currently allow you to set up a routing method. In order to configure a routing method, and route traffic to the nearest available datacenter, please modify your Load Balancing subscription. You can do so by clicking on “Configure Additional Features” under Load Balancing.
Off: Cloudflare will route pools in failover order.
Dynamic steering: Route traffic to the fastest pool based on measured latency from health checks.
Geo steering: Route to specific pools based on the Cloudflare region serving the request.
Proximity steering: Route requests to the closest physical pool.
Random steering: Route to a healthy pool at random or weighted random.
Least outstanding requests steering: Route traffic based on pool weights and number of pending requests.

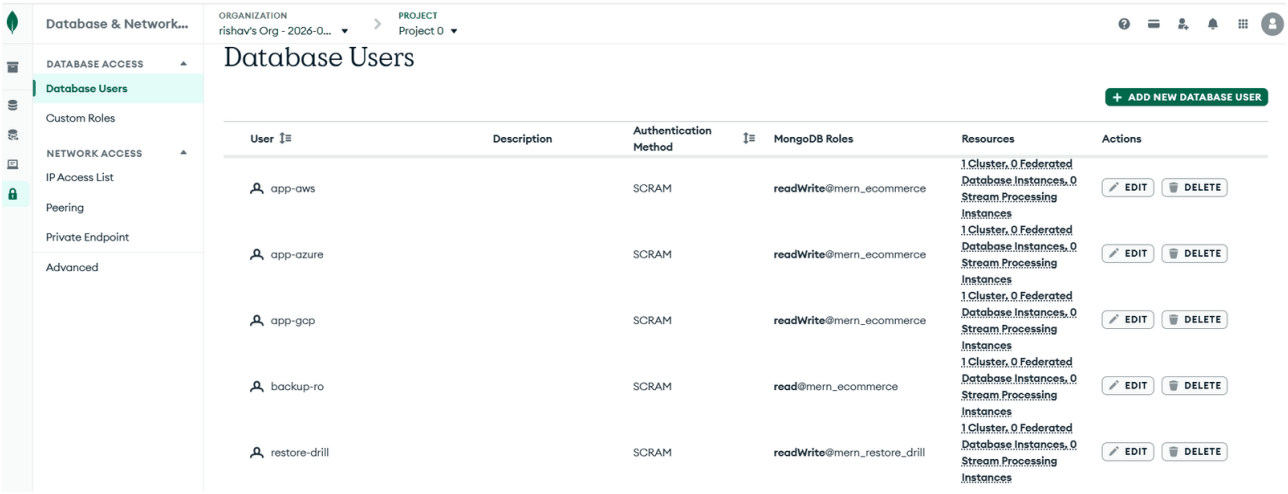

Rishav1994sonai@g...
rishavmukhopadhyay.store
free
Ask AI
Support
Workers Routes
Rules
Error Pages
New
Network
Traffic
Argo Smart Routing
Load Balancing
Load Balancing Analytics
Health Checks
Waiting Room
Web3
Hostname
Pools
Monitors
Traffic Steering
Custom Rules
Review
Attach a Monitor to an Endpoint
To create a monitor for each of your pools from Cloudflare data centers, configure monitors to actively probe the health of each of your endpoint servers. An endpoint will be marked as “unhealthy” when it fails to match the required settings below - e.g. returning 404 “Not Found” instead of a 200 “OK”.
Health
Pool Name
No. of Endpoints
Monitor Name
Healthy
mern-gcp
1 endpoint
GET /api/health
Remove
Healthy
mern-azure
1 endpoint
GET /api/health
Remove
Healthy
mern-aws
1 endpoint
GET /api/health
Remove

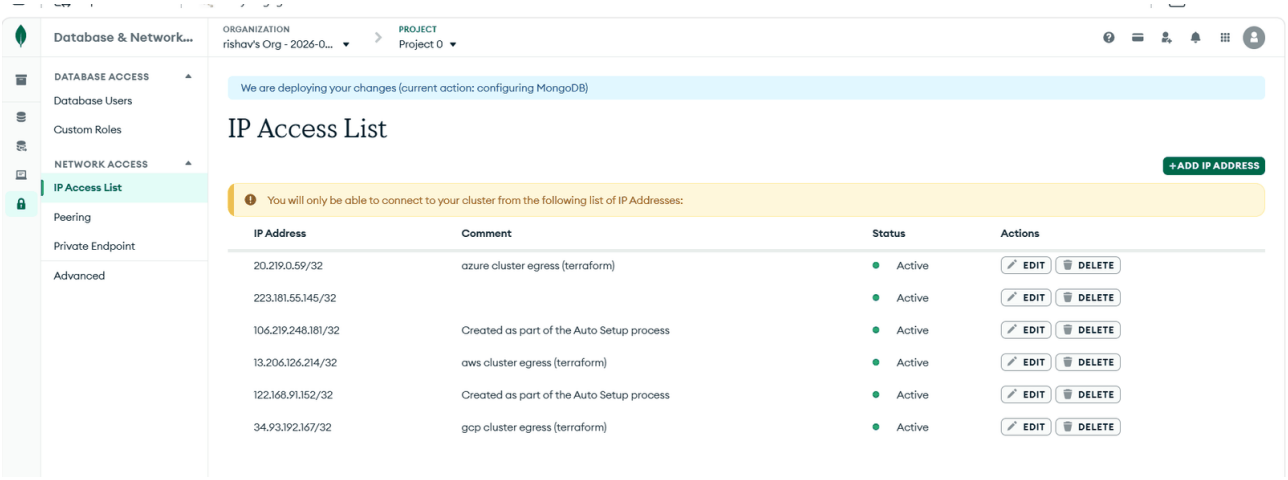

Rishav1994sonai@g...
rishavmukhopadhyay.store
free
Ask AI
Support
Analytics
Observability
New
DNS
Email
SSL/TLS
Overview
Edge Certificates
Client Certificates
Origin Server
Custom Hostnames
Security
Access
Speed
Caching
Workers Routes
Rules
SSL/TLS
Documentation
Configure
Choose how Cloudflare encrypts traffic between your visitors and Cloudflare, and between Cloudflare and your origin server, to prevent data theft and other tampering.
SSL/TLS encryption
Current encryption mode: Full (strict)
Browser
Cloudflare
Origin Server
Automatic mode disabled · 172 days ago
Mode last changed · 2 days ago
Traffic Served Over TLS
Last 24 hours
None (not secure)
122
TLS v1.2
4
TLS v1.3
1.57k

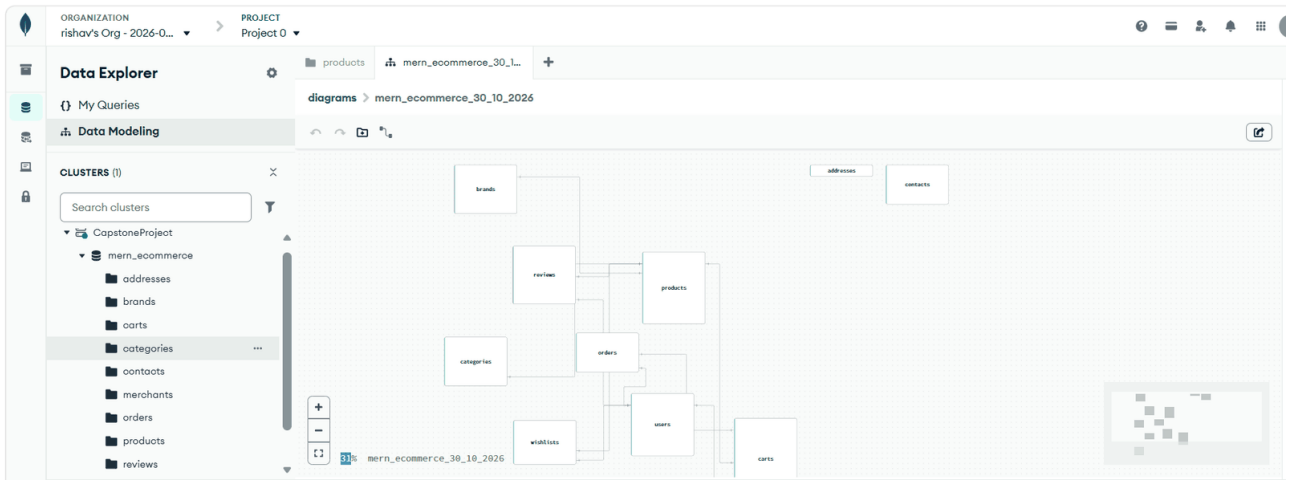

Rishav1994sonai@g...
Support
Protect & connect
Application security
Zero Trust
Overview
Insights & Logs
Team & Resources
Access controls
Overview
Applications
Policies
MCP Portals
Targets
Service credentials
Access settings
Traffic controls
Applications
Applications documentation
Create new application
Protect your Self-Hosted, SaaS, and Private applications with Zero Trust policies. Only users who match your policies can access your configured applications.
Search by app name or domain
Filters
Views
Application name
Destinations
Policies
Type
Observability backend - aws
obs-aws.rishavmukhopadhyay.store
Observability backends - centr... + 1 other policy
Self-hosted
Observability backend - azure
obs-azure.rishavmukhopadhyay.store
Observability backends - centr... + 1 other policy
Self-hosted
Grafana
grafana.rishavmukhopadhyay.store
Admins (email one-time PIN)
Self-hosted
Jenkins
jenkins.rishavmukhopadhyay.store
Jenkins - GitHub Actions (servi... + 1 other policy
Self-hosted
Showing 1-4 of 4
Per page:
20
1


Rishav1994sonai@g...
Workers & Pages
mern-failover
Production
Ask AI
Support
Quick search...
Ctrl K
Overview
Metrics
Deployments
Observability
Issues
Domains
Access
Settings
Invite
Edit code
Visit
Account home
Recents
mern-failover
Workers
Workers & Pages
Compute
Applications
Access controls
Overview
Zero Trust
Overview
rishavmukhopadhyay... / SSL...
Domains
mern-failover.rishav1994sonai.workers.dev
Automatic deployment on upload.
by rishav1994sonai 7 hours ago
Domains 1
Workers 0
Queues 0
mern-failover
Observability
Workers Logs
Disabled
Workers Traces
Disabled
Bindings 0
Add a binding
No workers bound to this worker
Worker Previews
New
Every branch and pull request now gets its own preview environment.
Take a tour
Metrics
Last 24 hours
Invocations
CPU Time
Errors
Domains and routes
workers.dev


Cluster
PROJECT
Project 0
CLUSTER
CapstoneProject
Overview
Real Time
Metrics
Auto-Scaling
Query Diagnostics
Online Archive
Command Line Tools
Infrastructure as Code
SHORTCUTS
Search & Vector Search
Data Explorer
Back to Clusters
Overview
CapstoneProject
VERSION
8.0.34
REGION
AWS Mumbai (ap-south-1)
CLUSTER TIER
Free
BACKUP: INACTIVE
Enable backups
Connect
Configuration
0 features active · View active features →
TAGS
Use tags to efficiently label and categorize your clusters. Any tags you apply will display here. Learn more about tagging.
ADD TAG
REGION Mumbai (ap-south-1)
ac-tgxlc...shard-00-00.fnonc...
SECONDARY
ac-tgxlc... shard-00-01.fnonc...
PRIMARY
ac-tgxlc...shard-00-02.fnonc...
SECONDARY
This is a Free Tier Cluster
If you need a database that's better for high-performance production applications, upgrade to a dedicated cluster.
Upgrade
Operations
R: 0 W:
0.2/s
0
Last 6 Hours

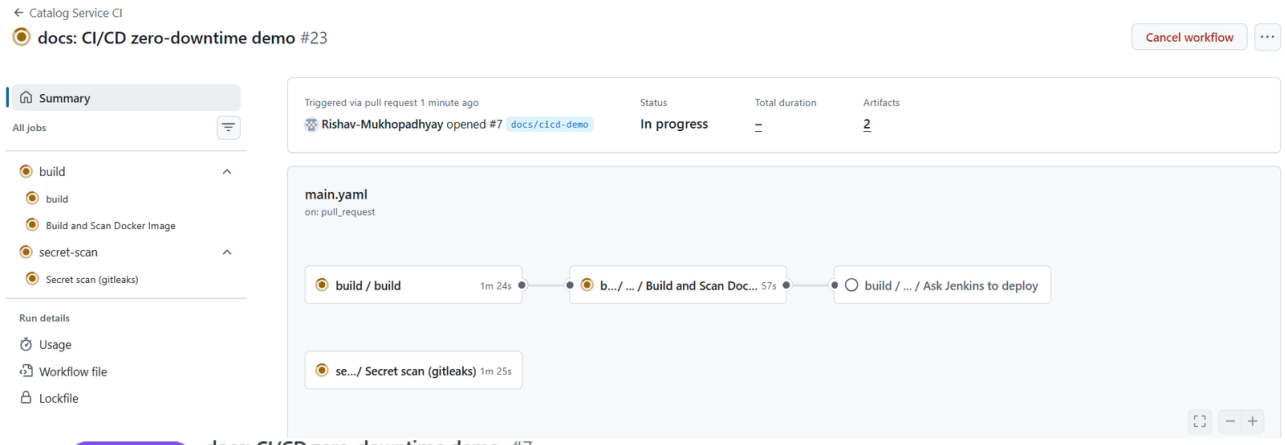

Database & Network...
ORGANIZATION
rishav's Org - 2026-0...
PROJECT
Project 0
DATABASE ACCESS
Database Users
Custom Roles
NETWORK ACCESS
IP Access List
Peering
Private Endpoint
Advanced
Database Users
ADD NEW DATABASE USER
User
Description
Authentication Method
MongoDB Roles
Resources
Actions
app-aws
SCRAM
readWrite@mern_ecommerce
1 Cluster, 0 Federated Database Instances, 0 Stream Processing Instances
EDIT
DELETE
app-azure
SCRAM
readWrite@mern_ecommerce
1 Cluster, 0 Federated Database Instances, 0 Stream Processing Instances
EDIT
DELETE
app-gcp
SCRAM
readWrite@mern_ecommerce
1 Cluster, 0 Federated Database Instances, 0 Stream Processing Instances
EDIT
DELETE
backup-ro
SCRAM
read@mern_ecommerce
1 Cluster, 0 Federated Database Instances, 0 Stream Processing Instances
EDIT
DELETE
restore-drill
SCRAM
readWrite@mern_restore_drill
1 Cluster, 0 Federated Database Instances, 0 Stream Processing Instances
EDIT
DELETE

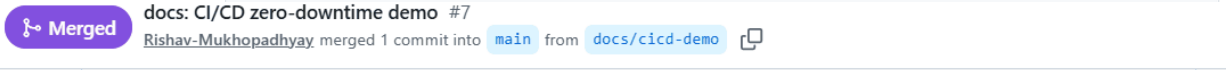


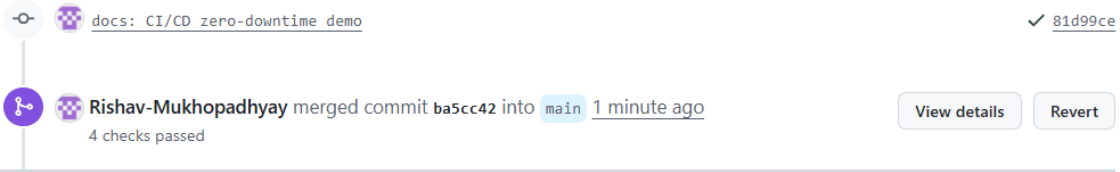


D. CI → CD: one merge, three clouds

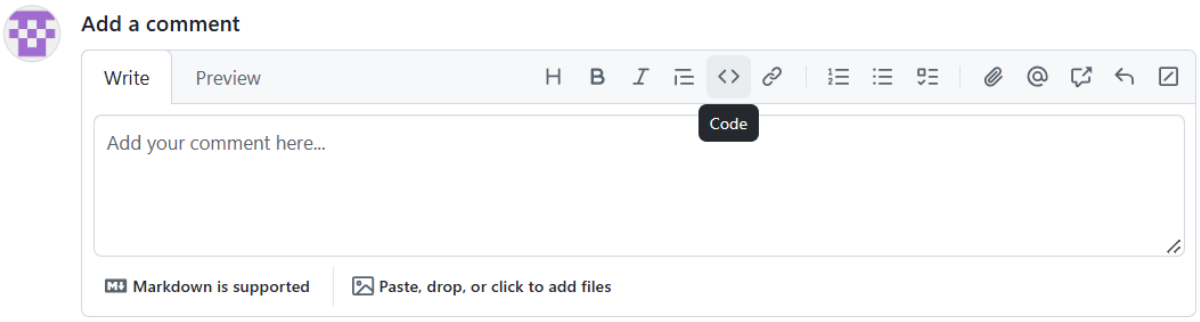
A one-line documentation change in `catalog-service` (PR #7) went through the whole pipeline.

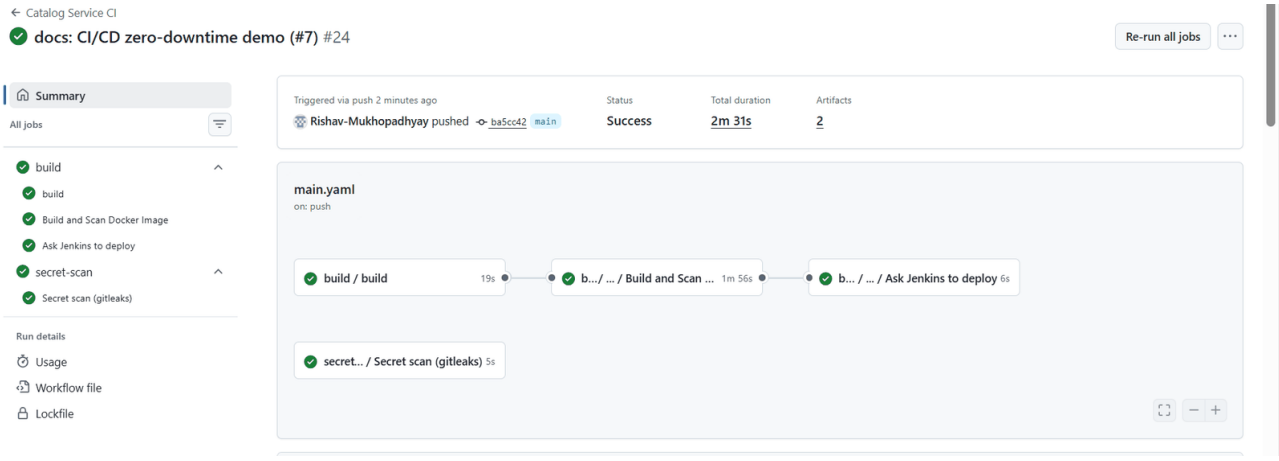

Catalog Service CI
docs: CI/CD zero-downtime demo #23
Cancel workflow
Summary
All jobs
build
build
Build and Scan Docker Image
secret-scan
Secret scan (gitleaks)
Run details
Usage
Workflow file
Lockfile
Triggered via pull request 1 minute ago
Rishav-Mukhopadhyay opened #7 docs/cicd-demo
Status
In progress
Total duration
–
Artifacts
2
main.yaml
on: pull_request
build / build 1m 24s
b.../ ... / Build and Scan Doc... 57s
build / ... / Ask Jenkins to deploy
se.../ Secret scan (gitleaks) 1m 25s

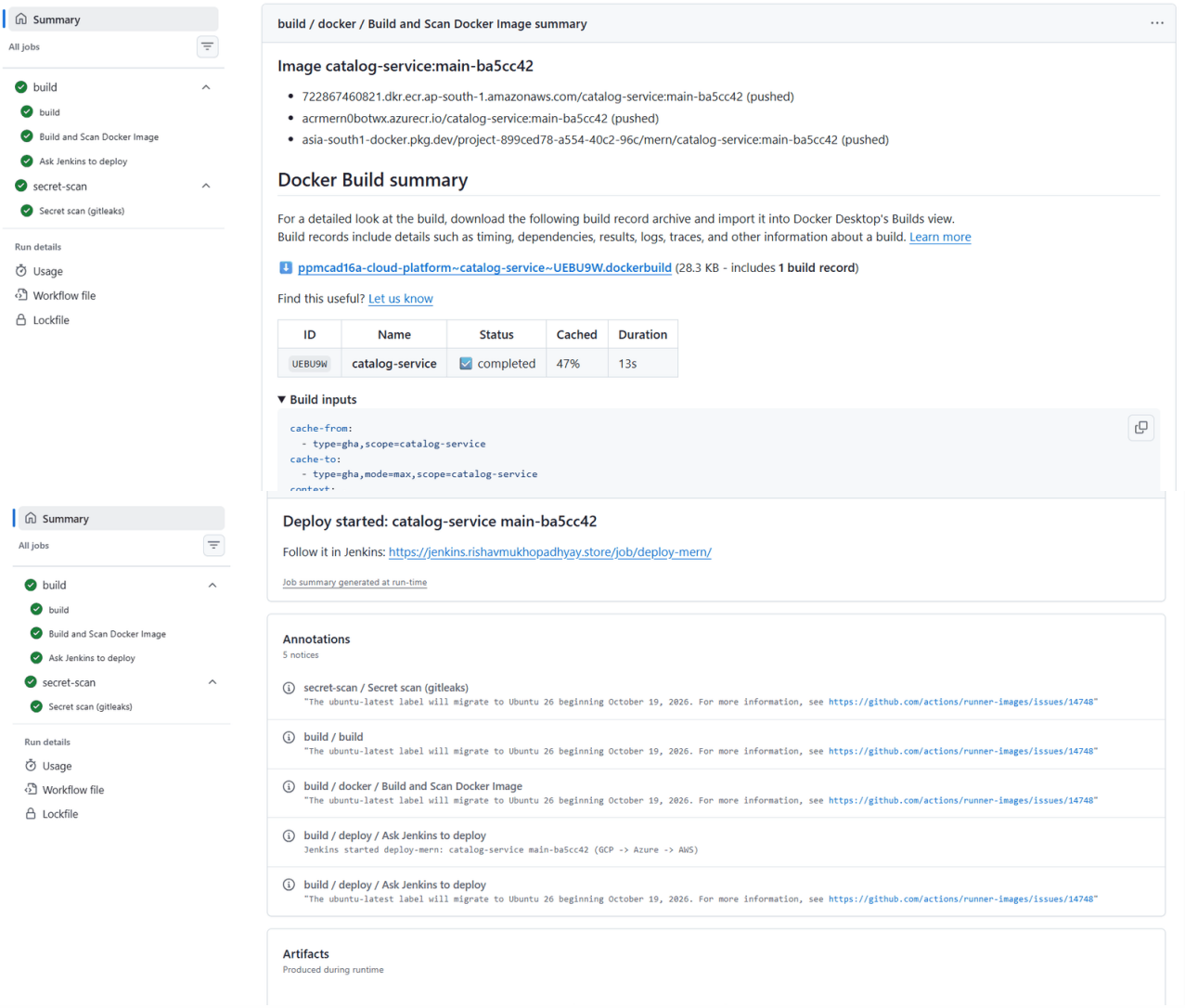

Merged
docs: CI/CD zero-downtime demo #7
Rishav-Mukhopadhyay merged 1 commit into main from docs/cicd-demo


docs: CI/CD zero-downtime demo
81d99ce
Rishav-Mukhopadhyay merged commit ba5cc42 into main 1 minute ago
4 checks passed
View details
Revert
Rishav-Mukhopadhyay deleted the docs/cicd-demo branch 1 minute ago
Restore branch

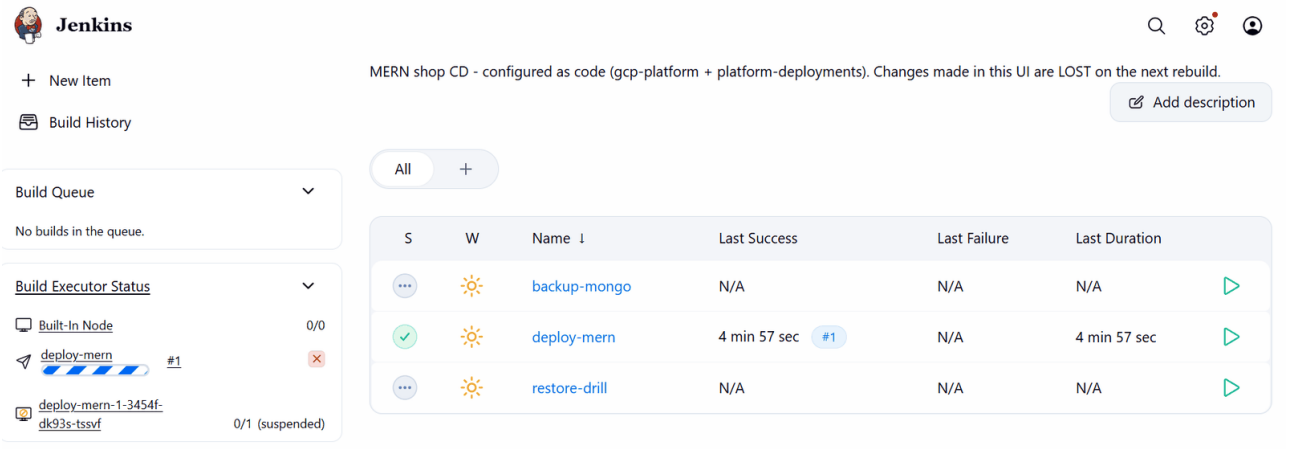

Pull request successfully merged and closed
You're all set — the branch has been merged.

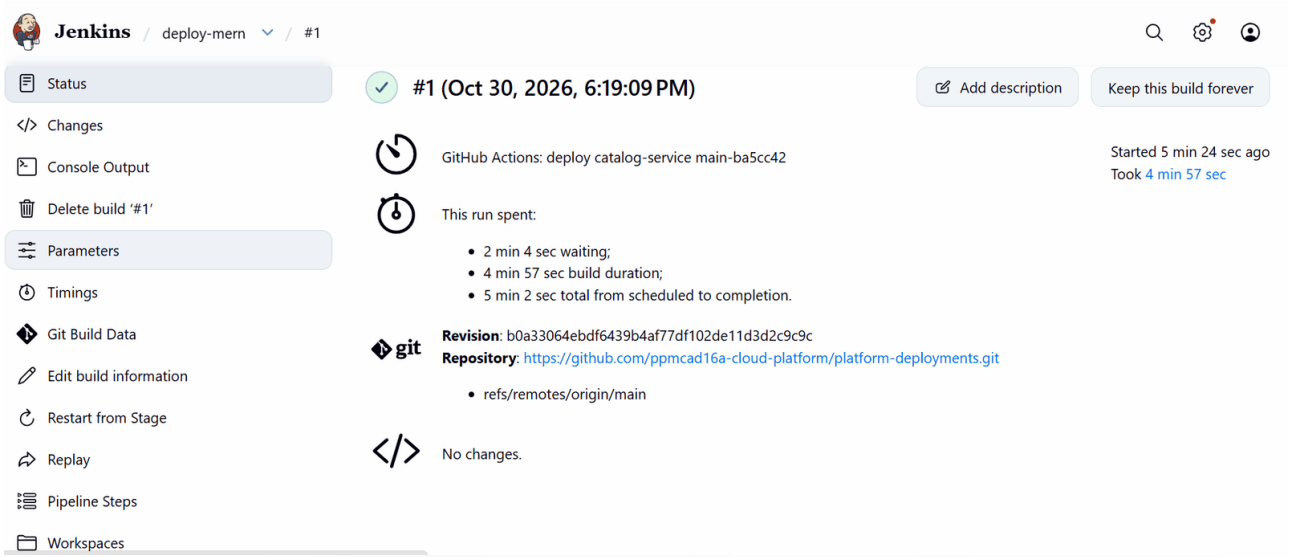

Add a comment
Write
Preview
Code
Add your comment here...
Markdown is supported
Paste, drop, or click to add files
Comment

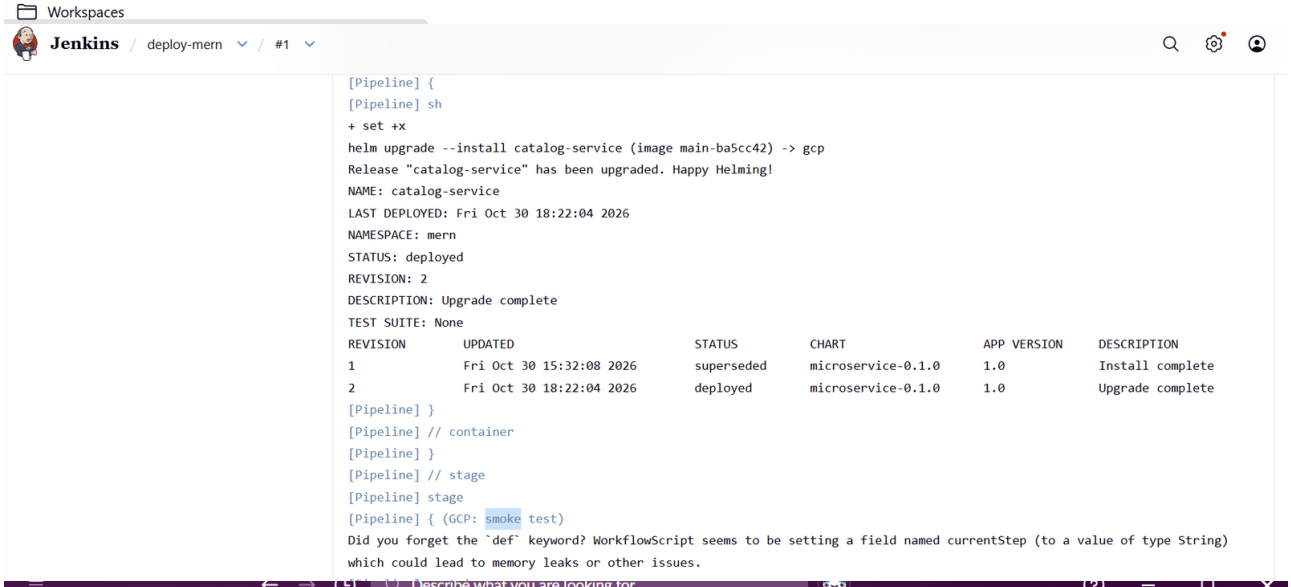

Catalog Service CI
docs: CI/CD zero-downtime demo (#7) #24
Re-run all jobs
Summary
All jobs
build
build
Build and Scan Docker Image
Ask Jenkins to deploy
secret-scan
Secret scan (gitleaks)
Run details
Usage
Workflow file
Lockfile
Triggered via push 2 minutes ago
Rishav-Mukhopadhyay pushed ba5cc42 main
Status
Success
Total duration
2m 31s
Artifacts
2
main.yaml
on: push
build / build 19s
b.../ ... / Build and Scan ... 1m 56s
b... / ... / Ask Jenkins to deploy 6s
secret... / Secret scan (gitleaks) 5s

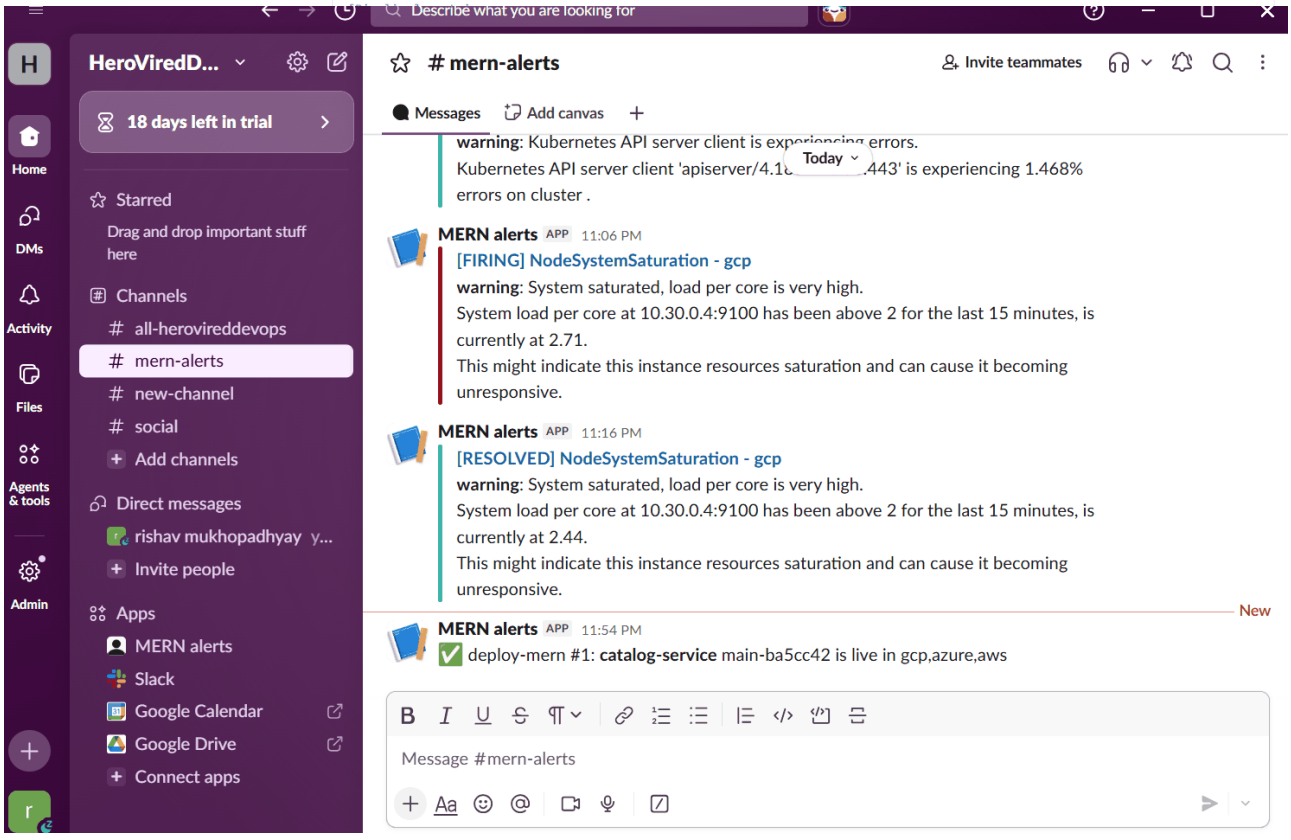

Summary
All jobs
build
build
Build and Scan Docker Image
Ask Jenkins to deploy
secret-scan
Secret scan (gitleaks)
Run details
Usage
Workflow file
Lockfile
build / docker / Build and Scan Docker Image summary
Image catalog-service:main-ba5cc42
722867460821.dkr.ecr.ap-south-1.amazonaws.com/catalog-service:main-ba5cc42 (pushed)
acrmern0botwx.azurecr.io/catalog-service:main-ba5cc42 (pushed)
asia-south1-docker.pkg.dev/project-899ced78-a554-40c2-96c/mern/catalog-service:main-ba5cc42 (pushed)
Docker Build summary
For a detailed look at the build, download the following build record archive and import it into Docker Desktop's Builds view.
Build records include details such as timing, dependencies, results, logs, traces, and other information about a build. Learn more
ppmcad16a-cloud-platform~catalog-service~UEBU9W.dockerbuild (28.3 KB - includes 1 build record)
Find this useful? Let us know
ID
Name
Status
Cached
Duration
UEBU9W
catalog-service
completed
47%
13s
Build inputs
cache-from:
- type=gha,scope=catalog-service
cache-to:
- type=gha,mode=max,scope=catalog-service
context:

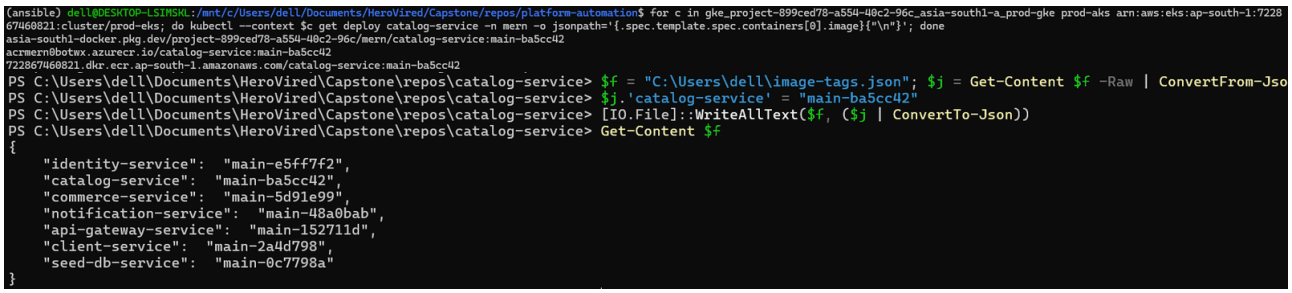
Summary
All jobs
build
build
Build and Scan Docker Image
Ask Jenkins to deploy
secret-scan
Secret scan (gitleaks)
Run details
Usage
Workflow file
Lockfile
Deploy started: catalog-service main-ba5cc42
Follow it in Jenkins: https://jenkins.rishavmukhopadhyay.store/job/deploy-mern/
Job summary generated at run-time
Annotations
5 notices
secret-scan / Secret scan (gitleaks)
"The ubuntu-latest label will migrate to Ubuntu 26 beginning October 19, 2026. For more information, see https://github.com/actions/runner-images/issues/14748"
build / build
"The ubuntu-latest label will migrate to Ubuntu 26 beginning October 19, 2026. For more information, see https://github.com/actions/runner-images/issues/14748"
build / docker / Build and Scan Docker Image
"The ubuntu-latest label will migrate to Ubuntu 26 beginning October 19, 2026. For more information, see https://github.com/actions/runner-images/issues/14748"
build / deploy / Ask Jenkins to deploy
Jenkins started deploy-mern: catalog-service main-ba5cc42 (GCP -> Azure -> AWS)
build / deploy / Ask Jenkins to deploy
"The ubuntu-latest label will migrate to Ubuntu 26 beginning October 19, 2026. For more information, see https://github.com/actions/runner-images/issues/14748"
Artifacts
Produced during runtime


Jenkins
New Item
Build History
MERN shop CD - configured as code (gcp-platform + platform-deployments). Changes made in this UI are LOST on the next rebuild.
Add description
Build Queue
No builds in the queue.
Build Executor Status
Built-In Node
0/0
deploy-mern
#1
deploy-mern-1-3454f-dk93s-tssvf
0/1 (suspended)
All
S
W
Name
Last Success
Last Failure
Last Duration
backup-mongo
N/A
N/A
N/A
deploy-mern
4 min 57 sec
#1
N/A
4 min 57 sec
restore-drill
N/A
N/A
N/A


Jenkins / deploy-mern / #1
Status
Changes
Console Output
Delete build '#1'
Parameters
Timings
Git Build Data
Edit build information
Restart from Stage
Replay
Pipeline Steps
Workspaces
#1 (Oct 30, 2026, 6:19:09 PM)
Add description
Keep this build forever
GitHub Actions: deploy catalog-service main-ba5cc42
Started 5 min 24 sec ago
Took 4 min 57 sec
This run spent:
2 min 4 sec waiting;
4 min 57 sec build duration;
5 min 2 sec total from scheduled to completion.
Revision: b0a33064ebdf6439b4af77df102de11d3d2c9c9c
Repository: https://github.com/ppmcad16a-cloud-platform/platform-deployments.git
refs/remotes/origin/main
No changes.


Jenkins / deploy-mern / #1
[Pipeline] {
[Pipeline] sh
+ set +x
helm upgrade --install catalog-service (image main-ba5cc42) -> gcp
Release "catalog-service" has been upgraded. Happy Helming!
NAME: catalog-service
LAST DEPLOYED: Fri Oct 30 18:22:04 2026
NAMESPACE: mern
STATUS: deployed
REVISION: 2
DESCRIPTION: Upgrade complete
TEST SUITE: None
REVISION UPDATED STATUS CHART APP VERSION DESCRIPTION
1 Fri Oct 30 15:32:08 2026 superseded microservice-0.1.0 1.0 Install complete
2 Fri Oct 30 18:22:04 2026 deployed microservice-0.1.0 1.0 Upgrade complete
[Pipeline] }
[Pipeline] // container
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (GCP: smoke test)
Did you forget the `def` keyword? WorkflowScript seems to be setting a field named currentStep (to a value of type String)
which could lead to memory leaks or other issues.


HeroViredD...
18 days left in trial
Home
DMs
Activity
Files
Agents & tools
Admin
Starred
Drag and drop important stuff here
Channels
all-herovireddevops
mern-alerts
new-channel
social
Add channels
Direct messages
rishav mukhopadhyay y...
Invite people
Apps
MERN alerts
Slack
Google Calendar
Google Drive
Connect apps
mern-alerts
Invite teammates
Messages
Add canvas
Today
warning: Kubernetes API server client is experiencing errors.
Kubernetes API server client 'apiserver/4.1 .443' is experiencing 1.468% errors on cluster .
MERN alerts APP 11:06 PM
[FIRING] NodeSystemSaturation - gcp
warning: System saturated, load per core is very high.
System load per core at 10.30.0.4:9100 has been above 2 for the last 15 minutes, is currently at 2.71.
This might indicate this instance resources saturation and can cause it becoming unresponsive.
MERN alerts APP 11:16 PM
[RESOLVED] NodeSystemSaturation - gcp
warning: System saturated, load per core is very high.
System load per core at 10.30.0.4:9100 has been above 2 for the last 15 minutes, is currently at 2.44.
This might indicate this instance resources saturation and can cause it becoming unresponsive.
New
MERN alerts APP 11:54 PM
deploy-mern #1: catalog-service main-ba5cc42 is live in gcp,azure,aws
Message #mern-alerts

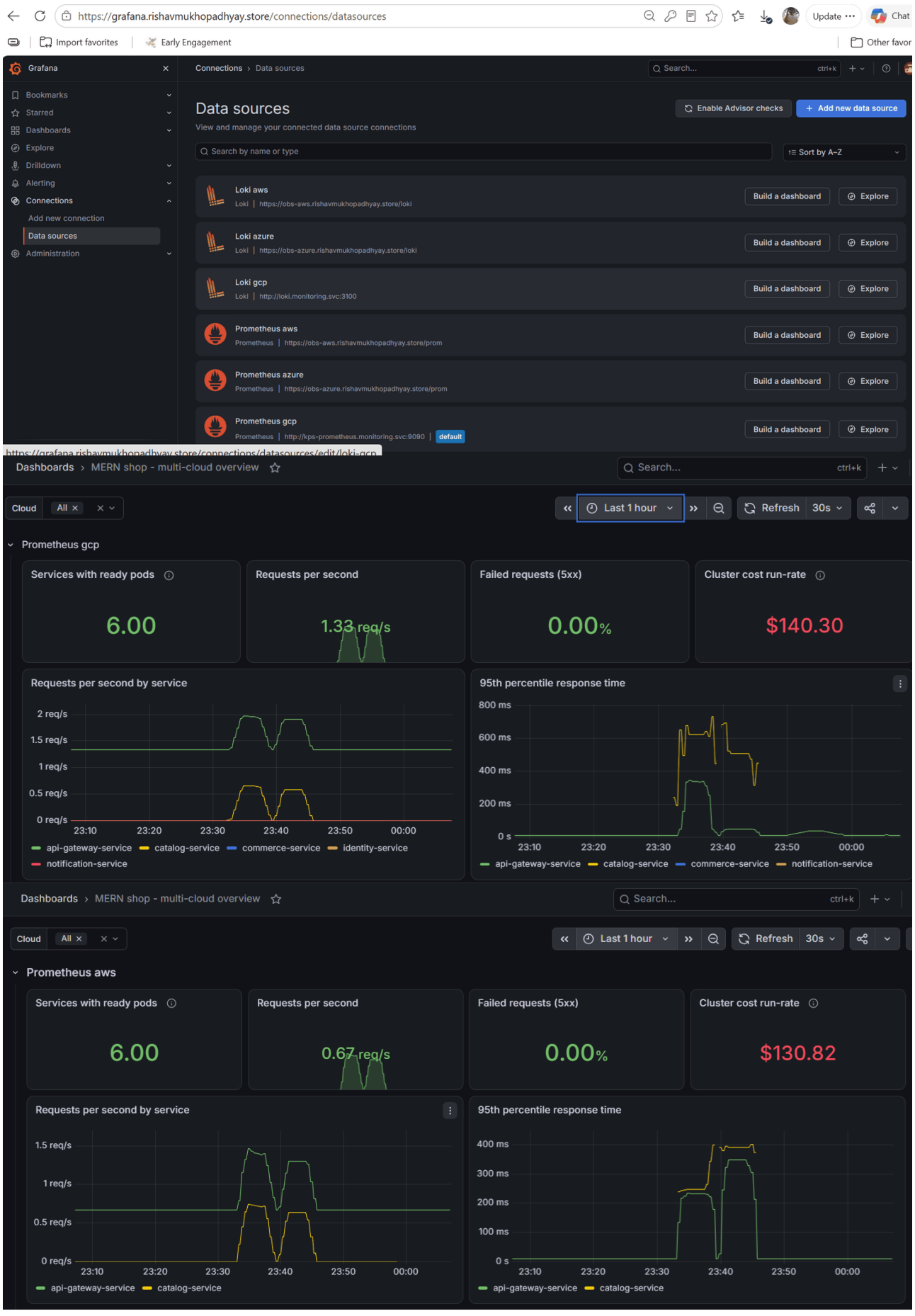


Zero-downtime measurement. The probe below asks each cloud for `/api/category/list` twice in parallel, continuously, and counts every answer that is not 200. In this test it ran from 18:09 to 18:10 UTC, but the deploy started at 18:19 UTC, so its "0 failed" result is **not** evidence for this deploy and is not shown. Zero downtime was demonstrated on 29 Oct 2026 (catalog-service PR #6, 0 failed requests). To repeat the measurement, start the probe **before** merging and stop it only after Slack reports *live in gcp,azure,aws*:

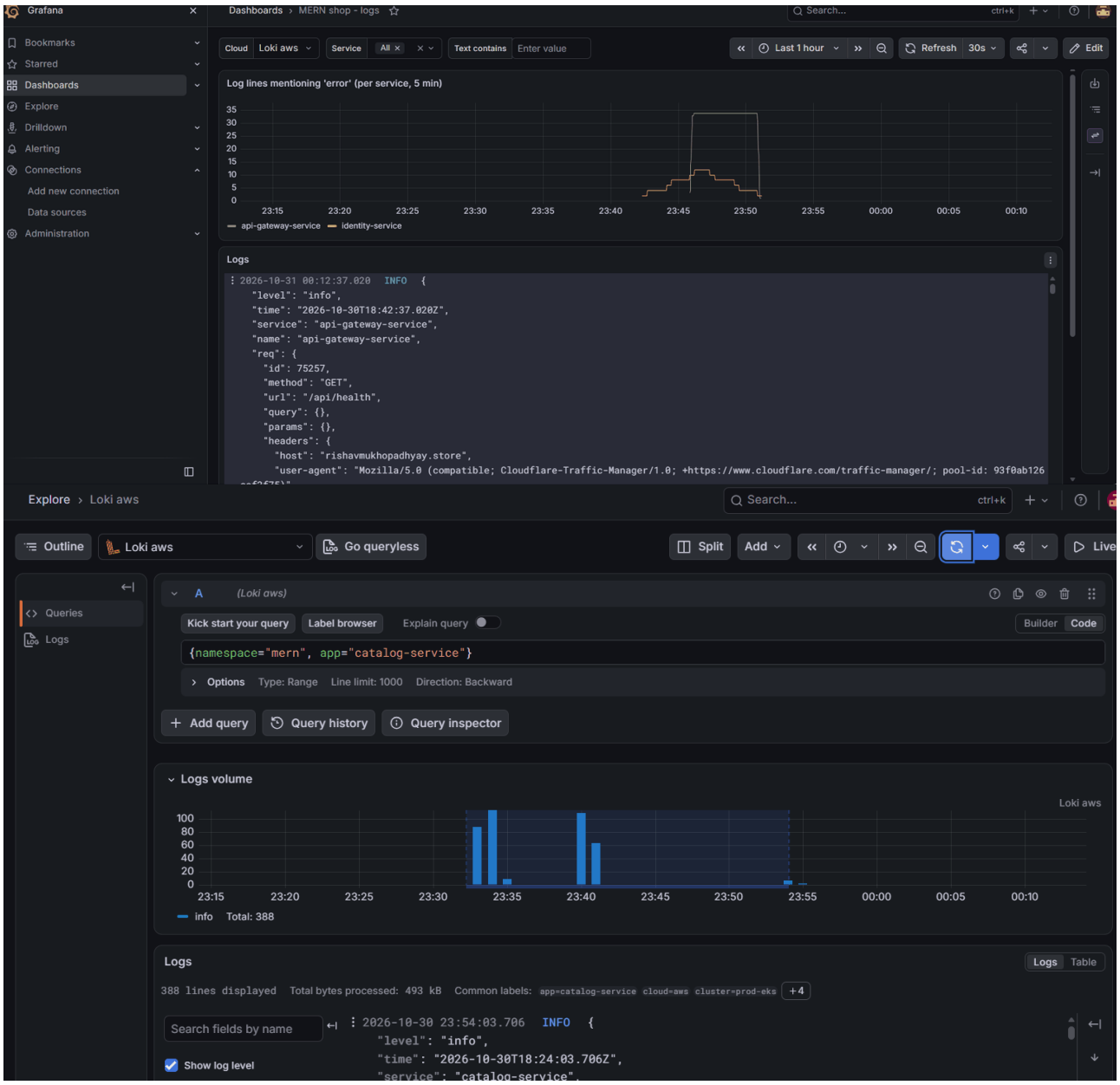
```
#!/bin/bash
# zdt.sh — zero-downtime probe. Ctrl+C prints the totals.
URL_PATH=/api/category/list; LOG=~/zdt-$(date +%H%M).csv; echo "time,cloud,code" > "$LOG"
worker() { while true; do code=$(curl -s -o /dev/null -w '%{http_code}' --max-time 10
"https://$1.rishavmukhopadhyay.store${URL_PATH}"); echo "$(date +%T),$1,${code}" >> "$LOG"; done; }
count() { grep -c ",$1," "$LOG"; }; failed() { grep ",$1," "$LOG" | grep -vc ',200$'; }
for c in gcp azure aws; do worker "$c" & worker "$c" & done
trap 'kill $(jobs -p) 2>/dev/null; for c in gcp azure aws; do echo "$c: $(count $c) requests, $(failed
$c) failed"; done; exit' INT
while true; do sleep 10; l="$(date +%T)"; for c in gcp azure aws; do l="$l | $c $(count $c) req,
$(failed $c) failed"; done; echo "$l"; done
```

E. Monitoring, alerts and cost

Dashboards and logs

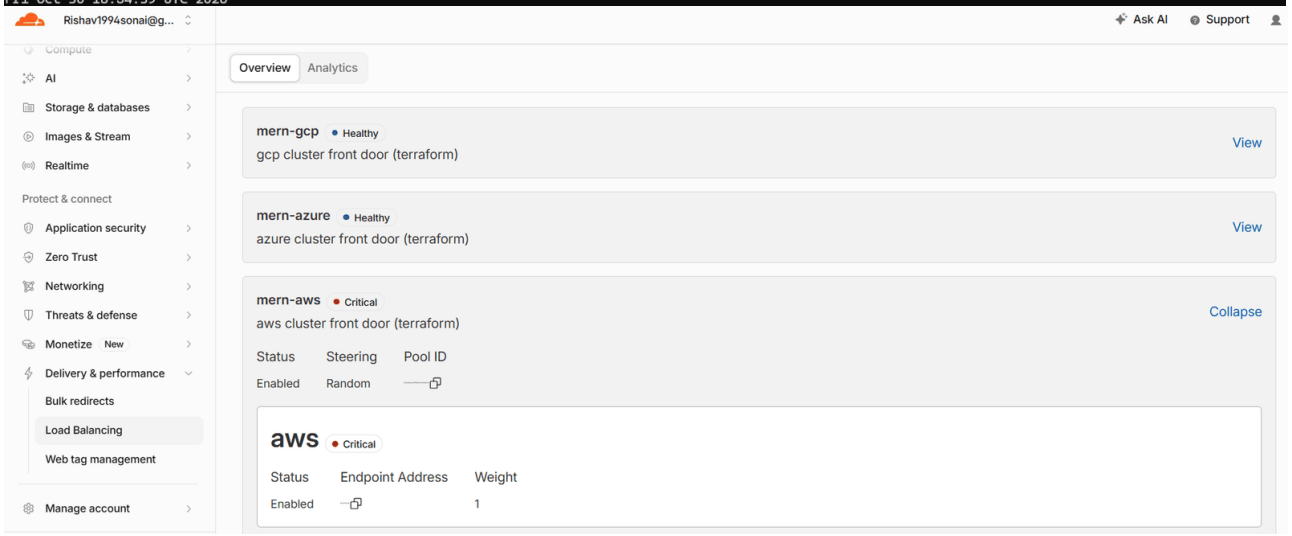

https://grafana.rishavmukhopadhyay.store/connections/datasources
Import favorites
Early Engagement
Grafana
Connections › Data sources
Search...
Bookmarks
Starred
Dashboards
Explore
Drilldown
Alerting
Connections
Add new connection
Data sources
Administration
Data sources
View and manage your connected data source connections
Enable Advisor checks
Add new data source
Search by name or type
Sort by A–Z
Loki aws
Loki | https://obs-aws.rishavmukhopadhyay.store/loki
Loki azure
Loki | https://obs-azure.rishavmukhopadhyay.store/loki
Loki gcp
Loki | http://loki.monitoring.svc:3100
Prometheus aws
Prometheus | https://obs-aws.rishavmukhopadhyay.store/prom
Prometheus azure
Prometheus | https://obs-azure.rishavmukhopadhyay.store/prom
Prometheus gcp
Prometheus | http://kps-prometheus.monitoring.svc:9090
default
Build a dashboard
Explore
https://grafana.rishavmukhopadhyay.store/connections/datasources/edit/loki-gcp
Dashboards › MERN shop - multi-cloud overview
Search...
Cloud
All
Last 1 hour
Refresh
30s
Prometheus gcp
Services with ready pods
6.00
Requests per second
1.33 req/s
Failed requests (5xx)
0.00%
Cluster cost run-rate
$140.30
Requests per second by service
2 req/s
1.5 req/s
1 req/s
0.5 req/s
0 req/s
23:10
23:20
23:30
23:40
23:50
00:00
api-gateway-service
catalog-service
commerce-service
identity-service
notification-service
95th percentile response time
800 ms
600 ms
400 ms
200 ms
0 s
api-gateway-service
catalog-service
commerce-service
notification-service
Dashboards › MERN shop - multi-cloud overview
Search...
Cloud
All
Last 1 hour
Refresh
30s
Prometheus aws
Services with ready pods
6.00
Requests per second
0.67 req/s
Failed requests (5xx)
0.00%
Cluster cost run-rate
$130.82
Requests per second by service
1.5 req/s
1 req/s
0.5 req/s
0 req/s
23:10
23:20
23:30
23:40
23:50
00:00
api-gateway-service
catalog-service
95th percentile response time
400 ms
300 ms
200 ms
100 ms
0 s
23:10
23:20
23:30
23:40
23:50
00:00
api-gateway-service
catalog-service

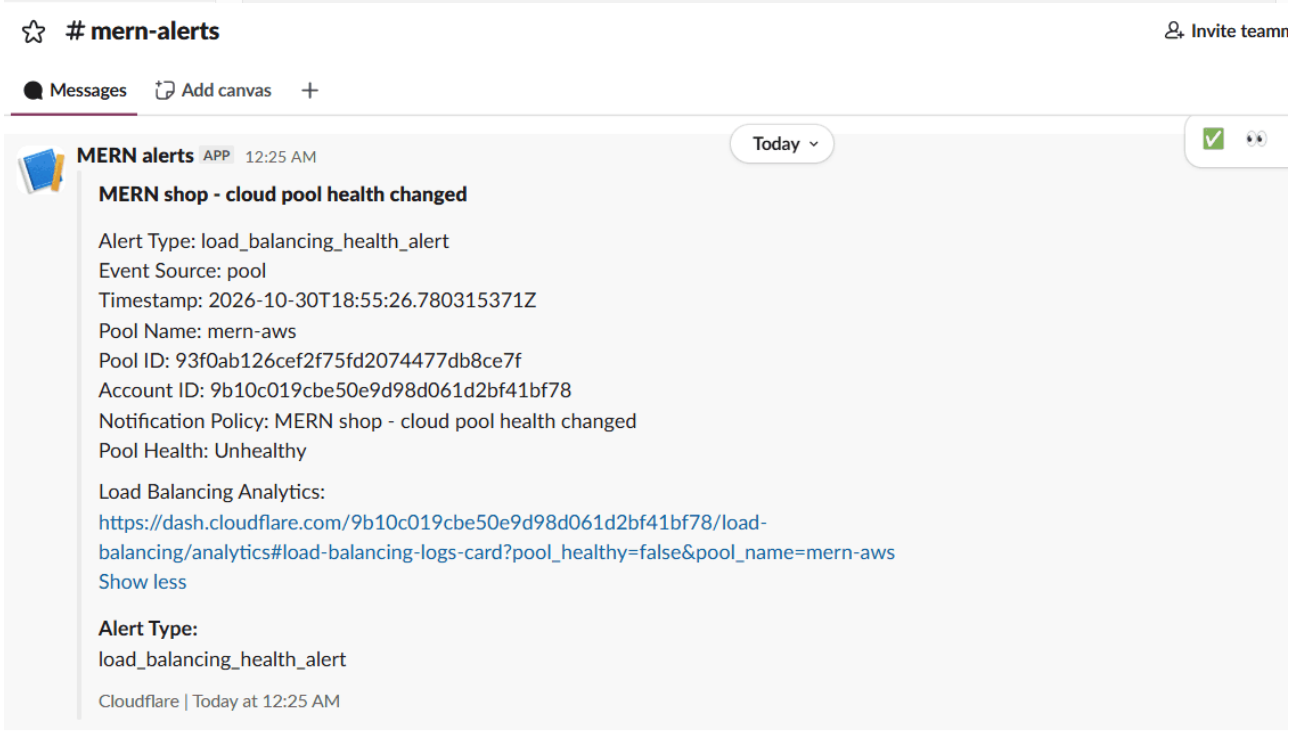

Dashboards › MERN shop - multi-cloud overview
Search... ctrl+k
Cloud All
Last 1 hour
Refresh 30s
Prometheus azure
Services with ready pods
6.00
Requests per second
0.67 req/s
Failed requests (5xx)
0.00%
Cluster cost run-rate
$81.17
Requests per second by service
1.5 req/s
1 req/s
0.5 req/s
0 req/s
23:10 23:20 23:30 23:40 23:50 00:00
api-gateway-service catalog-service
95th percentile response time
500 ms
400 ms
300 ms
200 ms
100 ms
0 s
23:10 23:20 23:30 23:40 23:50 00:00
api-gateway-service catalog-service
Dashboards › MERN shop - multi-cloud overview › View panel
Search... ctrl+k
Cloud Prometheus gcp
Last 1 hour
Refresh 30s
95th percentile response time
800 ms
750 ms
700 ms
650 ms
600 ms
550 ms
500 ms
450 ms
400 ms
350 ms
300 ms
250 ms
200 ms
150 ms
100 ms
50 ms
0 s
23:15 23:20 23:25 23:30 23:35 23:40 23:45 23:50 23:55 00:00 00:05 00:10
api-gateway-service catalog-service commerce-service notification-service



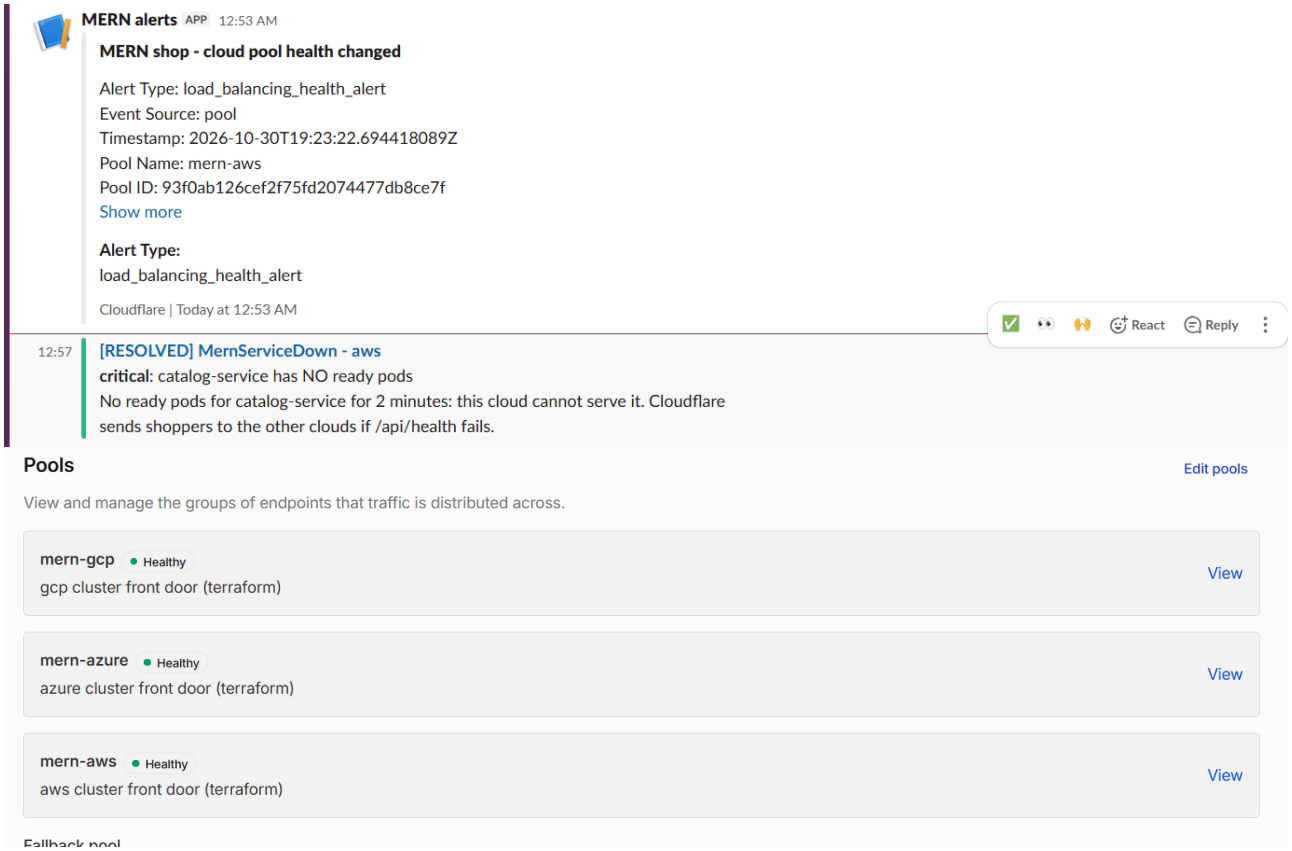
Alert test: catalog-service scaled to 0 in AWS → the AWS pool turned critical, Slack received the Cloudflare pool alert and **MernServiceDown — aws**; the shop stayed up (served by GCP). Scaled back → RESOLVED.

```
(ansible) dell@DESKTOP-LSIMSKL:/mnt/c/Users/dell/Documents/HeroVired/Capstone/repos/platform-automation$ EKS=arn:aws:eks:ap-south-1:722867460821:cluster/prod-eks
kubectl --context $EKS scale deploy catalog-service -n mern --replicas=0
date    # note the time
deployment.apps/catalog-service scaled
Fri Oct 30 18:54:39 UTC 2026
```

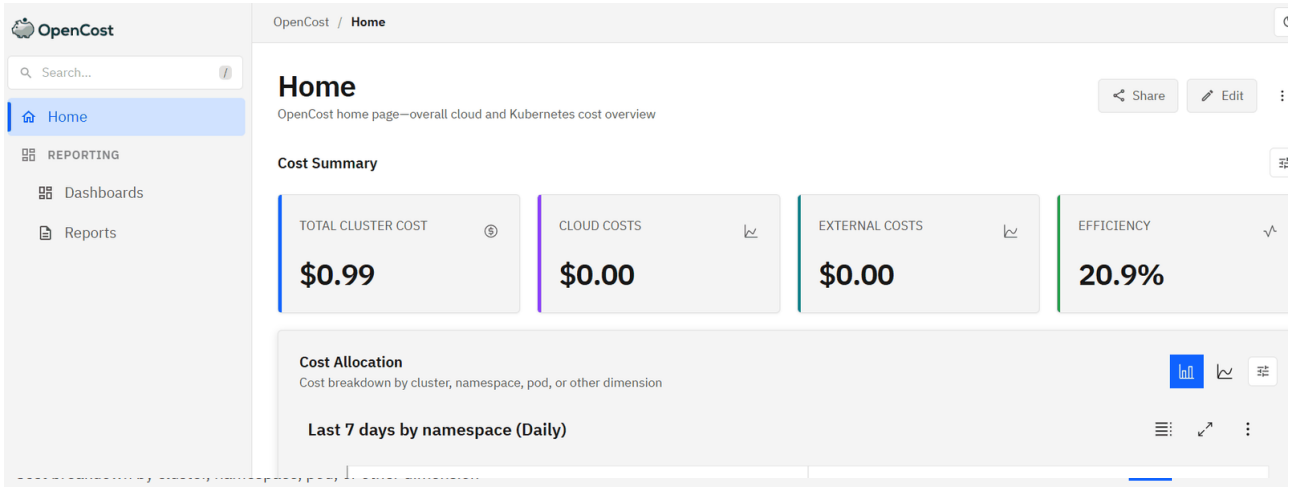


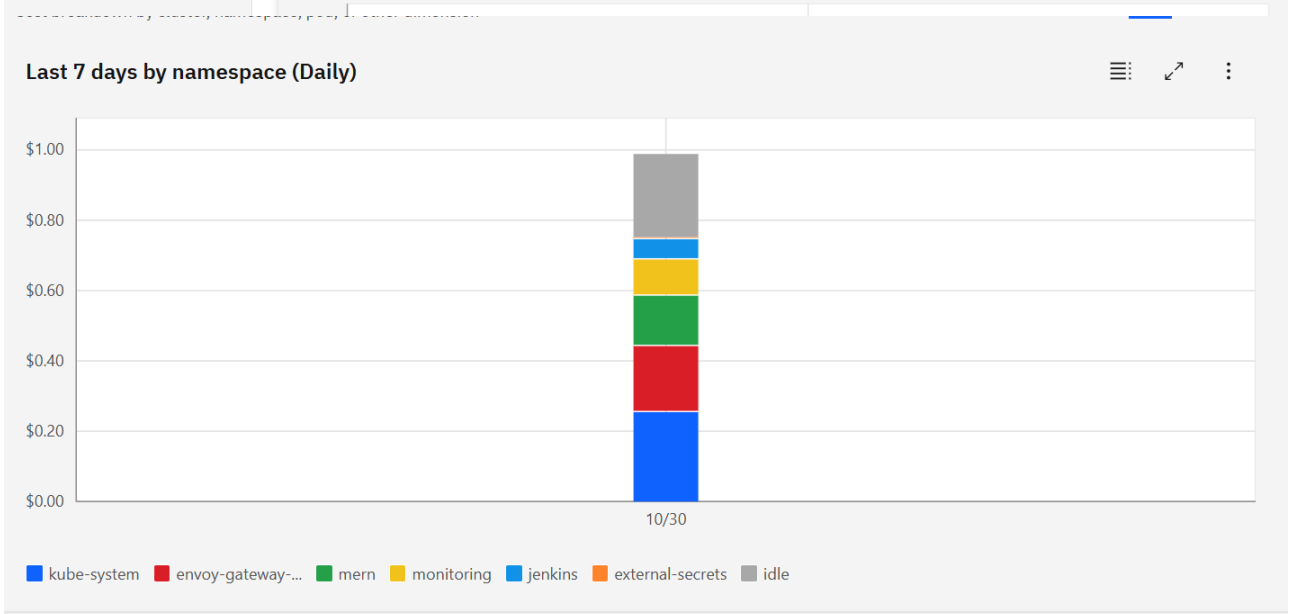


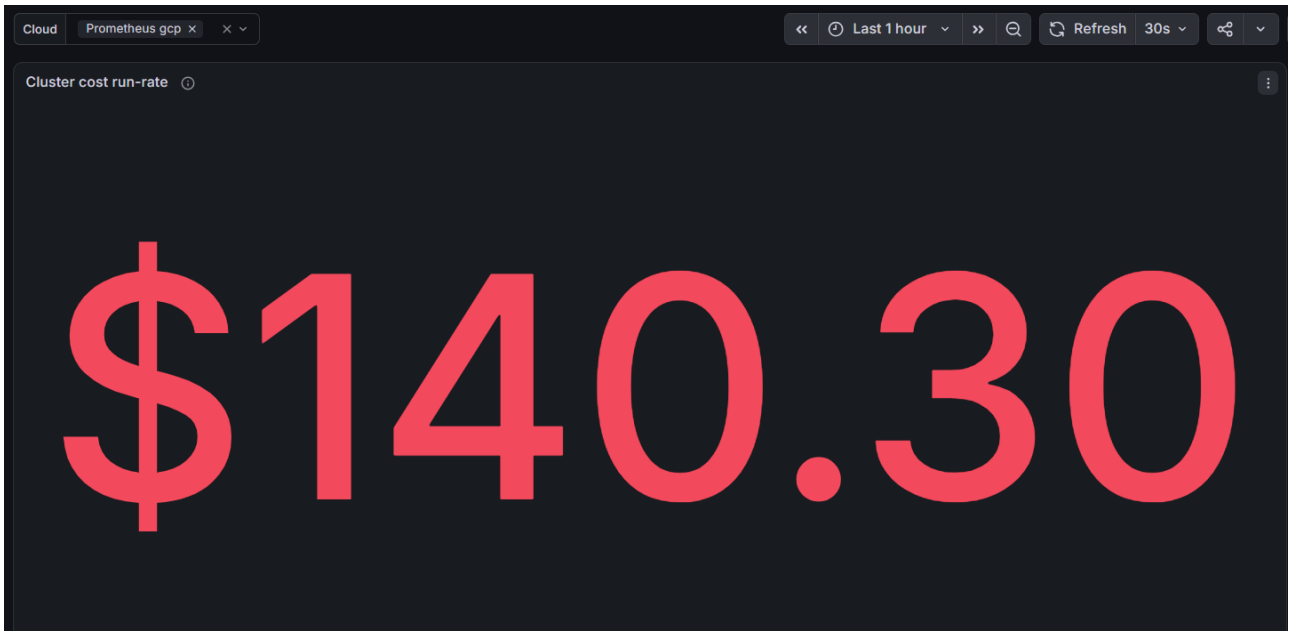

```
(ansible) dell@DESKTOP-LSIMSKL:/mnt/c/Users/dell/Documents/HeroVired/Capstone/repos/platform-automation$ EKS=arn:aws:eks:ap-south-1:722867460821:cluster/prod-eks
kubectl --context $EKS scale deploy catalog-service -n mern --replicas=2
kubectl --context $EKS rollout status deploy catalog-service -n mern
curl -s https://aws.rishavmukhopadhyay.store/api/health; echo
deployment.apps/catalog-service scaled
Waiting for deployment "catalog-service" rollout to finish: 0 of 2 updated replicas are available...
Waiting for deployment "catalog-service" rollout to finish: 1 of 2 updated replicas are available...
deployment "catalog-service" successfully rolled out
{"status":"ok","services":{"identity":"up","catalog":"up","commerce":"up","notification":"up"}}
```

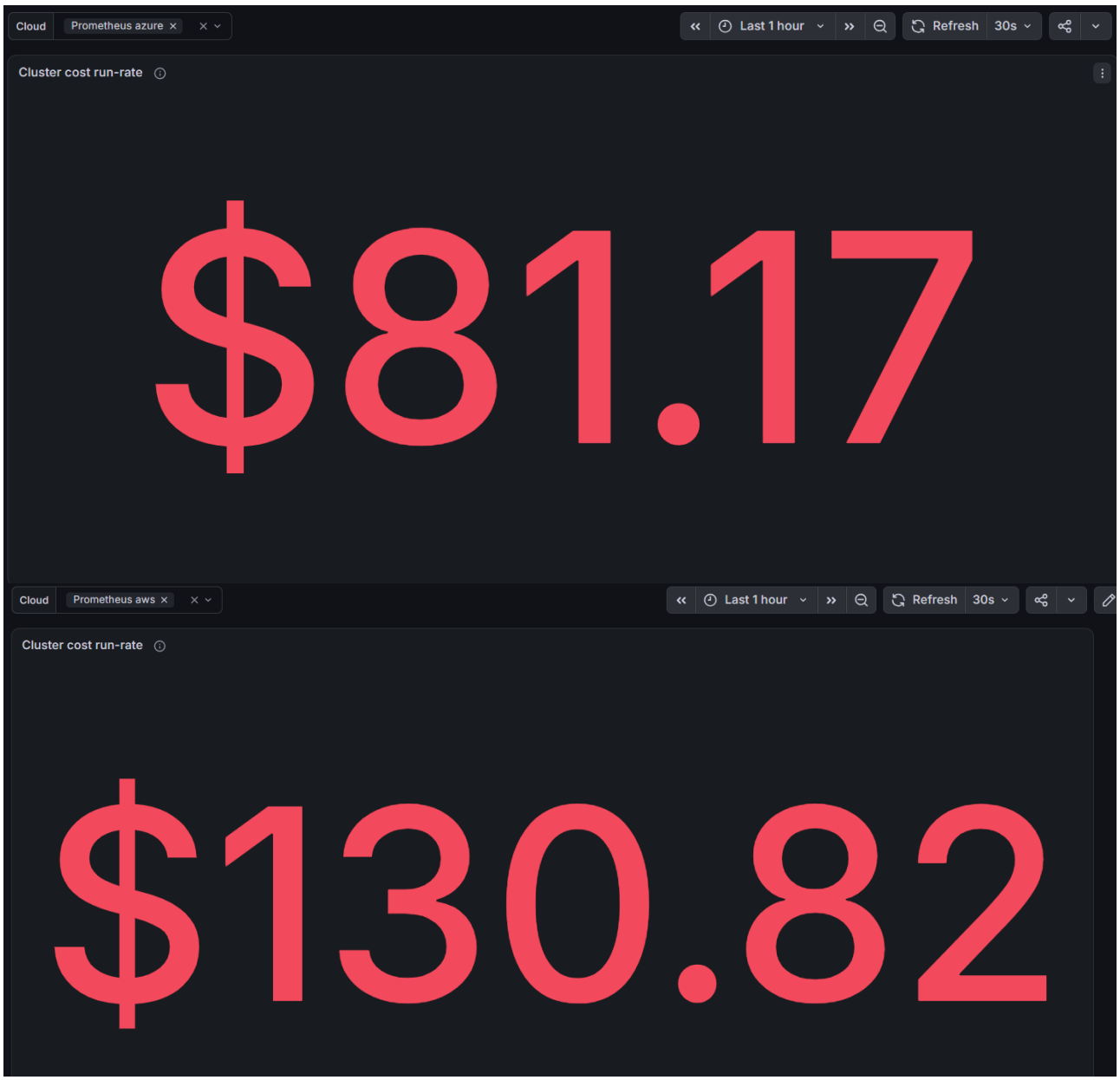

MERN alerts APP 12:53 AM
MERN shop - cloud pool health changed
Alert Type: load_balancing_health_alert
Event Source: pool
Timestamp: 2026-10-30T19:23:22.694418089Z
Pool Name: mern-aws
Pool ID: 93f0ab126cef2f75fd2074477db8ce7f
Show more
Alert Type:
load_balancing_health_alert
Cloudflare | Today at 12:53 AM
React
Reply
12:57
[RESOLVED] MernServiceDown - aws
critical: catalog-service has NO ready pods
No ready pods for catalog-service for 2 minutes: this cloud cannot serve it. Cloudflare sends shoppers to the other clouds if /api/health fails.
Pools
Edit pools
View and manage the groups of endpoints that traffic is distributed across.
mern-gcp Healthy
gcp cluster front door (terraform)
View
mern-azure Healthy
azure cluster front door (terraform)
View
mern-aws Healthy
aws cluster front door (terraform)
View
Fallback pool

Cost


OpenCost
Search...
Home
REPORTING
Dashboards
Reports
OpenCost / Home
Home
OpenCost home page—overall cloud and Kubernetes cost overview
Share
Edit
Cost Summary
TOTAL CLUSTER COST
$0.99
CLOUD COSTS
$0.00
EXTERNAL COSTS
$0.00
EFFICIENCY
20.9%
Cost Allocation
Cost breakdown by cluster, namespace, pod, or other dimension
Last 7 days by namespace (Daily)

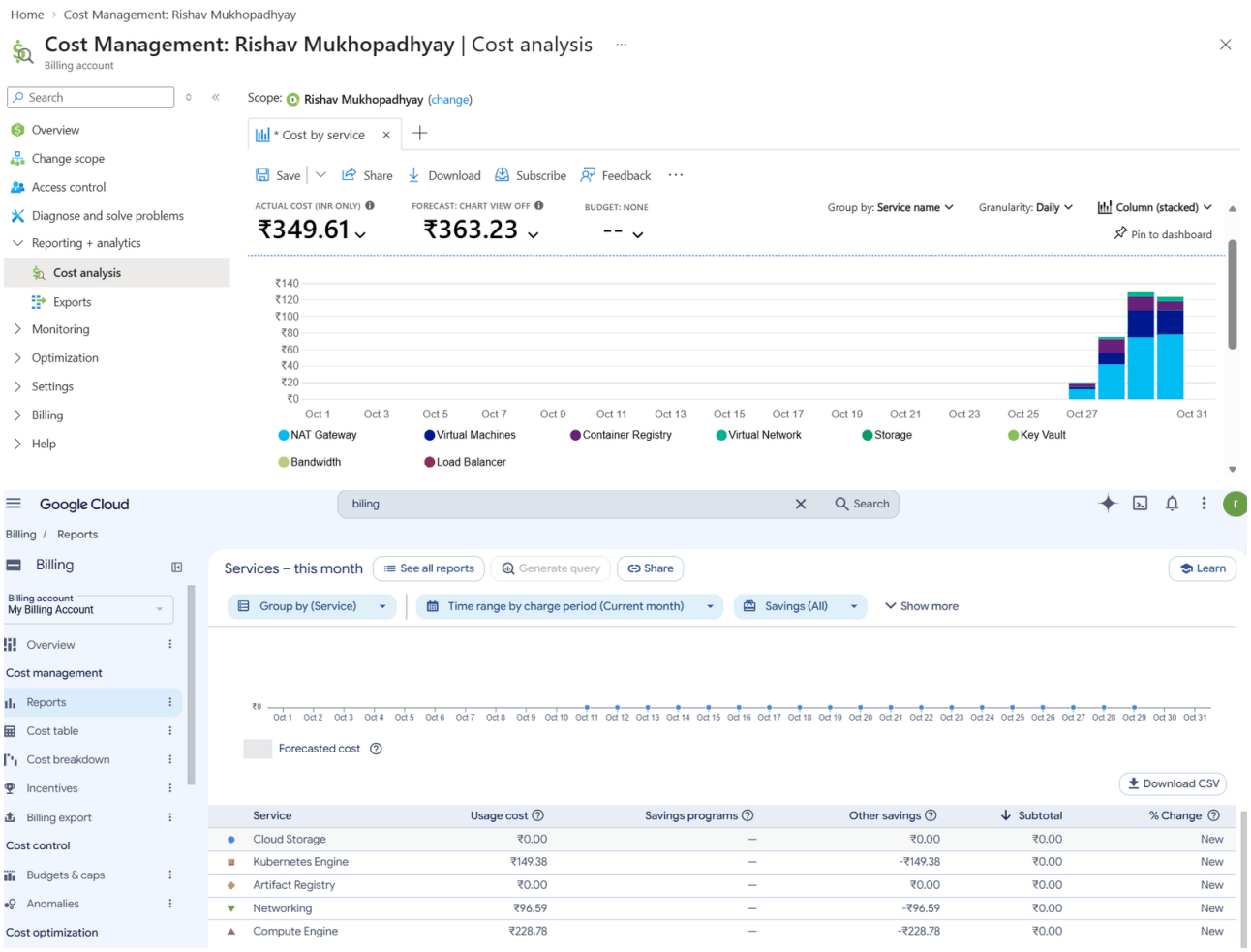

Last 7 days by namespace (Daily)
$1.00
$0.80
$0.60
$0.40
$0.20
$0.00
10/30
kube-system
envoy-gateway-...
mern
monitoring
jenkins
external-secrets
idle


Cloud
Prometheus gcp
Last 1 hour
Refresh
30s
Cluster cost run-rate
$140.30


Cloud
Prometheus azure
Last 1 hour
Refresh
30s
Cluster cost run-rate
$81.17

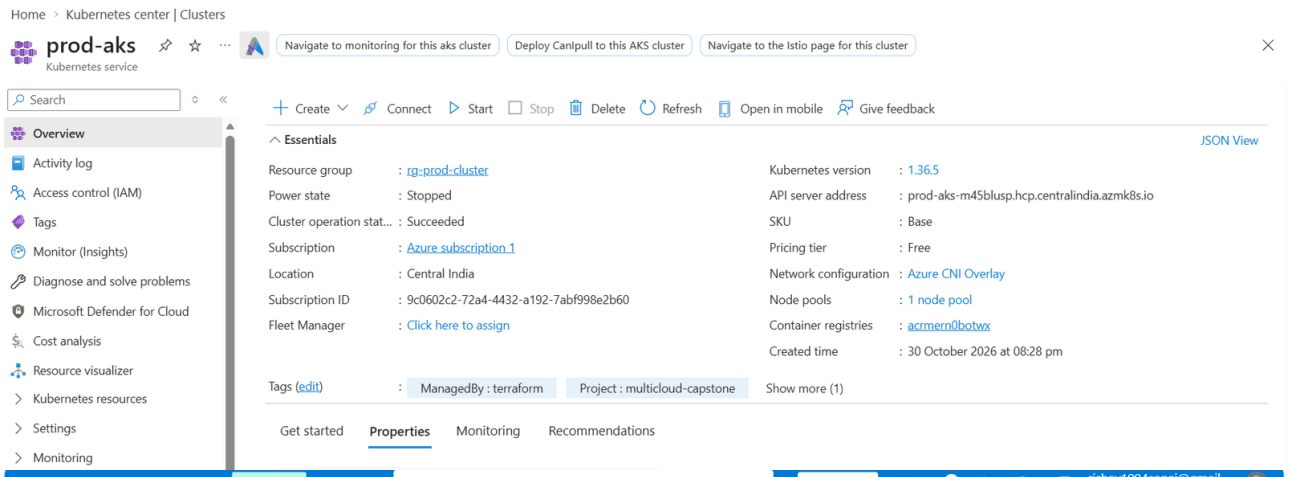
Cloud
Prometheus aws
Last 1 hour
Refresh
30s
Cluster cost run-rate
$130.82

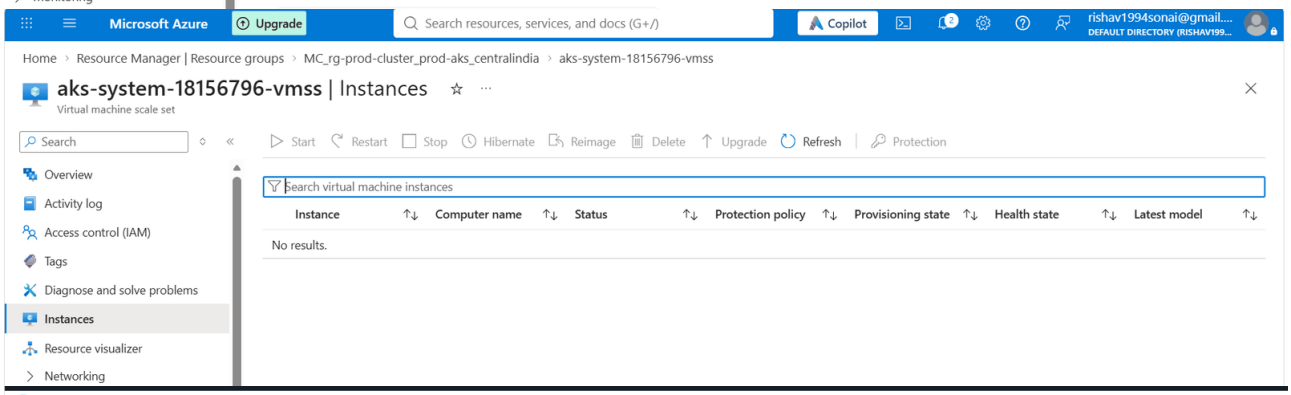

Cost Explorer
New cost and usage report
Total cost
$6.76
Average daily cost
$0.23
Service count
14
Cost and usage graph
Costs ($)
2.5
2
1.5
1
0.5
0
-0.5
Report parameters
Time
Standard
Compare
Date Range
2026-10-01 — 2026-10-30
Displaying month to date
Granularity
Daily
Group by
Dimension
Clear
Service
Filters
Info
Applied filters (0)
Clear all
Service
Clear
Choose services
Linked account
Clear
Choose linked accounts

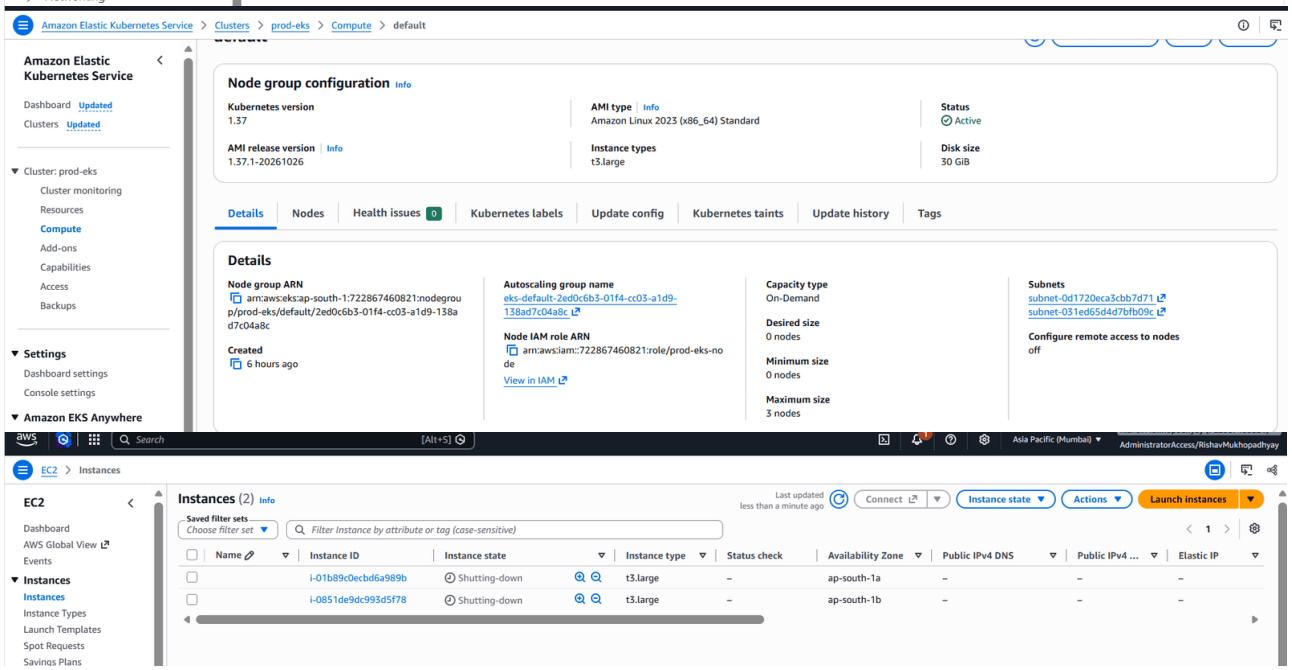


F. Standbys asleep

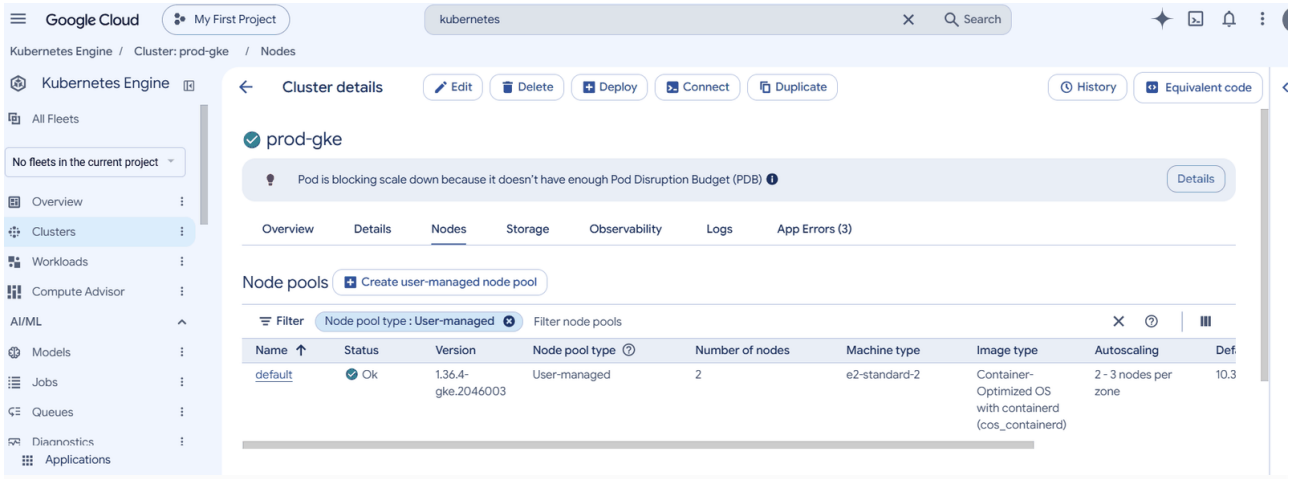
`ansible-playbook playbooks/standby.yml`: AKS stopped, EKS node group at 0, GKE still serving.


Home › Kubernetes center | Clusters
prod-aks
Kubernetes service
Navigate to monitoring for this aks cluster
Deploy CanIpull to this AKS cluster
Navigate to the Istio page for this cluster
Search
Overview
Activity log
Access control (IAM)
Tags
Monitor (Insights)
Diagnose and solve problems
Microsoft Defender for Cloud
Cost analysis
Resource visualizer
Kubernetes resources
Settings
Monitoring
Create
Connect
Start
Stop
Delete
Refresh
Open in mobile
Give feedback
Essentials
JSON View
Resource group : rg-prod-cluster
Power state : Stopped
Cluster operation stat... : Succeeded
Subscription : Azure subscription 1
Location : Central India
Subscription ID : 9c0602c2-72a4-4432-a192-7abf998e2b60
Fleet Manager : Click here to assign
Kubernetes version : 1.36.5
API server address : prod-aks-m45blusp.hcp.centralindia.azmk8s.io
SKU : Base
Pricing tier : Free
Network configuration : Azure CNI Overlay
Node pools : 1 node pool
Container registries : acrmern0botwx
Created time : 30 October 2026 at 08:28 pm
Tags (edit) : ManagedBy : terraform
Project : multicloud-capstone
Show more (1)
Get started
Properties
Monitoring
Recommendations


Microsoft Azure
Upgrade
Search resources, services, and docs (G+/)
Copilot
rishav1994sonai@gmail....
DEFAULT DIRECTORY (RISHAV199...
Home › Resource Manager | Resource groups › MC_rg-prod-cluster_prod-aks_centralindia › aks-system-18156796-vmss
aks-system-18156796-vmss | Instances
Virtual machine scale set
Search
Overview
Activity log
Access control (IAM)
Tags
Diagnose and solve problems
Instances
Resource visualizer
Networking
Start
Restart
Stop
Hibernate
Reimage
Delete
Upgrade
Refresh
Protection
Search virtual machine instances
Instance
Computer name
Status
Protection policy
Provisioning state
Health state
Latest model
No results.


Amazon Elastic Kubernetes Service › Clusters › prod-eks › Compute › default
Amazon Elastic Kubernetes Service
Dashboard Updated
Clusters Updated
Cluster: prod-eks
Cluster monitoring
Resources
Compute
Add-ons
Capabilities
Access
Backups
Settings
Dashboard settings
Console settings
Amazon EKS Anywhere
Node group configuration Info
Kubernetes version
1.37
AMI type | Info
Amazon Linux 2023 (x86_64) Standard
Status
Active
AMI release version | Info
1.37.1-20261026
Instance types
t3.large
Disk size
30 GiB
Details
Nodes
Health issues 0
Kubernetes labels
Update config
Kubernetes taints
Update history
Tags
Details
Node group ARN
arn:aws:eks:ap-south-1:722867460821:nodegroup/prod-eks/default/2ed0c6b3-01f4-cc03-a1d9-138ad7c04a8c
Created
6 hours ago
Autoscaling group name
eks-default-2ed0c6b3-01f4-cc03-a1d9-138ad7c04a8c
Node IAM role ARN
arn:aws:iam::722867460821:role/prod-eks-node
View in IAM
Capacity type
On-Demand
Desired size
0 nodes
Minimum size
0 nodes
Maximum size
3 nodes
Subnets
subnet-0d1720eca3cbb7d71
subnet-031ed65d4d7bfb09c
Configure remote access to nodes
off

EC2 › Instances
EC2
Dashboard
AWS Global View
Events
Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Instances (2) Info
Last updated less than a minute ago
Connect
Instance state
Actions
Launch instances
Saved filter sets
Choose filter set
Filter Instance by attribute or tag (case-sensitive)
Name
Instance ID
Instance state
Instance type
Status check
Availability Zone
Public IPv4 DNS
Public IPv4 ...
Elastic IP
i-01b89c0ecbd6a989b
Shutting-down
t3.large
ap-south-1a
i-0851de9dc993d5f78
Shutting-down
t3.large
ap-south-1b



Overview | Analytics

Pools

Edit pools

View and manage the groups of endpoints that traffic is distributed across.

mern-gcp Healthy
gcp cluster front door (terraform)
View

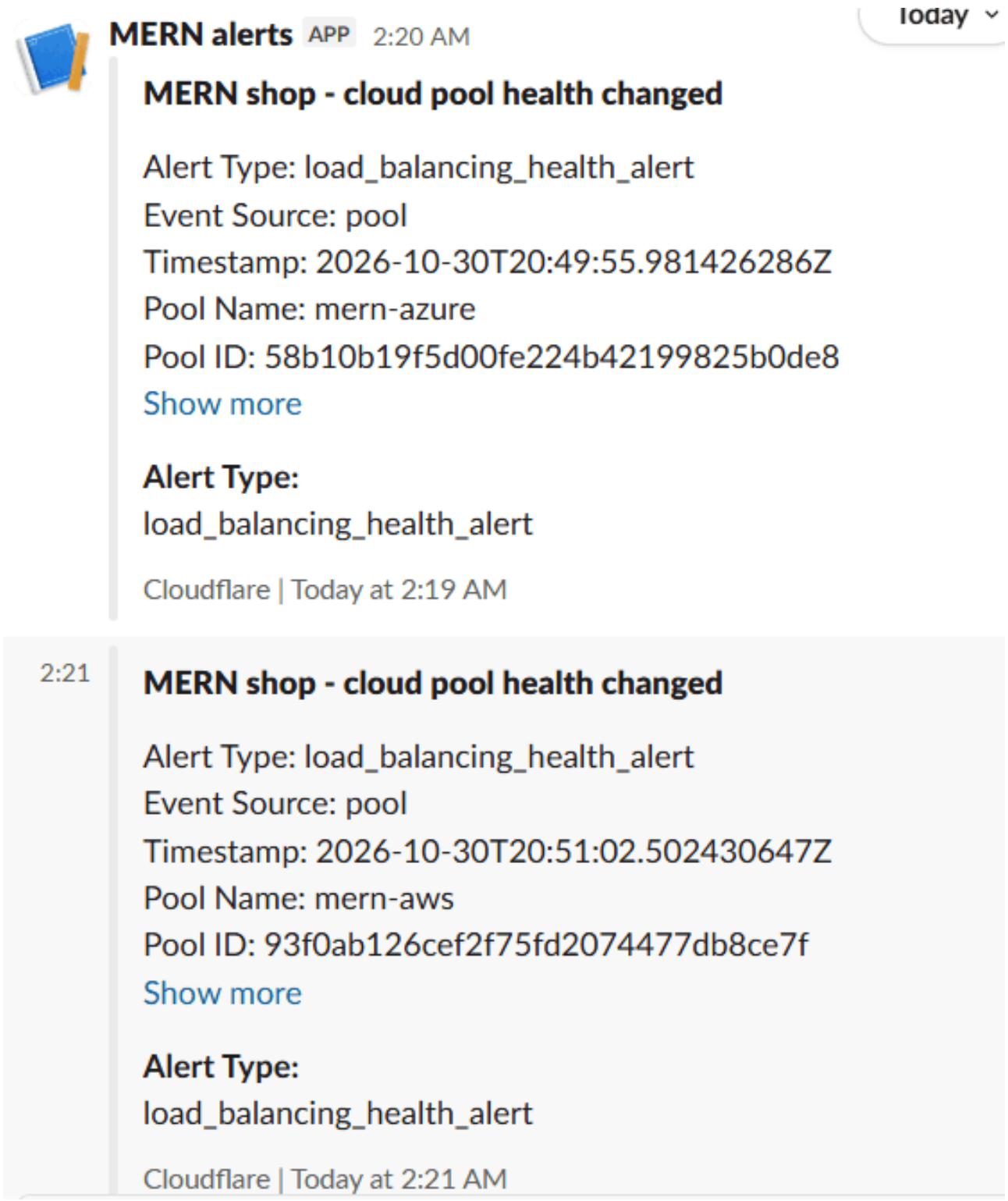
mern-azure Critical
azure cluster front door (terraform)
View

mern-aws Critical
aws cluster front door (terraform)
View

Fallback pool

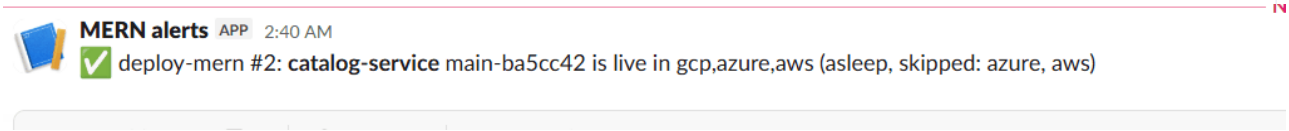
In the event that all pools are marked down, Cloudflare uses the fallback pool as a last resort for successfully sending traffic to an endpoint.

mern-gcp



Jenkins `deploy-mern` with the running tag: GCP deployed, Azure and AWS skipped as sleeping standbys.

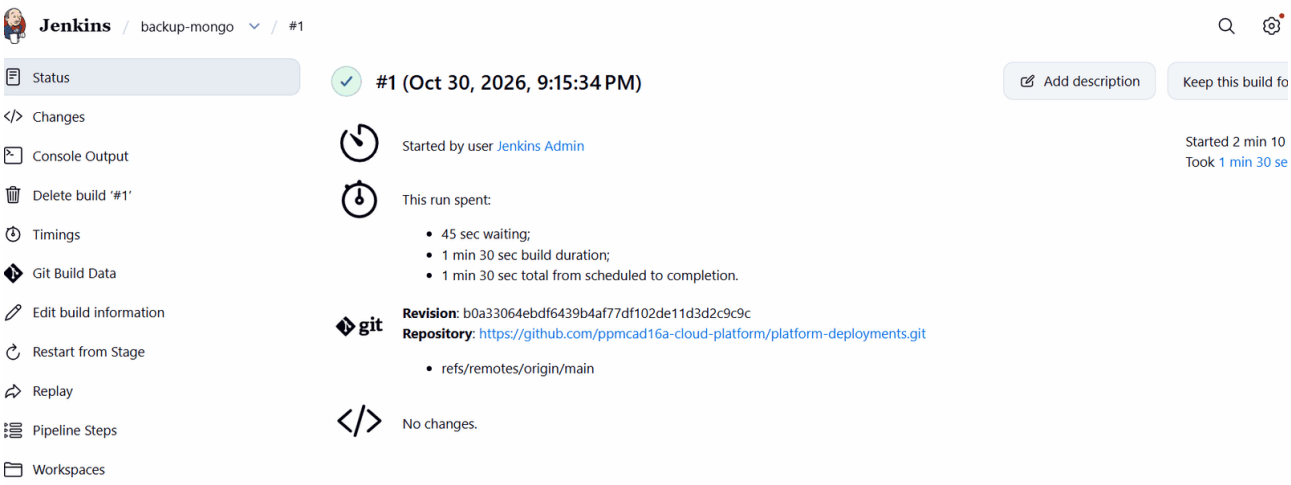
```
[Pipeline] { (Azure: awake?)
Did you forget the `def` keyword? WorkflowScript seems to be setting a field named currentStep (to a value of type String)
which could lead to memory leaks or other issues.
[Pipeline] container
[Pipeline] {
[Pipeline] script
[Pipeline] {
[Pipeline] sh
+ set +x
[Pipeline] echo
azure: Stopped - sleeping standby, skipped
[Pipeline] { (AWS: awake?)
Did you forget the `def` keyword? WorkflowScript seems to be setting a field named currentStep (to a value of type String)
which could lead to memory leaks or other issues.
[Pipeline] container
[Pipeline] {
[Pipeline] script
[Pipeline] {
[Pipeline] sh
+ aws eks describe-nodegroup --cluster-name prod-eks --nodegroup-name default --query nodegroup.scalingConfig.desiredSize --
output text
[Pipeline] echo
aws: 0 nodes - sleeping standby, skipped
```



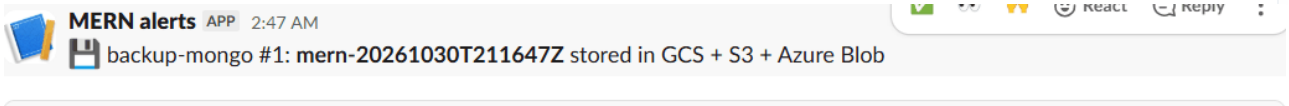
G. Disaster recovery

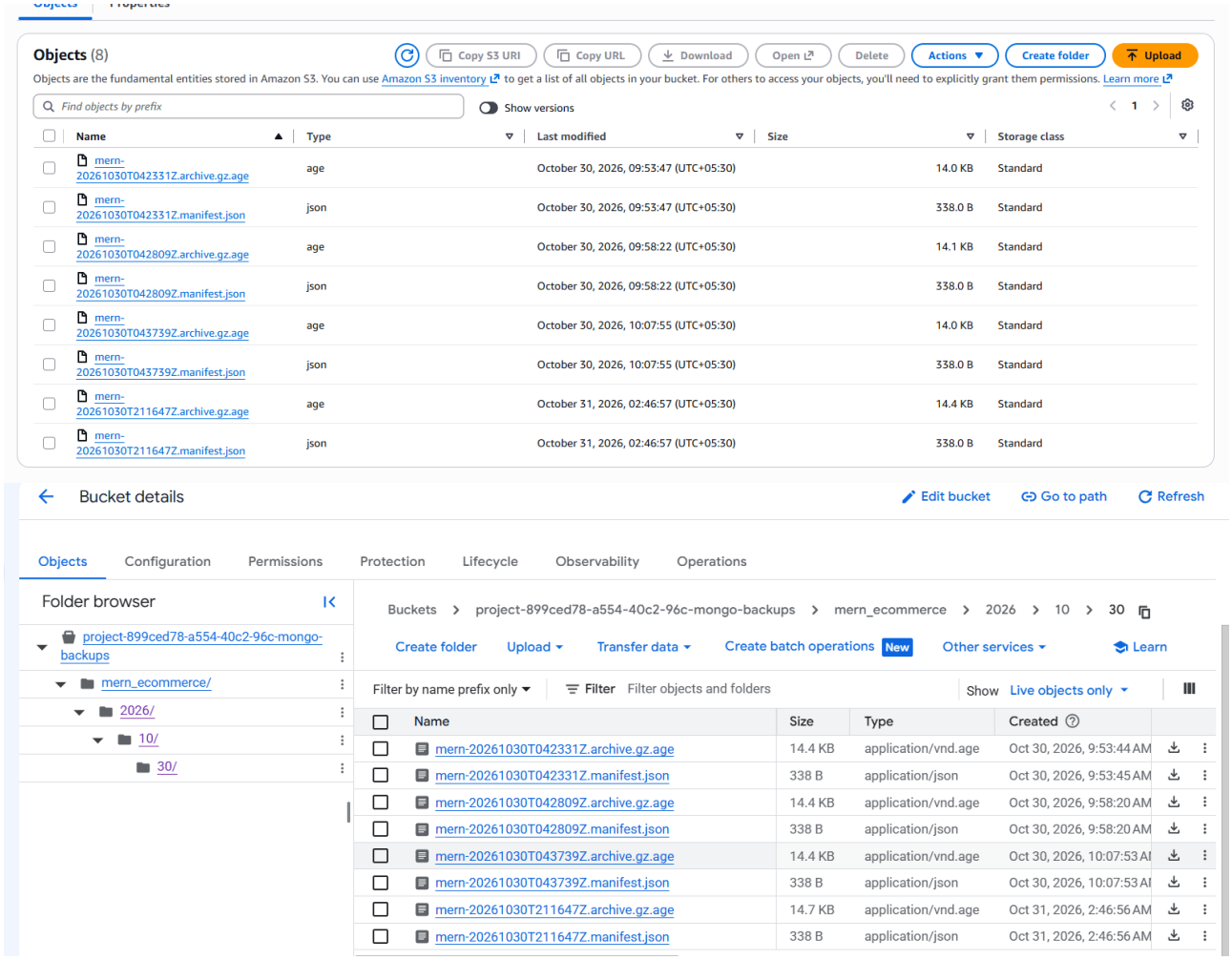
G1. Backup to three clouds





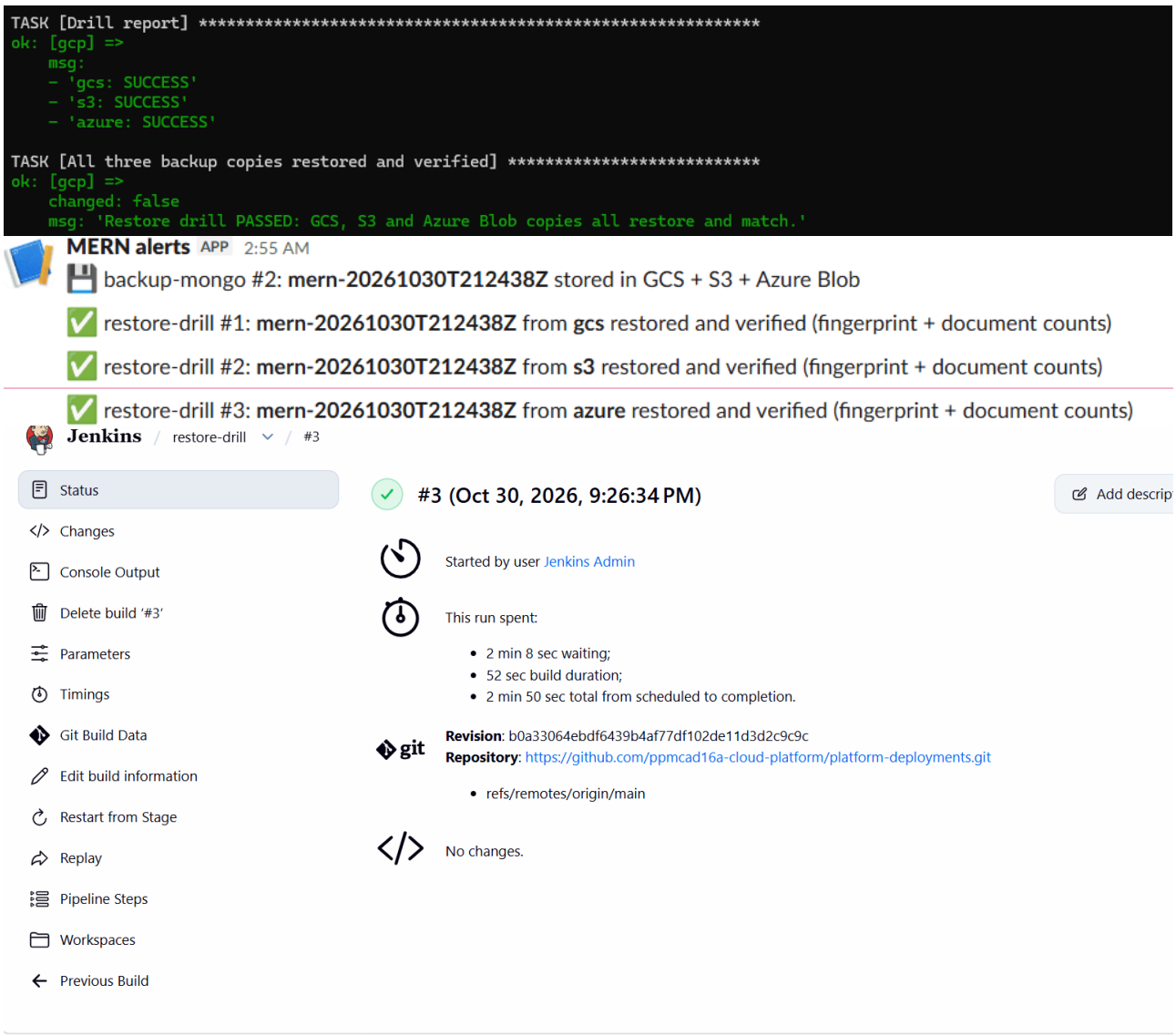

```
[Pipeline] sh
+ rm -f .backup-uri
+ cat mern-20261030T211647Z.manifest.json
{"name":"mern-20261030T211647Z","database":"mern_ecommerce","created":"2026-10-
30T21:16:52Z","sha256":"e28b65b127e1e24a17eedf2d8dc62bd68d0f16e2956b7c38cc4ccbe5452a8b05","bytes":14711,"counts":
{"addresses":0,"brands":10,"carts":13,"categories":10,"contacts":8,"merchants":1,"orders":13,"products":100,"reviews":2,"users
":5,"wishlists":1}}
[Pipeline] }
[Pipeline] // container
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Upload to 3 clouds)
[Pipeline] container
[Pipeline] {
[Pipeline] sh
+ sh scripts/dr/upload.sh mern-20261030T211647Z
2026/10/30 21:16:55 NOTICE: Config file "/config/rclone/rclone.conf" not found - using defaults
stored mern-20261030T211647Z in gcs:project-899ced78-a554-40c2-96c-mongo-backups/mern_ecommerce/2026/10/30/
2026/10/30 21:16:56 NOTICE: Config file "/config/rclone/rclone.conf" not found - using defaults
stored mern-20261030T211647Z in s3:ppmcad16a-mongo-backups-722867460821-ap-south-1/mern_ecommerce/2026/10/30/
```





G2. Restore drill from three clouds

```
ansible-playbook playbooks/drill-restore.yml
```



G3. Failover drill: GCP goes down

`ansible-playbook playbooks/drill-failover.yml` — GCP's gateway scaled to 0 while both standbys slept; the wake-up was **automatic** (Cloudflare → Worker → GitHub Actions).

View and manage the groups of endpoints that traffic is distributed across.

mern-gcp • Critical
gcp cluster front door (terraform) — View

mern-azure • Critical
azure cluster front door (terraform) — View

mern-aws • Critical
aws cluster front door (terraform) — View

Fallback pool

MERN alerts APP 3:03 AM

MERN shop - cloud pool health changed

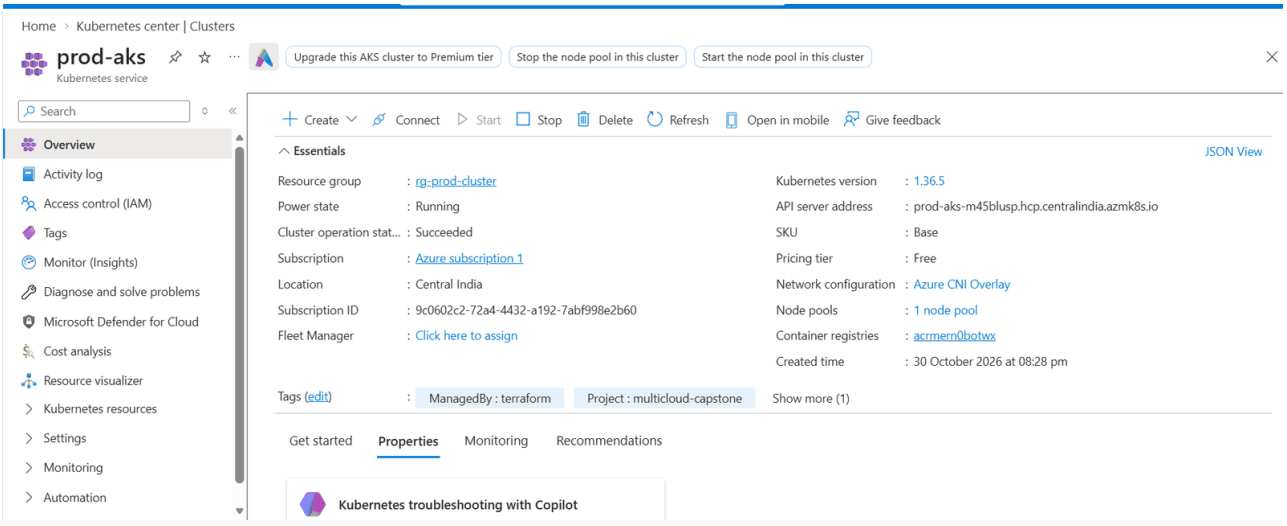
Alert Type: load_balancing_health_alert
Event Source: pool
Timestamp: 2026-10-30T21:33:19.665983624Z
Pool Name: mern-gcp
Pool ID: 388b0b8c57a4e8e30b984c01e60970d1
Show more

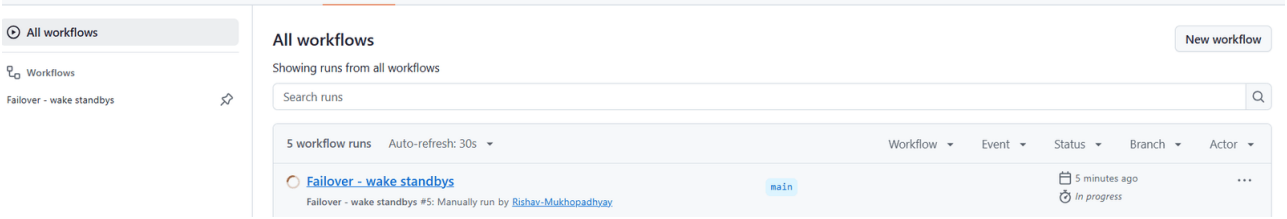
Alert Type:
load_balancing_health_alert

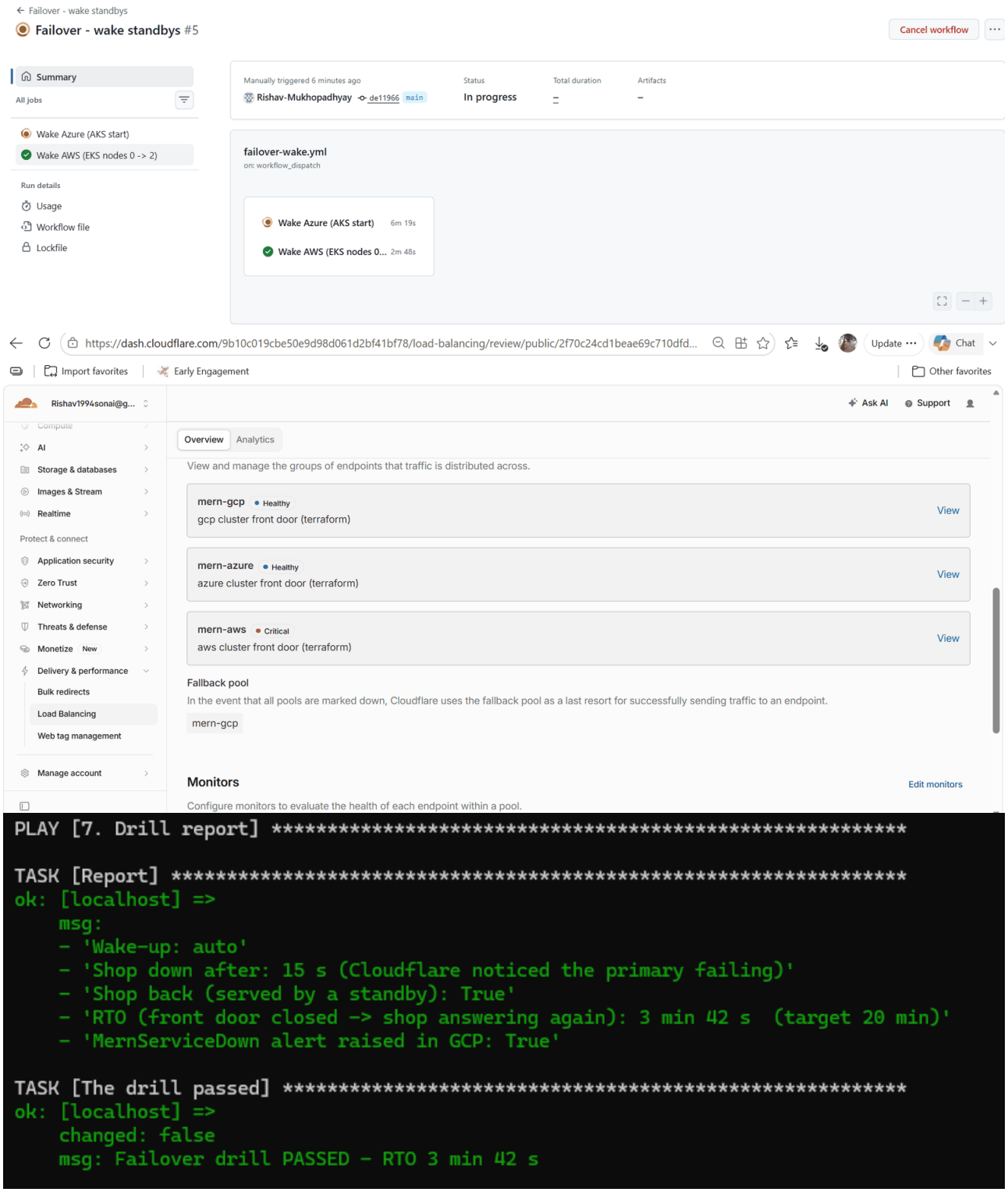
Cloudflare | Today at 3:03 AM

✅ 👀 🙌 React Reply

3:05 **[FIRING] MernServiceDown - gcp**
critical: api-gateway-service has NO ready pods
No ready pods for api-gateway-service for 2 minutes: this cloud cannot serve it.
Cloudflare sends shoppers to the other clouds if /api/health fails.







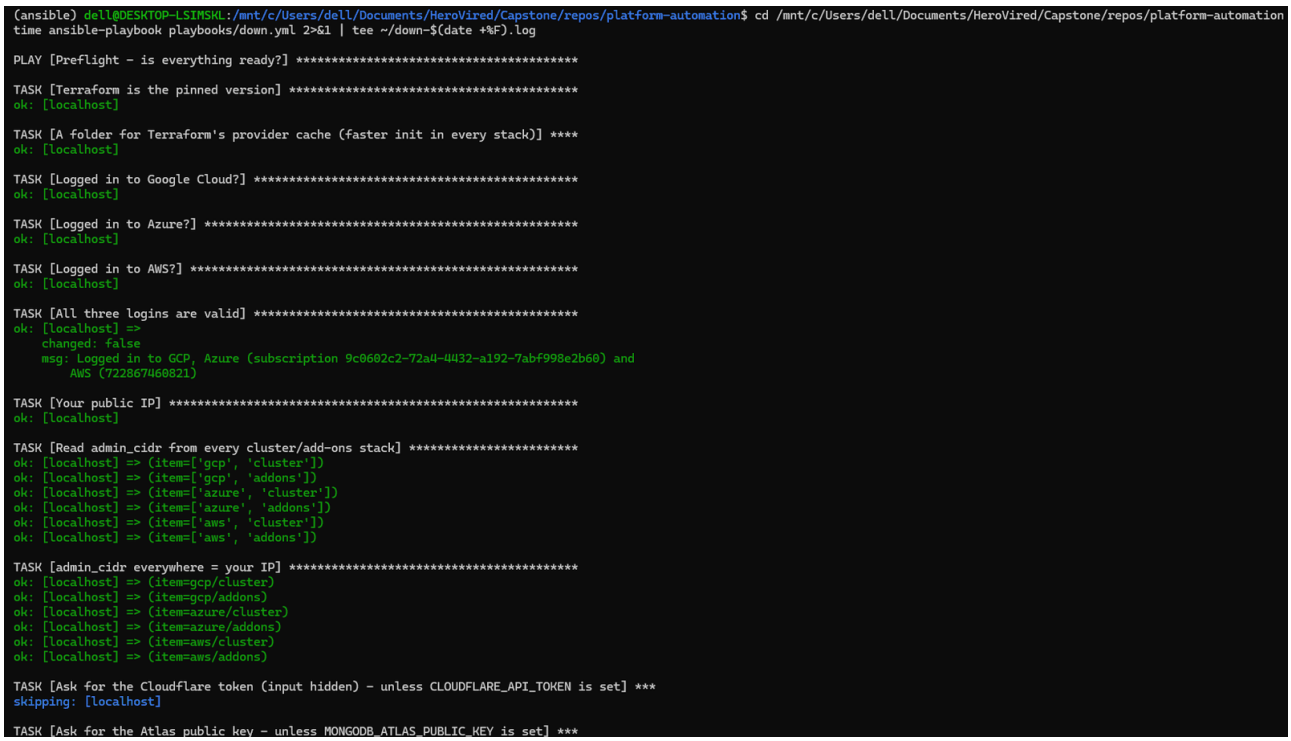

```
PLAY [7. Drill report] ***************************************************

TASK [Report] ************************************************************
ok: [localhost] =>
    msg:
    - 'Wake-up: auto'
    - 'Shop down after: 15 s (Cloudflare noticed the primary failing)'
    - 'Shop back (served by a standby): True'
    - 'RTO (front door closed -> shop answering again): 3 min 42 s  (target 20 min)'
    - 'MernServiceDown alert raised in GCP: True'

TASK [The drill passed] **************************************************
ok: [localhost] =>
    changed: false
    msg: Failover drill PASSED - RTO 3 min 42 s
```

H. Teardown

`ansible-playbook playbooks/down.yml` — wake standbys → add-ons destroy → state empty → leftover check → cluster destroy, AKS/EKS first, GKE last, then the global layer.



down.yml preflight

The teardown needed four runs because of issues 1–3 and 5 above; each stopped safely with nothing half-deleted. Final state: see the end-of-session check in Runbook §2.

10 — Troubleshooting

Every entry below happened to us. Format: **symptom → cause → fix**.

Sessions (`up.yml` / `down.yml`) and WSL

WSL downloads time out after 30 s

Symptom: `terraform plan/destroy` of an add-ons stack fails with `Error locating chart … release-assets.githubusercontent.com … context deadline exceeded (Client.Timeout exceeded while awaiting headers)`; only some runs fail. **Cause:** one of GitHub's four download addresses (`185.199.109.133`) was unreachable from the home ISP. Each download picks an address at random; Helm's client waits 30 s on the dead one. Windows browsers hide the problem by retrying the next address quickly. **Diagnose:**

```
for ip in 185.199.108.133 185.199.109.133 185.199.110.133 185.199.111.133; do
  curl -s -o /dev/null -w "$ip %{time_connect}s rc=%{exitcode}\n" --max-time 8 \
    --resolve release-assets.githubusercontent.com:443:$ip https://release-
assets.githubusercontent.com/; done
```

Fix: pin a working address for the session: `echo "185.199.108.133 release-assets.githubusercontent.com objects.githubusercontent.com raw.githubusercontent.com" | sudo tee -a /etc/hosts` (WSL rewrites `/etc/hosts` at restart). *Lesson:* we first suspected WSL's DNS relay — measuring connect time per address found the real cause.

AWS steps fail with `refresh cached SSO token failed … InvalidRequestException`

Cause: the AWS SSO session expired. `aws sts get-caller-identity` can still pass on cached short-term keys while Terraform cannot renew them, so a failed host drops out of the play and the global layer fails later. **Fix:** `aws sso login --profile HeroviredProfile` at the start of every session, then re-run the playbook.

Your IP changed mid-session — `kubectl dial tcp …:443: i/o timeout`, Atlas `IP_ADDRESS_NOT_ON_ACCESS_LIST`

Cause: home/hotspot IPs change; the cluster APIs and the Atlas API key only accept the old one. **Fix:** Runbook §6 — update the six `terraform.tfvars`, the Atlas API key access list, then re-apply the cluster stacks.

`down.yml` stops: "Something the add-ons created is still there (k8s-envoygat-…)"

Cause: the AWS Load Balancer Controller was removed before it deleted Envoy's NLB. The safety stop keeps the cluster so nothing is orphaned. **Fix:** wait 5 min and re-run, or delete the NLB, its target group and the two `k8s-*` security groups by hand (Runbook §7).

AWS standby "asleep" but EC2 instances keep running for up to 30 min

Cause: scaling the node group to 0 drains the nodes; PodDisruptionBudgets (`minAvailable: 1`) refuse to evict the last copy while every node is going away; the termination hook times out after 30 min. **Fix:** `kubectl drain -l eks.amazonaws.com/nodegroup=default --disable-eviction --force --ignore-daemonsets --delete-emptydir-data`.

`az aks stop` → `OperationNotAllowed … in-progress start managed cluster operation`

Cause: a standby woken by the failover drill was still starting when the drill put it back to sleep. **Fix:** wait for `provisioningState = Succeeded`, then stop.

Never run `terraform destroy -refresh=false` on add-ons

What happened: with a stale GKE token the Helm provider skipped the uninstalls but the state was emptied — releases kept running with no state. **Fix:** restored the previous state version from the versioned bucket (`gcloud storage cp "<obj>#<generation>" <obj>`) and destroyed normally. State buckets are versioned exactly for this.

Global teardown blocked by an "update" of `cloudflare_notification_policy_webhooks.slack`

Cause: the provider plans an update on every run (read-only fields come back empty). **Fix:** the teardown guard allows `[delete, update]` for the global stack — never create or replace.

Ansible `ANSIBLE_CONFIG` ignored / play names with inventory variables fail / strict booleans

Cause: an old export in `~/.bashrc`; Ansible 2.21 does not template play names with host variables and requires real booleans in `when`/`until`. **Fix:** `sed -i '/ANSIBLE_CONFIG/d' ~/.bashrc`; plain-text play names; write conditions like `x is defined and x is not none`.

CI/CD

Symptom	Cause	Fix
`Not authorized to perform sts:AssumeRoleWithWebIdentity`	trust expects GitHub's immutable subject (org id + repo id)	use `repo:<org>@<org-id>/<repo>@<repo-id>:ref:refs/heads/main`
PR shows "no checks" / `startup_failure`	caller passes an input the referenced `@v2` does not know yet	merge + tag `core-cloud-platform` first, then re-run
`base branch policy prohibits the merge` right after `gh pr create`	checks not registered yet	`gh pr merge --squash --delete-branch --auto`
Trivy fails on a new CVE	vulnerability published after the last build	update the package; never disable the gate
`docker login` to ECR gives 400 in Windows PowerShell 5.1	PS 5.1 pipe re-encodes the token	`cmd /c "aws ecr get-login-password … | docker login … --password-stdin …"`
Jenkins `awk: command not found`	`azure-cli` image has no awk	use `grep`/`cut`/`tr`
a token visible in a Jenkins log	`sh` runs with `set -x`	`set +x` before commands that use tokens; delete the build
Helm `context deadline exceeded`, then `cannot re-use a name` on AKS	images from ghcr.io pulled very slowly in Azure	mirror the images into ACR (`up.yml` does it)
failover Worker gets 403 from GitHub	fine-grained PAT waiting for org-owner approval	approve it in the org settings

Data and security

Symptom	Cause	Fix
Atlas IP access list contains `0.0.0.0/0`	added by hand in early local development	removed on 31 Oct 2026; only Terraform-managed cluster IPs should remain
backup upload to Azure: `403` on HEAD	rclone reads before writing; the identity is write-only	upload with two REST `Put Blob` calls (`upload-azure.sh`)
backup to GCS: bucket check refused	rclone tries to create/check the bucket	`RCLONE_CONFIG_GCS_NO_CHECK_BUCKET=true`
orphaned AWS EBS volumes after teardown	the EBS CSI policy was detached before the volumes were deleted	`depends_on` on the policy attachment + leftover check in `down.yml`
`terraform init -upgrade` pulled a brand-new provider	upgrade ignores the lock file	always plain `terraform init`; adopt versions ≥ 14 days old

11 — Limitations and possible improvements

An honest list of what the platform does **not** do yet, why, and what we would do next.

Known limitations

#	Limitation	Impact	Mitigation today
1	**One shared database** (MongoDB Atlas M0, single region in AWS Mumbai)	the database is a shared dependency for all three clouds; the problem statement's "replicate databases across clouds" is met by encrypted copies, not live replication	nightly encrypted backups in GCS, S3 and Azure Blob + restore drills; RPO ≤ 24 h
2	**Terraform is applied by Ansible from the admin laptop**, not by a GitHub Actions pipeline	the problem statement asks for CI/CD of Terraform; changes are reviewed in pull requests but applied by hand	every apply is guarded (plan → check change types → apply that plan); the GitHub plan/apply identities already exist in each `bootstrap/`
3	**Jenkins and the central Grafana run in GKE** (the primary)	during a GCP outage there is no automatic deploy and no central dashboard	break-glass `deploy.yml`; each cloud's Alertmanager still alerts Slack
4	Backups run only while the platform is up (Jenkins cron)	no backups between sessions (nothing is written then either)	backup-on-demand job; take one before `down.yml` if data changed
5	`notification-service` keeps Socket.IO chat state in memory	runs as 1 replica per cloud; a restart drops open chat sessions	—
6	Waking a standby takes minutes (RTO 3 min 42 s, Azure ≈ 6 min)	a GCP outage is visible to shoppers for a few minutes	deliberate cost trade-off (−60 % node cost)
7	Failover Worker uses a fine-grained **personal access token**	a long-lived secret (scoped to one repo, `actions:write`)	stored only in GCP Secret Manager and as a Worker secret
8	No Cloudflare WAF custom rules / rate limiting	only Cloudflare's built-in DDoS protection	origins accept only Cloudflare IPs
9	Standby sleep issues found in the final test	AWS sleep can take 30 min (PDB drain); stopping an AKS that is still starting fails	manual commands in the runbook
10	Teardown can leave an AWS NLB behind	`down.yml` stops safely; manual cleanup needed	leftover check + runbook §7
11	Zero downtime proven on 29 Oct 2026, not in the final test	the final-test probe stopped before the deploy started	repeat the CI/CD test with the probe running for the whole deploy

Possible improvements

Priority	Item	Why
1	Fix standby sleep: drain AWS nodes with `--disable-eviction` and wait until instances are terminated; wait for an in-progress AKS start before stopping	correct "asleep" state, no hidden cost
2	Order the AWS add-on teardown so the	`down.yml` without manual cleanup

Priority	Item	Why
	LB Controller removes Envoy's NLB before it is uninstalled	
3	Preflight checks for the Atlas API key access list, the current IP and GitHub download reachability; correct per-cloud task labels	catch problems in seconds instead of mid-run
4	Terraform plan on every pull request and apply after approval via GitHub Actions (identities exist)	full GitOps for infrastructure
5	Replace the PAT with a GitHub App installation token	no long-lived personal token
6	Atlas on a dedicated tier with multi-region / multi-cloud nodes	live database replication across clouds
7	Move Jenkins (or a second runner) and Grafana out of the primary cloud	CD and central monitoring survive a GCP outage
8	Redis adapter for Socket.IO	notification-service can scale beyond 1 replica
9	Cloudflare WAF managed rules and rate limiting (paid plan)	application-layer protection